
Reinforcement Learning for Quantitative Trading: From High-Frequency Trading to Formulaic Alpha Mining



Molei Qin

College of Computing and Data Science

A thesis submitted to the Nanyang Technological University
in partial fulfillment of the requirements for the degree of
Doctor of Philosophy

2026

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research, is free of plagiarised materials, and has not been submitted for a higher degree to any other University or Institution.

Disclosure on AI Use. I used Claude Fable 5 through Claude Code and GPT-5.6-sol through Codex to polish the prose by editing the corresponding $\text{\LaTeX}$ source files, review equation formatting, and cross-check the thesis against publicly available source code. I confirm that I have adhered to NTU’s Policy on the Use of Generative AI in Research. I reviewed and edited all tool-assisted changes and accept full responsibility for the content of this thesis.

May 2026

.....

Date

NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU

Molei Qin

.....

Molei Qin

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice.

May 2026

.....

Date

NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
.....

Prof. BO AN

Authorship Attribution Statement

This thesis contains material from three research papers of which I am the first author. Two of these papers have been accepted for publication at peer-reviewed international conferences; the third is currently under review.

The technical chapters of this thesis are based on the following papers, with the author list and per-author contributions given in full for each.

Chapter 4 is published as Molei Qin, Shuo Sun, Wentao Zhang, Haochong Xia, Xinrun Wang, and Bo An, “EarnHFT: Efficient Hierarchical Reinforcement Learning for High Frequency Trading,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI’24)*, vol. 38, no. 13, pp. 14669–14676, 2024. Molei Qin and Shuo Sun are designated as equal contributors, and Xinrun Wang and Bo An as corresponding authors.

The contributions of the co-authors are as follows:

- Prof. Bo An proposed the project direction and revised the manuscript.
- Shuo Sun provided guidance on the problem formulation and revised the manuscript.
- I proposed the algorithm, designed and conducted the experiments, and wrote the manuscript.
- Wentao Zhang and Haochong Xia helped with the implementation and the experiments.
- Xinrun Wang provided insightful comments and revised the manuscript.

Chapter 5 has been published as Molei Qin, Xinyu Cai, Yewen Li, Haochong Xia, Chuqiao Zong, Shuo Sun, Xinrun Wang, and Bo An, “FineFT: Efficient and Risk-Aware Ensemble Reinforcement Learning for Futures Trading,” in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Volume 1 (KDD’26)*, pp. 1194–1203, 2026, doi:10.1145/3770854.3780187. Xinrun Wang is designated as the corresponding author.

The contributions of the co-authors are as follows:

- Prof. Bo An proposed the project direction and revised the manuscript.
- I proposed the problem, designed the algorithm, conducted the experiments, and wrote the manuscript.

- Xinyu Cai and Yewen Li helped with the implementation and the baseline experiments.
- Haochong Xia and Chuqiao Zong helped with the experiments and revised the manuscript.
- Shuo Sun and Xinrun Wang provided insightful comments and carefully revised the manuscript.

Chapter 6 is based on Molei Qin, Meiyong Zhu, Chuqiao Zong, and Bo An, “AlphaQD: Quality-Diversity Reinforcement Learning for Formulaic Alpha Mining,” a manuscript currently under review for the *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD’27, Cycle 1)*.

The contributions of the co-authors are as follows:

- Prof. Bo An proposed the project direction and revised the manuscript.
- I proposed the problem and the method, designed and conducted the experiments, and wrote the manuscript.
- Meiyong Zhu ran the baseline experiments.
- Chuqiao Zong helped with the implementation and the experiment scripts.

May 2026

.....

Date

NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
NTU NTU NTU NTU NTU NTU NTU NTU
.....

Molei Qin

Molei Qin

Acknowledgements

As my PhD journey at Nanyang Technological University comes to a close, I would like to express my sincere gratitude to the people who have taught, helped and supported me throughout these years. The time of my candidature has been an unforgettable period of growth, exploration and collaboration, and I am fortunate to have worked with people who have consistently pushed me to be more curious, more rigorous and more honest about my own ideas.

First and foremost, I would like to express my deepest gratitude to my supervisor, **Prof. Bo An**, for offering me the opportunity to pursue my PhD under his supervision. Prof. An's patience, sharp judgement and high standards have been invaluable in shaping how I think about research. He has taught me to look for the simplest structure behind a difficult problem, to insist on solid motivation and real-world impact, and to write and present my work with clarity. As my research role model, I have learned from him not only how to conduct research but also a sense of responsibility, professionalism and persistence that I hope to carry into the rest of my career. I am grateful for his guidance throughout this journey and for the AMI research family that he has built and continues to nurture at NTU.

I would also like to thank my Thesis Advisory Committee members, Assoc. Prof. **Long Cheng** (College of Computing and Data Science) and Assoc. Prof. **Tao Chen** (Nanyang Business School, Division of Banking and Finance), for the time, the feedback and the perspective they have generously offered during the milestones of this candidature. Their questions and suggestions helped me sharpen the scope of the thesis and strengthen the technical depth of my work.

This thesis is the result of close collaboration, and I would like to acknowledge the co-authors who shaped the three works it is built on. I owe a particular debt to **Shuo Sun**, my senior in the AMI group, who first introduced me to reinforcement learning for quantitative trading, set an example of rigorous and deployment-minded research, and was a co-author on both EarnHFT and FineFT. I am also

deeply grateful to **Prof. Xinrun Wang**, whose insight and steady mentorship have been a constant throughout the projects that make up this thesis. For EarnHFT, I thank **Wentao Zhang** and **Haochong Xia** for their contributions to the framework, the implementation and the experiments. For FineFT, I thank **Xinyu Cai**, **Yewen Li**, **Haochong Xia** and **Chuqiao Zong** for the technical discussions, the implementation help and the careful reading of the manuscript at every stage. Working with each of them has made the research substantially better than I could have made it alone.

I would also like to thank the broader members of the AMI group at NTU—Xinrun Wang, Rundong Wang, Wei Qiu, Jakub Cerny, Renchunzi Xie, Yanchen Deng, Shuxin Li, Wanqi Xue, Xinyu Cai, Shuo Sun, Pengdeng Li, Hongxin Wei, Pengjie Gu, Yewen Li, Chaojie Wang, Zhenghai Xue, Longtao Zheng, Chuqiao Zong, Sheng Zang, Ying Zheng, Dong Xing, Shanqi Liu, Wentao Zhang, Weihao Tan, Ruhao Jiang, Yuzhou Cao, Zhenxing Ge, Penghui Yang, Qi Wei, Jiacheng Xu, Fuxiang Zhang, Shengming Wang, Jianing Ju, Haochong Xia and Yao Long Teng—for the daily exchange that sustains research: the late-night deadlines, the rebuttal sessions, the group dinners and the everyday discussions in the office. It has been an amazing experience to work in this group, and I have learned a great deal from each of you.

Finally, but most importantly, I would like to thank my parents, my family and my close friends for their unwavering support, patience and love throughout this journey. Their encouragement, from near and far, has carried me through the difficult moments; without it, this thesis would not have been possible. This thesis is dedicated to them.

Abstract

Financial markets attract vast pools of capital worldwide. Quantitative trading converts market data into trading decisions—at heart, sequential decision making under uncertainty, for which reinforcement learning (RL), optimising a long-horizon return rather than a one-step prediction, is a natural tool. Markets, however, violate the assumptions on which standard RL rests: distributions drift, rewards have low signal-to-noise ratios, decision horizons can be extremely long, and errors can have asymmetric financial consequences. This thesis studies that promise–deployment gap through three tasks spanning both frequency and function: second-level position control for a single spot asset, five-minute trading of leveraged cryptocurrency futures, and daily formulaic alpha mining over a full cross-section of equities. For each task we identify the main obstacle to off-the-shelf RL and propose a method that addresses it.

First, we study second-level high-frequency trading, where an agent faces horizons on the order of one million steps and rapid regime changes challenge a monolithic policy. We propose EarnHFT, a three-stage hierarchical RL framework: (i) a hindsight, sample-path Q-teacher computed by dynamic programming over replayed future prices to provide auxiliary supervision for second-level training; (ii) a pool of trend-specialised agents trained under different preference parameters and selected by profitability; and (iii) a minute-level router, trained offline and frozen at deployment, that picks a specialist as conditions change. In the reported single-seed historical replays of four cryptocurrency markets, EarnHFT attains the highest total return in every dataset.

Second, we investigate leveraged futures trading on five-minute bars, where leverage magnifies the profit-and-loss consequences of noisy returns and unfamiliar states can precipitate liquidation. We propose FineFT, a three-stage ensemble RL framework: (i) a selective-update mechanism, driven by an ensemble temporal-difference matrix, that trains diverse specialists efficiently and is intended to encourage differentiated specialists; (ii) one variational autoencoder per inferred market dynamic

to provide a reconstruction-based familiarity score; and (iii) a risk-aware heuristic router that switches between specialists and a conservative fallback using recent VAE losses. Across the four reported markets, FineFT has the highest average total return among the learned methods and the lowest unfiltered average maximum drawdown, although it is not uniformly dominant on every individual market.

Third, turning from trading to signal generation, we study formulaic alpha mining, where a dominant family of pool-based RL miners couples each candidate’s reward to an evolving pool, creating a moving target. We propose AlphaQD, a three-stage quality-diversity RL method: (i) a relational-graph GFlowNet generator trained by trajectory balance against an archive-independent terminal reward; (ii) a Return-Source MAP-Elites archive whose behaviour descriptor is the normalised daily information-coefficient series of each formula; and (iii) a top- k cell-selection rule that exports one high-quality representative per selected cell as the deployed pool. The method is evaluated on full-market Chinese A-share and U.S. panels; on A-shares it attains the highest annualised return among the learned formula-mining methods, while the U.S. results are reported as a cross-market robustness check rather than evidence of causal alpha attribution.

The three studies share a broader design move: structure a heterogeneous task into distinguishable regions or behaviours, construct candidates specialised to those regions, and aggregate them through a task-appropriate mechanism. The objects differ materially—market-trend policies in EarnHFT, selectively updated learners in FineFT, and formula elites organised by return-source behaviour in AlphaQD—so the pattern is a comparative perspective, not a claim that all three decompose the market into stationary MDPs. Across the studies, aggregation is progressively separated from the generator’s learning reward: an offline-trained RL router in EarnHFT, a reconstruction-based heuristic router in FineFT, and a quality-diversity archive followed by a ridge combiner in AlphaQD.

Contents

Acknowledgements	ix
Abstract	xi
List of Figures	xvii
List of Tables	xix
List of Algorithms	xxi
Symbols and Acronyms	xxiii
I Foundations	1
1 Introduction	3
1.1 Quantitative Trading and Reinforcement Learning	3
1.2 Why Reinforcement Learning is Hard in Financial Markets	4
1.3 Tasks, Challenges, and Solutions	5
1.4 A Shared Design Perspective	8
1.5 Contributions	9
1.6 Organisation of the Thesis	10
2 Preliminaries	11
2.1 Financial Markets and Quantitative Trading	11
2.1.1 Instruments and Market Microstructure	11
2.1.2 The Trading-Frequency Spectrum	13
2.1.3 Evaluation Metrics	14
2.2 Reinforcement Learning Foundations	15
2.2.1 Markov Decision Processes	15
2.2.2 Value-Based Methods	16
2.2.3 Policy-Based Methods	16
2.2.4 Distributional and Ensemble RL	17
2.2.5 Hierarchical Reinforcement Learning	17

2.2.6	Generative Flow Networks	17
2.2.7	Quality-Diversity Search and MAP-Elites	18
2.2.8	Variational Autoencoders and OOD Detection	18
3	Literature Review	19
3.1	Reinforcement Learning for Quantitative Trading	19
3.1.1	Single-Asset and Intraday Trading	20
3.1.2	High-Frequency Trading	20
3.1.3	Portfolio Management and Asset Allocation	20
3.1.4	Order Execution	21
3.1.5	Hierarchical and Routing Approaches	21
3.1.6	Formulaic Alpha Mining	22
3.1.7	Uncertainty and Non-Stationary RL	22
3.1.8	Further Related Work	23
3.2	Open Challenges	24
II	Reinforcement Learning for Trading	25
4	EarnHFT: Hierarchical RL for HFT	27
4.1	Introduction	27
4.2	Problem Formulation	28
4.2.1	A Hierarchical Markov Decision Process	30
4.3	Method	31
4.3.1	Stage I: Efficient Reinforcement Learning with a Q-Teacher	31
4.3.2	Stage II: Construction of the Agent Pool	37
4.3.3	Stage III: Dynamic Routing	39
4.4	Experimental Setup	39
4.5	Results and Analysis	41
4.5.1	Comparison with Baselines	41
4.5.2	Effectiveness of the Hierarchical Framework	43
4.5.3	Ablation of the Hindsight Teacher	43
4.5.4	Trading Behaviour and Computational Cost	45
4.6	Chapter Summary	45
5	FineFT: Risk-Aware Ensemble Reinforcement Learning for Futures Trading	47
5.1	Introduction	47
5.2	Problem Formulation	50
5.2.1	A Dynamic-Parameter Markov Decision Process	50
5.3	Method	51
5.3.1	Stage I: Efficient Ensemble with Selective Update	51
5.3.2	Stage II: Ensemble Filtering and Familiarity Models	55
5.3.3	Stage III: Risk-Aware Heuristic Routing	56

5.4	Experimental Setup	57
5.5	Results and Analysis	59
5.5.1	Comparison with Baselines	59
5.5.2	Effectiveness of the Selective-Update Ensemble	63
5.5.3	Effectiveness of Risk-Aware Routing and Case Studies	63
5.5.4	Parameter Sensitivity and Computational Cost	64
5.5.5	Generalisation to Other Asset Classes	66
5.6	Chapter Summary	68
 III Reinforcement Learning for Signal Generation		69
6	AlphaQD: Quality-Diversity Reinforcement Learning for Formulaic Alpha Mining	71
6.1	Introduction	71
6.2	Problem Formulation	74
6.3	Method	75
6.3.1	Efficient GFlowNet Exploration	75
6.3.2	Return-Source MAP-Elites Archiving	77
6.3.3	Top- K Cell Selection	79
6.4	Generalisation Properties	79
6.4.1	Decoupling Quality from Shape	81
6.4.2	Reward Stationarity	81
6.4.3	Search Complexity and a Boosting Counterexample	82
6.4.4	Per-Cell Selection Bias	84
6.4.5	Ensemble ICIR Lower Bound	85
6.5	Experimental Setup	86
6.6	Results and Analysis	88
6.6.1	Main Results	88
6.6.2	Archive Diagnostics	90
6.6.3	Ablations	91
6.7	Chapter Summary	93
 IV Epilogue		95
7	Conclusion and Future Work	97
7.1	Summary of Contributions	97
7.2	The Unifying Perspective Revisited	98
7.3	Limitations	99
7.4	Future Work	100
 List of Author's Publications		103
 Bibliography		105

List of Figures

2.1	A limit order book snapshot.	12
4.1	The EarnHFT framework.	29
4.2	An EarnHFT trading episode.	41
4.3	Router versus specialist pool.	43
4.4	Router selection distribution.	44
5.1	Two challenges of RL in futures trading.	48
5.2	The FineFT framework.	49
5.3	Routing proportions on BTC around the validation–test boundary.	64
5.4	Update-index clustering under selective update.	65
5.5	Out-of-distribution detection with a VAE.	65
6.1	Reward drift under two reward functions.	72
6.2	The AlphaQD pipeline.	73
6.3	Cumulative NAV on the A-share test window.	90
6.4	The AlphaQD MAP-Elites archive.	90

List of Tables

1.1	Overview of the three studies.	6
1.2	Thesis spine: structure, candidate, aggregation.	7
4.1	EarnHFT dataset statistics.	40
4.2	EarnHFT main results.	42
4.3	Ablation of the Q-teacher.	44
5.1	FineFT dataset statistics.	58
5.2	FineFT main results.	61
5.3	Ablation of the selective-update ensemble.	63
5.4	FineFT hyperparameter sensitivity.	66
5.5	Reported FineFT results on two other asset classes.	67
6.1	AlphaQD main results.	89
6.2	Full AlphaQD ablation grid.	92

List of Algorithms

1	Construction of the Hindsight Teacher Values	34
2	Efficient Reinforcement Learning with a Q-Teacher	35
3	Market Segmentation and Labelling	38
4	Local Feasible-Transition Action-Value Teacher	53
5	Construction of the Weight Matrix	55
6	Risk-Aware Heuristic Routing	58
7	AlphaQD: three-stage training and export	80

Symbols and Acronyms

Symbols

$\mathcal{S}, \mathcal{A}$	the state space and the action space of an MDP
π_θ	a policy parameterised by θ
$Q(s, a)$	the action-value (Q) function
$Q^*(s, a)$	the Bellman-optimal action-value function; Chapters 4–5 retain uppercase $Q^*[t, p, a]$ for a trajectory-conditioned hindsight teacher table
γ	the discount factor
r_t	the reward received at time t
V_t	the account value at time t : spot net value or futures margin balance, according to context
H_t, L_t	the position and the leverage at time t
$\mathcal{H}(\cdot)$	the Huber loss
$\text{KL}(\cdot \parallel \cdot)$	the Kullback–Leibler divergence
$\text{softmax}(\cdot)$	action-wise softmax (unit temperature unless stated otherwise)
α_{KL}	the scheduled coefficient of the KL supervision term (EarnHFT, FineFT)
α	a formulaic alpha (symbolic expression)
$\text{IC}_t(\alpha)$	the daily cross-sectional information coefficient at time t
$\text{ICIR}(\alpha)$	the information ratio of the IC series
$\mathbf{r}(\alpha)$	the centred, approximately unit-normalised IC series (return-source descriptor)
β	the trend-preference sampling coefficient (EarnHFT)
β_q	the reward-temperature coefficient (AlphaQD)

Acronyms

RL	Reinforcement Learning
HFT	High-Frequency Trading
MDP	Markov Decision Process
DP-MDP	Dynamic-Parameter Markov Decision Process
HRL	Hierarchical Reinforcement Learning
DQN	Deep Q-Network
DDQN	Double Deep Q-Network
PPO	Proximal Policy Optimisation
TD	Temporal Difference
ETD	Ensemble Temporal Difference
MoE	Mixture of Experts
VAE	Variational Autoencoder
OOD	Out-of-Distribution
ID	In-Distribution
QD	Quality-Diversity
GFlowNet	Generative Flow Network
TB	Trajectory Balance
RGCN	Relational Graph Convolutional Network
MAP-Elites	Multi-dimensional Archive of Phenotypic Elites
PFS	Perturbation Fidelity Score
PCA	Principal Component Analysis
RPN	Reverse Polish Notation
GP	Genetic Programming
DTW	Dynamic Time Warping
LOB	Limit Order Book
OHLC	Open-High-Low-Close
IC	Information Coefficient
ICIR	Information-Coefficient Information Ratio
MACD	Moving Average Convergence Divergence
IV	Five-level order-book volume imbalance (legacy implementation label)
TR	Total Return
ASR	Annualised Sharpe Ratio
ACR	Annualised Calmar Ratio

ASoR	Annualised Sortino Ratio
MDD	Maximum Drawdown
AVOL	Annualised Volatility

Part I

Foundations

Chapter 1

Introduction

1.1 Quantitative Trading and Reinforcement Learning

Quantitative trading is the discipline of converting market data into trading decisions through explicit, data-driven rules. A quantitative strategy ingests a stream of prices, volumes, and order-flow signals, and at each decision point it must choose what to hold and how to adjust that holding. Stated this way, quantitative trading is, at its core, a problem of *sequential decision-making under uncertainty*: the present action changes the portfolio, the portfolio interacts with a stochastic, non-stationary market—one often informally described as adversarial, although throughout this thesis the market is modelled as an exogenous process replayed from historical data—and the consequences are realised only over a horizon of subsequent steps. The natural formal language for such problems is reinforcement learning (RL), in which an agent learns a policy that maps states to actions so as to maximise a cumulative reward [1].

Reinforcement learning is attractive for trading for reasons that distinguish it from a one-step predictive pipeline. The supervised baselines used in this thesis predict a future return or direction and optimise a predictive loss; sophisticated supervised time-series models need not assume independent and identically distributed (IID) data, but their loss still does not by itself specify how predictions become positions under costs and constraints. Transaction costs and inventory make the payoff of an

action depend on the current position, coupling consecutive decisions. Reinforcement learning offers one way to model that path dependence and optimise a trajectory return directly, although it is not automatically preferable to a well-designed predictive strategy. Researchers have applied it across the quantitative-trading stack, from the direct-reinforcement systems of Moody and Saffell [2] through order execution [3], intraday trading [4–6], and portfolio management [7].

The premise of this thesis is therefore that reinforcement learning is a useful framework for the sequential trading problems studied here; its subject is the gap between that premise and practice. Financial markets violate, sometimes severely, the assumptions on which standard RL algorithms were designed and validated. The contribution of this thesis is to isolate, for several distinct and economically important trading tasks, the obstacle treated as decisive in each stated setting and to design a method that mitigates or works around it, rather than claiming one remedy for all financial reinforcement learning.

1.2 Why Reinforcement Learning is Hard in Financial Markets

The difficulties are clearest when each property of the data-generating and trading process is connected to its consequence for learning. The technical chapters then address different parts of this chain rather than three interchangeable versions of the same problem.

- **Distribution drift and finite history.** Relationships between features, actions, and returns change as market conditions change, while only a finite historical window is available for fitting. The immediate learning consequence is train–deployment mismatch: arguments derived for stationary Markov decision processes (MDPs) need not transfer to the future, and model or selection complexity can turn repeated backtesting into overfitting [8].
- **Weak feedback over long horizons.** The predictable component of returns is small relative to price noise, and at second resolution an evaluation period can contain millions of decisions. The resulting reward signal is noisy and temporally diffuse, which lengthens credit assignment and makes training

and evaluation expensive. Long-horizon financial RL also faces the out-of-sample generalisation difficulty documented in trade execution [9].

- **Path-dependent and asymmetric payoffs.** Positions, transaction costs, funding, and margin rules couple successive decisions. Leverage does not make the underlying price process intrinsically more uncertain, but it magnifies profit-and-loss variation and makes errors asymmetric: an unfamiliar state can lead to liquidation before an expected-return learner recovers. The learning problem therefore requires risk controls and careful account-state modelling, not only a larger return objective.
- **Endogenous learning targets.** When a candidate is rewarded by its marginal contribution to an evolving set of strategies or signals, changing that set changes the reward attached to the same candidate. This creates objective drift inside the algorithm even if the historical data window is held fixed, complicating optimisation and post-selection analysis.

No single method removes all of these obstacles at once, nor should one expect it to: the obstacles interact, and the binding constraint differs by task. This thesis instead studies three concrete tasks, isolates the obstacle that is decisive for each, and addresses it.

1.3 Tasks, Challenges, and Solutions

The three studies that constitute this thesis are organised along the spectrum of trading frequency and function, from second-level position control to daily signal generation. EarnHFT is a trading-timing and position-control method, not an optimal-order-execution method. Table 1.1 summarises the studies; the remainder of this section introduces each in turn.

Table 1.2 makes the thesis spine explicit by tracing, across the three studies, what is decomposed, what is specialised, how the specialists are aggregated, and how directly the aggregation mechanism is driven by a learning reward.

TABLE 1.1: The three tasks studied in this thesis, the principal obstacle each presents to standard reinforcement learning, and the method developed in response. The tasks span second-level position control, five-minute leveraged trading, and daily signal generation.

Chapter	Task	Decisive challenge	Method
Chapter 4 (Earn-HFT)	Second-level spot high-frequency trading	Extremely long horizon ($\sim 10^6$ steps) limits data efficiency; regime changes challenge a single policy	Three-stage <i>hierarchical</i> RL: hindsight sample-path Q-teacher, a pool of trend specialists, and a minute-level router
Chapter 5 (FineFT)	Five-minute leveraged futures trading	Leverage magnifies profit-and-loss variation and unfamiliar states can trigger liquidation	Three-stage <i>ensemble</i> RL: selective-update specialists, per-dynamic VAE familiarity scores, and risk-aware routing with a conservative fallback
Chapter 6 (Al-phaQD)	Daily formulaic alpha mining over a full cross-section	Pool-coupled rewards create an endogenous moving target	Three-stage <i>quality-diversity</i> RL: GFlowNet exploration under an archive-independent reward, MAP-Elites archiving, and top- k cell export

Task I: High-frequency trading (Chapter 4). The first task is second-level high-frequency trading (HFT) of a single cryptocurrency, where the agent repeatedly adjusts its position to profit from short-lived price fluctuations. Two obstacles dominate: the decision horizon contains millions of steps, and one policy must span changing trends. Within the released historical-replay model, future prices are exogenous to the agent because the simulator has no market-impact response. Earn-HFT therefore computes a hindsight, sample-path action-value table by dynamic programming and uses its action distribution to supervise the second-level learner. It then trains a pool of trend-conditioned agents and an offline minute-level router that is frozen for deployment. In the reported single-seed experiments, EarnHFT attains the highest total return on each of the four datasets; this is evidence from

TABLE 1.2: A comparative spine across the three studies. Each method structures heterogeneity, constructs candidates suited to the resulting regions, and aggregates them, but the regions and candidates are not mathematically identical: AlphaQD partitions formula behaviour rather than market regimes, and its elites are formulas rather than RL learners.

Chapter	Structuring criterion	Candidate	Aggregation mechanism	Coupling to learning reward
Chapter 4 (Earn-HFT)	Market trend (slope-based segmentation)	DQN-style specialist indexed by trend and initial position	Minute-level DQN-style router trained offline by RL on historical data and frozen for deployment	<i>Tightly coupled:</i> the aggregation mechanism is itself a reward-maximising policy
Chapter 5 (FineFT)	Latent dynamic of a dynamic-parameter MDP	Selectively-updated learner per dynamic	Heuristic router driven by per-dynamic VAE losses and a conservative fallback	<i>Partly decoupled:</i> selection uses a generative familiarity score, not a learned policy
Chapter 6 (AlphaQD)	Return-source behaviour (normalised daily IC series)	Per-cell elite of a MAP-Elites archive	Static top- k cell export with a shared ridge combiner	<i>Fully decoupled:</i> the generator's reward never reads the archive, and the archive never edits the reward

those replays, not a claim of universal optimality or market-impact-aware execution.

Task II: Leveraged futures (Chapter 5). The second task moves from spot to leveraged futures, introducing long and short positions, funding, margin constraints, and high leverage. Leverage leaves the stochasticity of the underlying price process unchanged but magnifies the consequences for profit and loss; it can also turn an error in an unfamiliar state into liquidation. FineFT uses selective updates to produce differentiated ensemble members, then fits one variational autoencoder to the states associated with each inferred dynamic. Reconstruction losses serve as heuristic familiarity scores, not calibrated competence boundaries. At deployment, a rule-based router uses those scores to select a learner or a conservative fallback. Across the four reported markets, FineFT has the highest average total

return among the learned methods and the lowest unfiltered average maximum drawdown, but it is not uniformly dominant on every market and the experiments do not isolate the router as the sole cause of the gain.

Task III: Formulaic alpha mining (Chapter 6). The third task no longer chooses positions; it generates short symbolic predictive signals, or “alphas”, for a downstream portfolio. A dominant pool-based line rewards a new formula by its marginal contribution to an evolving pool, so the same formula’s target can change as the pool changes. AlphaQD removes that coupling: a graph-based GFlowNet receives a candidate-only training-window reward, while a separate MAP-Elites archive organises formulas by the shape of their normalised daily information-coefficient series. The resulting archive-conditioned selection analysis is explicitly conditional on the realised archive and its stated assumptions; it is not a general finite-sample guarantee for the fitted ridge portfolio. The method is evaluated on Chinese A-share and U.S. panels, with its strongest portfolio result on A-shares.

1.4 A Shared Design Perspective

Although the three studies were motivated by different tasks and developed independently, comparing them reveals a useful design perspective. It is deliberately broader than a common mathematical model:

1. **Structure** the relevant heterogeneity: market trends in EarnHFT, transition subsets inferred from ensemble TD errors in FineFT, and return-source behaviours in AlphaQD.
2. **Construct differentiated candidates:** trading policies in the first two studies and elite formulas in the third.
3. **Aggregate** those candidates through a task-appropriate mechanism: routing to one policy or combining several signals.

This comparison does not imply that AlphaQD partitions a non-stationary market, that an elite formula is an RL specialist, or that each region is stationary. The narrower connective thread concerns where aggregation information enters learning.

EarnHFT trains its router after freezing the specialists; FineFT uses a non-learned rule over reconstruction scores; AlphaQD keeps archive membership out of the generator’s reward. Joint learning can be appropriate in other settings, so the thesis advances the following as a falsifiable working hypothesis rather than a principle established by these three non-controlled studies:

When aggregation would make a candidate’s target change as other candidates enter or leave the set, separating the aggregation criterion from that candidate’s learning reward can improve target stability and make selection assumptions easier to state.

1.5 Contributions

The contributions of this thesis are as follows.

- **EarnHFT** (Chapter 4): a hierarchical reinforcement-learning framework for single-asset high-frequency position control. Its technical core is a hindsight, sample-path Q-teacher computed by dynamic programming under exogenous historical replay; together with trend-conditioned training and an offline minute-level router, it attains the highest total return in each of the four reported single-seed datasets.
- **FineFT** (Chapter 5): a risk-aware ensemble framework for leveraged futures. It introduces a selective-update mechanism driven by an ensemble TD matrix and uses per-dynamic VAE reconstruction losses as heuristic familiarity scores for rule-based routing. Across the four reported markets, it has the highest average total return among learned methods and the lowest unfiltered average maximum drawdown.
- **AlphaQD** (Chapter 6): a quality-diversity reinforcement-learning method for formulaic alpha mining. It separates a candidate-only GFlowNet reward from a MAP-Elites archive keyed on a return-source descriptor. The chapter proves an abstract equal-weight ICIR lemma and gives a separate finite-class selection-complexity comparison conditional on a realised archive; neither result is presented as a finite-sample guarantee for the fitted ridge portfolio.

- **A comparative perspective** (Chapter 1, Chapter 7): the observation that all three methods structure heterogeneity, construct differentiated candidates, and aggregate them, together with the narrower working hypothesis that aggregation information should be separated from a candidate’s reward when it would otherwise make that target evolve.

1.6 Organisation of the Thesis

The thesis is organised into four parts. **Part I** (Foundations) comprises this introduction, Chapter 2, which establishes the shared financial and reinforcement-learning background of the technical chapters, and Chapter 3, which surveys related work organised by task and synthesises the open challenges that motivate them. **Part II** (Reinforcement Learning for Trading) contains the two trading studies: Chapter 4 presents EarnHFT for high-frequency trading, and Chapter 5 presents FineFT for leveraged futures. **Part III** (Reinforcement Learning for Signal Generation) contains Chapter 6, which presents AlphaQD for formulaic alpha mining. **Part IV** (Epilogue) consists of Chapter 7, which revisits the unifying perspective, states the limitations of each study, and outlines directions for future work. Supporting proofs and extended derivations are presented within the technical chapters in which they arise.

Chapter 2

Preliminaries

This chapter introduces the shared vocabulary used throughout the thesis. Section 2.1 introduces the financial instruments, market microstructure, and evaluation metrics common to all three studies; these definitions provide the baseline, and later chapters restate them only where study-specific notation or an implementation-specific convention requires it. Section 2.2 is a brief primer on the reinforcement-learning machinery—from Markov decision processes to GFlowNets and quality-diversity search—used across the thesis. The survey of related work and the open challenges that motivate the technical chapters are deferred to Chapter 3.

2.1 Financial Markets and Quantitative Trading

2.1.1 Instruments and Market Microstructure

The three studies operate on three different financial settings. EarnHFT (Chapter 4) trades a single *spot* cryptocurrency, where the asset is bought and sold outright and only long positions are taken. FineFT (Chapter 5) trades *perpetual futures*, standardised contracts that permit both long and short positions, magnify exposure through leverage, and exchange a periodic funding payment between the two sides of the contract; depending on the sign of the funding rate and the direction of the position, a trader may either pay or receive funding. AlphaQD (Chapter 6) operates on the *cross-section* of an entire equity market, where the

object of interest is the relative ranking of thousands of assets on each day rather than the timing of a single instrument. The following concepts recur throughout.

Limit order book (LOB). The limit order book aggregates all resting buy and sell orders by price level and is the standard description of market microstructure [10]. A d -level LOB at time t is denoted $B_t = \{(p_t^{b_i}, q_t^{b_i}, p_t^{a_i}, q_t^{a_i})\}_{i=1}^d$, where $p_t^{b_i}$ and $p_t^{a_i}$ are the level- i bid and ask prices and $q_t^{b_i}, q_t^{a_i}$ are the corresponding quantities. Figure 2.1 illustrates a snapshot. A *market order* executes immediately against the resting orders on the opposite side, filling at the best quote first; only when its size exceeds the depth available at the best level does it *walk the book*, consuming successively deeper levels, so that its average execution price is worse than the best quote—an effect known as slippage that any high-fidelity simulator must model.

Level	Price	Quantity
Ask 5	105	90
Ask 4	104	62
Ask 3	103	45
Ask 2	102	28
Ask 1 (best ask)	101	10
mid-price = 100.5		
Bid 1 (best bid)	100	12
Bid 2	99	30
Bid 3	98	48
Bid 4	97	66
Bid 5	96	85

FIGURE 2.1: A schematic five-level limit order book snapshot with illustrative integer prices and quantities. Resting asks (sell orders) sit above the mid-price and resting bids (buy orders) below it, each aggregated by price level. A market order first consumes the depth at the best quote; if its size exceeds that depth, it walks the book *away* from the mid-price into successively deeper levels, so larger orders execute at progressively worse average prices.

OHLC and trades. Executed transactions are summarised over a time interval by the open, high, low, and close prices, denoted $x_t = (p_t^o, p_t^h, p_t^l, p_t^c)$. The raw stream of executed orders, or trades, records price, quantity, and side. For leveraged contracts, a *mark price* M_t is used as the reference fair value at which the position is valued and against which liquidation is assessed.

Technical indicators. Both microstructure and price-history features are summarised by technical indicators—deterministic functions of a trailing window of LOB and trade data designed to expose latent patterns. For a look-back length w , write

$$y_t = \phi(B_{t-w+1:t}, x_{t-w+1:t}).$$

One EarnHFT feature is the five-level order-book volume imbalance: total displayed bid depth minus total displayed ask depth, divided by their sum. The released baseline labels this quantity “IV”, although *volume imbalance* is the clearer name. Another example is moving-average convergence divergence (MACD) [11], which contrasts trends over different windows. These indicators are inputs to the RL state in the two trading studies (Chapter 4 and Chapter 5); the alpha-mining study of Chapter 6 instead composes its own signals directly from raw price–volume fields. As standalone rules, however, technical indicators are sensitive to hyperparameters and produce false signals in volatile markets.

Position, leverage, and value. A trader’s position H_t is the signed quantity held; under leverage L_t , the same capital supports a larger position and amplifies both gains and losses. For spot trading, the account value (net value) is the cash balance plus the position marked at the current price, net of realised costs such as commissions. For leveraged futures, the corresponding quantity is the *margin balance*: the wallet balance plus the unrealised profit-and-loss of open positions, net of commissions and funding payments. The full notional value of the position is exposure, not account equity.

2.1.2 The Trading-Frequency Spectrum

Trading strategies are conventionally classified by the frequency at which they act, and the binding difficulty changes along this spectrum. *High-frequency* strategies act at the second or sub-second scale and profit from fleeting microstructure imbalances; their defining computational challenge is the sheer length of the decision horizon. *Medium-frequency* strategies act at the minute-to-hour scale. *Low-frequency* strategies act daily or slower; here the difficulty is not the horizon but the discovery of robust predictive signals from a low signal-to-noise cross-section.

Acting frequency should not be conflated with other, logically independent dimensions of a trading problem: leverage, funding payments, and overnight exposure are properties of the instrument traded and of the holding period, not of the sampling rate itself, although longer holding periods tend to make them more consequential. In this thesis these dimensions become material in the leveraged perpetual-futures setting of Chapter 5. Note also that the 24/7 operation of cryptocurrency markets removes the closed-market gap between trading sessions, but not the risk of holding a position overnight. This thesis spans second-level position control in Chapter 4, five-minute leveraged trading in Chapter 5, and daily signal generation in Chapter 6.

2.1.3 Evaluation Metrics

Strategies are evaluated at two layers. The *portfolio layer* measures the realised profit-and-loss of an executed strategy; the *signal layer*, used for alpha mining, measures the predictive quality of a signal independently of any portfolio construction. Let $V_1, \dots, V_{n+1}$ denote account values at the sampling points used for a reported metric, and let $\mathbf{ret} = (\text{ret}_1, \dots, \text{ret}_n)$ with $\text{ret}_i = (V_{i+1} - V_i)/V_i$ denote the corresponding *simple* (not log) returns. The sampling interval is study-specific: EarnHFT uses second-level returns, with $m = 365 \times 24 \times 60 \times 60 = 31,536,000$ intervals per year, whereas FineFT first aggregates its 288 five-minute decisions into daily returns and uses $m = 365$. Thus m counts *metric-return intervals*, not necessarily policy decisions. The risk-free rate is taken to be zero, and $\mathbb{E}[\cdot]$ and $\sigma[\cdot]$ denote the empirical mean and standard deviation over the selected return series; the released metric code uses NumPy's population normalisation (`ddof=0`).

Portfolio-layer metrics. The following six metrics are used throughout Chapter 4 and Chapter 5, and the return/risk pair recurs in Chapter 6.

- **Total Return (TR):** the overall return over the trading period, $\text{TR} = (V_{n+1} - V_1)/V_1$, where V_1 and V_{n+1} are the initial and final account values.
- **Annualised Volatility (AVOL):** $\text{AVOL} = \sigma[\mathbf{ret}] \sqrt{m}$, a measure of risk.
- **Maximum Drawdown (MDD)** [12]: the largest peak-to-trough decline of the value curve, $\text{MDD} = \max_{1 \leq i \leq j \leq n+1} (V_i - V_j)/V_i$, capturing the worst case.

MDD is a positive magnitude—a fraction in this definition and a percentage in the result tables—so smaller is better (marked $\downarrow$).

- **Annualised Sharpe Ratio (ASR)** [13]: $\text{ASR} = \sqrt{m} \frac{\mathbb{E}[\text{ret}]}{\sigma[\text{ret}]}$, return per unit of total risk (with zero risk-free rate); the $\sqrt{m}$ factor multiplies the whole ratio, so ASR equals the annualised mean return $m \mathbb{E}[\text{ret}]$ divided by AVOL.
- **Annualised Calmar Ratio (ACR)** [14]: $\text{ACR} = m \mathbb{E}[\text{ret}]/\text{MDD}$, the implementation’s annualised-return-per-maximum-drawdown statistic.
- **Annualised Sortino Ratio (ASoR)** [15]: a common zero-threshold definition is $\text{ASoR} = \sqrt{m} \mathbb{E}[\text{ret}]/\text{DD}_0$, where $\text{DD}_0 = \sqrt{n^{-1} \sum_{i=1}^n \min(\text{ret}_i, 0)^2}$. EarnHFT and FineFT use different return frequencies, evaluation spans, and study-specific downside-risk conventions; their ASoR values are therefore interpreted within each study rather than compared across chapters.

Signal-layer metrics. For a cross-sectional signal α and an h -day forward return $r^{(h)}$, the daily *information coefficient* is the cross-sectional correlation $\text{IC}_t(\alpha) = \text{corr}(\alpha_t, r_t^{(h)})$. We call the ratio $\text{ICIR}(\alpha) = \overline{\text{IC}(\alpha)}/\sigma[\text{IC}(\alpha)]$ the *IC information ratio*; it combines predictive strength with temporal stability and is distinct from the portfolio information ratio based on active return and tracking error. Cross-sectional IC is standard in active management [16], and IC-based scoring is common in formulaic-alpha research [17]. The rank-based variant, *RankIC*, replaces Pearson with Spearman correlation and is robust to outliers.

2.2 Reinforcement Learning Foundations

This section is a brief primer, confined to the reinforcement-learning machinery the technical chapters actually use; a reader fluent in RL may skip directly to the task survey of Chapter 3, which carries the weight of the literature review.

2.2.1 Markov Decision Processes

A finite-horizon Markov decision process (MDP) is the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma, T)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ the action space, $P(\cdot \mid s, a)$ a probability distribution

over next states, r the reward function, $\gamma \in (0, 1]$ the discount factor, and T the number of decision steps. At step t the agent observes s_t , samples $a_t \sim \pi_\theta(\cdot \mid s_t)$, and receives r_t and s_{t+1} . The objective is $\theta^* = \arg \max_\theta \mathbb{E}_{s_0 \sim \rho_0, \pi_\theta} [\sum_{t=0}^{T-1} \gamma^t r_t]$, where ρ_0 is the initial-state distribution; the resulting policy is $\pi^* = \pi_{\theta^*}$. The classical MDP model treats P and r as stationary. Financial environments need not be, a difficulty made explicit in Chapter 5 by augmenting the tuple with a slowly evolving latent dynamic.

2.2.2 Value-Based Methods

Value-based methods learn action-value functions. For a policy π , $Q^\pi(s, a)$ is the expected return of taking a in s and following π thereafter; the *optimal* action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ is the expected return of taking a in s and acting optimally thereafter. Q-learning [18] learns an estimate Q of Q^* by updating it towards the temporal-difference (TD) target $r + \gamma \max_{a'} Q(s', a')$. The Deep Q-Network (DQN) [19] parameterises Q with a neural network and stabilises learning through experience replay and a target network. Double DQN (DDQN) [20] decouples action selection from evaluation to reduce the overestimation bias of the max operator. EarnHFT and FineFT instead use target-network value learners whose released updates take the next-action maximum directly over the target network; despite the original DDQN terminology, these are DQN-style targets rather than strict Double-DQN targets. Both methods add supervision from a hindsight dynamic-programming teacher, a design related in spirit to deep Q-learning from demonstrations [21]. The analysis in Chapter 4 gives a conditional contraction result for an idealised discounted tabular operator and explains why it is not a convergence guarantee for EarnHFT’s reported $\gamma = 1$ function-approximation learner.

2.2.3 Policy-Based Methods

Policy-based methods optimise the policy directly. Proximal Policy Optimisation (PPO) [22] maximises a clipped surrogate objective with importance sampling, trading some bias for stability and sample reuse. In trading, policy-based methods are prone to collapsing onto trivial policies—holding the current position—when

transaction costs make any action locally unprofitable, a failure mode documented empirically in Chapter 4.

2.2.4 Distributional and Ensemble RL

Two families address the high stochasticity of financial rewards. *Distributional* RL learns the full return distribution rather than its mean; quantile-regression DQN [23] and its risk-averse variants represent the return by a set of quantiles and can act on the lower tail to avoid worst cases. *Ensemble* RL trains several learners and aggregates them: Averaged-DQN [24] reduces target variance, while SUNRISE [25] treats inter-learner disagreement as a measure of uncertainty and down-weights uncertain samples. FineFT belongs to the ensemble family but departs from prior work by updating learners *selectively* rather than uniformly, with the aim of encouraging specialists to emerge.

2.2.5 Hierarchical Reinforcement Learning

Hierarchical RL decomposes a long-horizon task into a hierarchy of sub-tasks, typically a high-level policy that selects among low-level policies or temporally extended options [26, 27]. In trading, this can map onto a separation of time scales: a slow policy selects a strategy while fast policies control positions. Prior hierarchical trading frameworks address portfolio management and order execution [28] or select among strategies by market situation [29]. EarnHFT brings this idea to single-asset high-frequency trading, where the high-level router operates at the minute scale and the low-level specialists at the second scale.

2.2.6 Generative Flow Networks

Generative flow networks (GFlowNets) [30] learn a stochastic policy that constructs compositional objects—here, symbolic expressions—step by step. With positive terminal rewards and support over the reachable terminal objects, a global optimum of the trajectory-balance objective [31] induces $\Pr(X = x) = R(x)/Z$, where $Z = \sum_{x'} R(x')$. A finitely trained model only approximates this relation, but the reward-proportional target is mode-covering rather than concentrated solely on an

argmax. AlphaQD therefore adopts a relational-graph GFlowNet to search for multiple high-reward expressions under a fixed budget.

2.2.7 Quality-Diversity Search and MAP-Elites

Quality-diversity (QD) algorithms [32] seek not a single optimum but an archive of high-quality solutions that are diverse along a chosen *behaviour descriptor*. MAP-Elites [33] discretises the descriptor space into cells and retains the best solution(s) per cell, yielding a structured population that covers the behaviour space. AutoAlpha [34] previously used PCA-QD ideas to bias evolutionary search in alpha mining. AlphaQD instead maintains a separate MAP-Elites archive with bounded per-cell sets, using the normalised daily IC series as a return-source descriptor and keeping archive membership outside the generator’s reward.

2.2.8 Variational Autoencoders and OOD Detection

A variational autoencoder (VAE) [35] learns a probabilistic encoder that maps inputs to a regularised latent distribution and a decoder that reconstructs them. Reconstruction loss is often used as a heuristic score for out-of-distribution (OOD) detection, but deep generative models can assign misleading scores under distribution shift [36]; it is not a calibrated probability of being in distribution. FineFT trains a VAE per inferred market dynamic and uses its loss as a reconstruction-based familiarity score for routing. Such a score can flag distributional mismatch, but it is not a calibrated boundary of a specialist’s trading competence.

Chapter 3

Literature Review

This chapter surveys the prior work on which the three technical chapters build. With the financial and reinforcement-learning preliminaries of Chapter 2 in hand, Section 3.1 reviews reinforcement learning for quantitative trading organised by task—single-asset and intraday trading, high-frequency trading, portfolio management, order execution, strategy routing, and formulaic alpha mining—before turning to the cross-cutting question of uncertainty and non-stationarity and closing with further related work. Section 3.2 then synthesises the open challenges that motivate Chapter 4, Chapter 5, and Chapter 6.

3.1 Reinforcement Learning for Quantitative Trading

Reinforcement learning has been applied across the entire quantitative-trading stack, and we survey that body of work organised by task. For each task we note the dominant design and the assumption it leaves unaddressed, which the open challenges of Section 3.2 then collect. Open frameworks and benchmarks have standardised much of this research—FinRL [37], TradeMaster [38], Qlib [39], and the PRUDEX evaluation compass [40]—and the studies in this thesis follow their evaluation conventions where applicable.

3.1.1 Single-Asset and Intraday Trading

The earliest and largest body of RL-for-trading work targets a single asset at the daily-to-intraday scale, where the agent maps a window of price and technical features to a discrete trading action. Foundational studies established deep RL as a viable trader on this task [41–43], and later work improved robustness and sample efficiency through imitation of profitable behaviour [44, 45], multimodal or convolutional state encoders [46], reward shaping for sparse-reward environments [47], and multi-objective formulations for intraday trading of stock-index futures [48]. DeepScalper [4] adds a hindsight bonus and an auxiliary price-prediction task to improve generalisation in intraday trading and remains a strong representative of the single-agent design. CDQNR [6], sometimes reused as a baseline in HFT studies, evaluates a perturbed convolutional DQN at daily frequency and belongs to this adjacent single-asset group rather than to HFT proper. The common thread is a single, monolithic policy: effective at its target frequency, but not by itself designed to address the horizon growth of second-level trading or the regime non-stationarity that erodes one policy over long deployments.

3.1.2 High-Frequency Trading

For high-frequency trading specifically, DRA [5] couples an LSTM [49] state encoder with PPO, and asynchronous deep double-duelling Q-learning has been applied to rapid trading-signal decisions [50]. A common non-learned alternative is rule-based technical analysis, built on signals such as MACD [11] and the five-level order-book volume-imbalance rule described in Section 2.1.1. EarnHFT addresses second-level position control through hierarchy; FineFT is a separate five-minute leveraged-futures study rather than an HFT method.

3.1.3 Portfolio Management and Asset Allocation

Beyond single-asset timing, a substantial line applies RL to portfolio management, where the action is a continuous allocation across many assets. Early frameworks cast allocation as a deterministic policy-gradient problem over price tensors [7, 51]; subsequent work added explicit risk–return balancing [52], ensemble strategies that

switch among actor–critic learners [53], hierarchical decomposition of selection, allocation, and commission-aware trade execution [28, 54, 55], and maskable representations that respect tradability constraints [56]. Margin Trader [57] extends the setting to leverage and short selling—the regime that Chapter 5 studies. Portfolio management is not the subject of this thesis, but it shares the central difficulty of non-stationarity and motivates much of the ensemble and hierarchical machinery reused here.

3.1.4 Order Execution

A complementary task is optimal order execution—acquiring or liquidating a large position while minimising market impact. Recent RL work on execution stresses generalisation across instruments and regimes [9], and the adjacent microstructure task of automated market making has been tackled with imitative RL over predictive state representations [58]. Execution is the task in which EarnHFT’s historical-replay assumption—that simulated actions do not alter the recorded price path (Section 4.3.1)—is most visibly an approximation, and the literature here is correspondingly attentive to market impact.

3.1.5 Hierarchical and Routing Approaches

A growing line of work does not commit to one policy but routes among several. MetaTrader [29] learns a router that picks the most suitable strategy for the current market, and mixture-of-experts designs train diversified traders and combine them [59]. Winnow [60] is originally a mistake-driven linear-threshold learning algorithm; the trading experiments later use a performance-based adaptation under that name, so it is not treated here as a generic theorem for expert weighting. EarnHFT [61] and MacroHFT [62] construct hierarchical MDPs for high-frequency trading and train the high-level router *by reinforcement learning on historical data*; in EarnHFT the hindsight Q supervision applies only to the second-level specialists, and the router itself is trained without a Q-teacher (Chapter 4). In both systems the router is fitted offline and then frozen for deployment rather than updated online.

3.1.6 Formulaic Alpha Mining

Formulaic alpha mining seeks short symbolic expressions whose cross-sectional values predict returns. Hand-crafted catalogues such as Alpha101 [17] gave way to automated search. Genetic programming [63] and its hierarchical and warm-started variants [34, 64] evolve expression trees by mutation and crossover. The broader quality-diversity lineage seeks a repertoire rather than one optimum [32]; MAP-Elites discretises a behaviour space and retains high-quality occupants of its cells [33] (see also Section 2.2.7). AutoAlpha is especially relevant because its PCA-QD component is an earlier use of these ideas in alpha mining; AlphaQD’s novelty is therefore the separate return-source MAP-Elites archive and its separation from the generator reward, not QD in the domain per se. AlphaEvolve [65] is another learning framework for discovering novel alphas and is reviewed here even though it is absent from the unified experimental table. Deep symbolic optimisation (DSO) [66] trains a recurrent generator against a standalone expression score, providing a counterexample to the claim that every RL-style symbolic generator must use a pool-coupled reward. The pool-based reinforcement-learning line includes AlphaGen [67], which casts mining as a PPO problem whose terminal reward is the marginal contribution of a new alpha to a running pool. AlphaQCM [68] adds a distributional value head and a variance bonus; RiskMiner [69] pairs risk-seeking Monte-Carlo tree search with a pool reward; Alpha² [70] searches logical formulas; AlphaForge [71] replaces the RL search with a generator–predictor surrogate and a dynamic re-combination stage; and AlphaSAGE [72] encodes expressions as typed graphs and trains a GFlowNet against a dense, multi-faceted reward. RiskMiner and Alpha² are contextual methods rather than reported baselines here; their omission limits the coverage of the empirical comparison. A parallel and very recent line replaces the search policy with a large language model [73–75]. As Chapter 6 details, AlphaQD addresses the moving target in this influential pool-based family; it does not claim that every symbolic RL objective is pool-coupled.

3.1.7 Uncertainty and Non-Stationary RL

The leverage setting of Chapter 5 foregrounds two kinds of uncertainty. *Aleatoric* uncertainty is the irreducible noise of the environment. Leverage does not change

the underlying transition uncertainty, but it magnifies that noise in account returns and rewards; one risk-aware quantile construction evaluated in automated driving [76] and adapted to the trading setting in Chapter 5 targets the resulting lower-tail risk. Ensemble value methods address a different axis: Averaged-DQN reduces target variance [24], while SUNRISE uses ensemble disagreement in its sample weighting and action selection [25]. *Epistemic* uncertainty is the model’s ignorance about states it has not seen and motivates OOD detection. A VAE [35] can supply a reconstruction-based distribution-familiarity score, although deep generative scores can be misleading under distribution shift [36]; the score is a heuristic rather than calibrated epistemic uncertainty (see Section 2.2.8). Modelling slowly drifting environments as a dynamic-parameter MDP [77] provides the formal setting in which a short window can be treated as approximately stationary. FineFT combines selective ensemble updates with one such VAE score per inferred dynamic.

3.1.8 Further Related Work

Beyond the task-specific lines above, four strands of prior work help situate the contributions of this thesis. Moody and Saffell [2] provide a foundational treatment of reinforcement learning for trading, training a recurrent trader by direct reinforcement on risk-adjusted performance rather than through an intermediate value function. Nevmyvaka, Feng, and Kearns [3] give the classical large-scale study of reinforcement learning for optimised trade execution on limit-order-book data, the setting in which the exogenous-price approximation exploited by Chapter 4 is most commonly invoked. The Q-teacher of Chapter 4 is related to deep Q-learning from demonstrations [21], which shapes value learning with a supervised loss on expert data; the Q-teacher differs in that dynamic programming computes sample-path hindsight continuation values under the exogenous-replay assumption rather than using externally supplied demonstrations. These are trajectory-conditioned training signals, not optimal action values of the unknown stochastic market. Finally, Harvey, Liu, and Zhu [8] document how multiple testing inflates apparent performance in cross-sectional return research—a winner’s-curse concern that applies directly to alpha mining, where many candidate formulas are evaluated on the same data (Chapter 6).

3.2 Open Challenges

This survey exposes three gaps that the technical chapters close.

- **Gap I (high-frequency efficiency and adaptivity).** Prior single-policy methods do not jointly address second-level horizon length and changing trends. A method is needed that improves learning efficiency at second scale and can route among differentiated policies. This motivates EarnHFT (Chapter 4).
- **Gap II (risk and distribution awareness under leverage).** Methods designed for low-leverage markets do not necessarily control the magnified P&L consequences of leverage or flag unfamiliar states, leaving them exposed during abrupt shifts. This motivates FineFT (Chapter 5).
- **Gap III (reward stationarity and selection in alpha mining).** A dominant family of pool-based RL alpha miners couples the reward to an evolving pool, training the policy against a moving target and complicating archive-conditional selection analysis. This motivates AlphaQD (Chapter 6).

A thread common to all three gaps is the placement of the aggregation mechanism. EarnHFT learns it, FineFT derives it heuristically from generative uncertainty estimates, and AlphaQD removes it from the reward entirely—a progression that the remainder of the thesis develops.

Part II

Reinforcement Learning for Trading

Chapter 4

EarnHFT: Hierarchical Reinforcement Learning for High-Frequency Trading

4.1 Introduction

High-frequency trading (HFT) uses computer algorithms to make trading decisions at very short time scales and accounts for a substantial share of activity in modern financial markets [78]. It is especially prevalent in cryptocurrency markets [79], whose round-the-clock trading removes market-closure gaps and non-tradable periods—although positions remain continuously exposed to price risk—and whose volatility creates short-lived trading opportunities. These features motivate strategies that react more frequently than conventional low-frequency systems.

Reinforcement learning has produced promising results on lower-frequency tasks such as daily or intraday stock and futures trading [4, 5], but moving to second-level decisions introduces two difficulties identified in Section 3.2 as Gap I.

- **The horizon is extremely long, so RL is data-inefficient.** Even twelve days at second resolution gives roughly one million steps— $60 \times 60 \times 24 \times 12 = 1,036,800$ —against the few-thousand-step horizon of Atari benchmarks [80]. Such horizons demand far more data to converge [9], straining computation.

- **The market drifts, so one policy need not transfer uniformly.** Training and testing periods can have different return dynamics. A policy fitted under one trend can therefore lose performance when the trend reverses.

This chapter presents EarnHFT, an **E**fficient **hier**archical **R**einforcement **lear**Ning method for **H**igh **F**requency **T**rading, whose three stages are summarised in Figure 4.1.¹ The design follows the decompose–specialise–aggregate pattern of Section 1.4.

1. **Stage I (Q-teacher).** Under the historical-replay assumption that the agent’s simulated orders do not alter the recorded future prices, we compute by dynamic programming the best continuation values on each realised training path. Used as a regulariser and a source of hindsight-greedy demonstrations, this “Q-teacher” supplies additional low-level learning signals.
2. **Stage II (specialist pool).** Four preference-conditioned training runs produce 200 candidate checkpoints. We label the validation market with a fixed segmentation configuration and select a small pool of specialists by their observed validation performance.
3. **Stage III (router).** We train a minute-level router that dynamically selects a second-level specialist from the pool as the observed features change.

Across the four reported cryptocurrency test paths, spanning bull, bear, and sideways periods, EarnHFT has the largest tabled total return among the six baselines on every path and the largest risk-adjusted metrics on three. These are single-seed, nine-day point estimates rather than population-level performance claims.

4.2 Problem Formulation

The financial concepts—limit order book, OHLC, technical indicators, market order, position, and net value—are defined in Section 2.1.1. We restrict attention to a single asset with long-only positions, and the objective is to maximise net

¹This chapter has been published in the *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI’24)* [61].

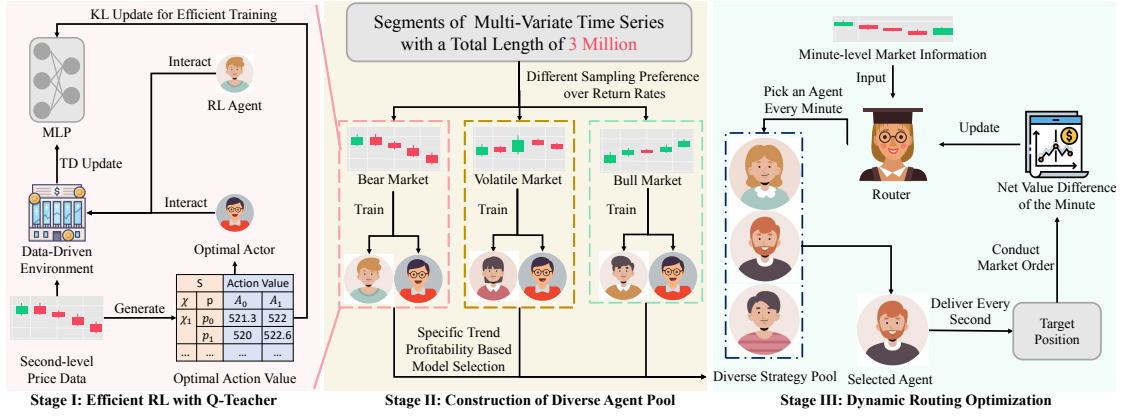


FIGURE 4.1: Overview of the three stages of EarnHFT. Stage I computes a sample-path Q-teacher that supplies auxiliary training signals to second-level learners. Stage II produces preference-conditioned checkpoints and selects a small validation-ranked pool. Stage III trains a minute-level router that selects one retained second-level specialist.

value $V_t = V_{ct} + H_t p_t^{b_1}$, where V_{ct} is cash and H_t the position valued at the best bid. In the historical simulator, a target-position change of Δ is priced immediately against the recorded book, level by level, and $E_t(\Delta)$ denotes the resulting commission-inclusive simulated cash flow—the signed cash amount, not an average execution price. A buy ($\Delta > 0$) consumes the recorded ask side,

$$E_t(\Delta) = \sum_{i=1}^4 (p_t^{a_i} \min(q_t^{a_i}, R_{i-1})) (1 + \sigma),$$

whereas a sell ($\Delta < 0$) consumes the recorded bid side with the commission deducted from the proceeds,

$$E_t(\Delta) = - \sum_{i=1}^4 (p_t^{b_i} \min(q_t^{b_i}, R_{i-1})) (1 - \sigma),$$

where $R_0 = |\Delta|$ and, on the relevant side of the book, $R_i = R_{i-1} - \min(q_t^{a_i}, R_{i-1})$ for a buy and $R_i = R_{i-1} - \min(q_t^{b_i}, R_{i-1})$ for a sell. Thus R_{i-1} is the quantity still unfilled after the first $i-1$ levels. Here σ is the commission rate and $E_t(0) = 0$. The released action-availability calculation sums the first four levels; a change requiring more displayed depth is masked rather than extrapolated beyond them. Under this convention a buy is a positive cash outflow and a sell a negative one. This is an accounting model over recorded snapshots, not a live execution engine: it assumes

immediate access to displayed depth and no latency or market impact. Accordingly, “high-frequency trading” in this chapter means second-level target-position control in historical replay; order placement and cancellation, queue priority, realistic partial-fill dynamics, latency, and endogenous impact are outside its scope.

4.2.1 A Hierarchical Markov Decision Process

The two challenges of Section 4.1 motivate a temporal decomposition: micro-level features support second-by-second target-position decisions, while their minute-level aggregation supplies a slower signal for switching among specialists. We therefore formulate a hierarchical MDP ($\text{MDP}_h, \text{MDP}_l$), in which a low-level MDP applies the replay cash-flow model at second scale and a high-level MDP selects a specialist at minute scale:

$$\text{MDP}_h = (S_h, A_h, P_h, R_h, \gamma_h, T_h), \quad \text{MDP}_l = (S_l, A_l, P_l, R_l, \gamma_l, T_l).$$

Low level (second scale). The low-level network receives the current row of 54 selected micro-level features, together with the current position H_t . The released preprocessing constructs trailing OHLC and LOB features with lookbacks of 5, 10, 30, and 60 seconds; the implementation’s state-stack length is one, so “length 60” denotes the maximum feature lookback rather than a 60-row tensor supplied to the network. LOB-derived features use the first five displayed levels. The action a_{lt} is a target position chosen from a finite pool $\{0, \frac{H}{|A|-1}, \dots, H\}$; target positions outside the change supported by the first four displayed depth levels are masked, and the teacher assigns an effectively prohibitive value to an unfillable change. Otherwise the simulator prices the change through the replay cash-flow model above. The reward $r_{lt} = H_{t+1}p_{t+1}^{b_1} - (H_t p_t^{b_1} + E_t(H_{t+1} - H_t))$ is the simulated per-second change in net value.

High level (minute scale). The high-level network analogously receives the current row of 19 selected macro-level features constructed from trailing minute OHLC, together with H_t . The action a_{ht} selects an agent from the pre-trained pool A_h . The reward $r_{hT} = \sum_{t=T}^{T+\tau_m-1} r_{lt}$, where $\tau_m = 60$ seconds is the length of one minute, is the simulated net-value change accrued by the selected low-level agent

over those τ_m steps. Every minute the router picks a frozen low-level specialist, which may adjust the target position every second; the specialists are trained first and the router afterwards rather than by joint optimisation.

Options view and information timing. The high level can be viewed as a semi-Markov decision process [26, 27]: each action hands control to a selected specialist for τ_m low-level steps. The reported code sets both discounts to one, $\gamma_l = \gamma_h = 1$, so the within-option reward is the undiscounted sum r_{hT} above; the scripts used for the reported runs do not override these defaults. The data alignment is causal with respect to OHLC completion: after aggregation, one-second bars are shifted forward by one second and one-minute bars by one minute before they are merged with the order-book stream, so a row time-stamped t contains bars that have ended by t , not a bar still forming after the decision. Minute bars are forward-filled between minute boundaries, and all rolling features are trailing. The contemporaneous LOB snapshot and these completed-bar features are read before the simulator advances to the next second.

4.3 Method

4.3.1 Stage I: Efficient Reinforcement Learning with a Q-Teacher

A long trajectory normally inflates the cost of RL. Under the simulator’s exogenous-replay assumption, however, the recorded future prices do not change with the agent’s action. Dynamic programming can therefore construct a *hindsight dynamic-programming teacher*: a table Q^* containing the best simulated continuation value on each realised training path, indexed by timestamp, previous position, and next target position [1]. Three boundaries are essential. First, Q^* is a sample-path hindsight benchmark, not the optimal action value of the unknown stochastic market; the star is retained for continuity with the published notation and always has this trajectory-conditioned meaning here. Second, its construction consumes future training prices, but the table is used only to generate training targets and demonstrations and is unavailable during validation, testing, and deployment. Third, the network state (χ, p) does not contain the timestamp, so identical feature states can

receive different hindsight targets on different visits. The table is therefore used as an auxiliary regulariser and demonstrator, following the general precedent of learning from demonstrations [21], rather than as a value function that the deployed network is expected to reproduce exactly.

Hindsight-value supervisor. Because the position set is finite, the teacher values can be computed backwards. In the reported configuration, $\gamma_l = 1$, so each entry is the immediate simulated reward plus the best undiscounted continuation on that same sample path, as in Algorithm 1.

The temporal-difference error updates the value of the sampled action, whereas the table and a network forward pass each provide a vector over all target positions. The implementation converts these vectors into action distributions and minimises the divergence from the teacher distribution to the network distribution:

$$\begin{aligned}\mu_{t,p}^* &:= \text{softmax}(Q^*[t, p, \cdot]/\tau_{\text{sm}}), \\ \mu_{\theta_i}(\chi, p) &:= \text{softmax}(Q_{\theta_i}(\chi, p, \cdot)/\tau_{\text{sm}}), \\ L(\theta_i) &= L_{td} + \alpha_{\text{KL}} \text{KL}(\mu_{t,p}^* \parallel \mu_{\theta_i}(\chi, p)), \quad \tau_{\text{sm}} = 1.\end{aligned}\tag{4.1}$$

For a sampled transition collected at historical time t , the first distribution is therefore a direct lookup in the trajectory-indexed table; it is not a function of χ . Writing d for the terminal indicator, the temporal-difference term is

$$\begin{aligned}y_i^{\text{TD}} &= r + \gamma_l(1 - d) \max_b Q_{\theta'_i}(\chi', a, b), \\ L_{td} &= (y_i^{\text{TD}} - Q_{\theta_i}(\chi, p, a))^2, \quad \gamma_l = 1.\end{aligned}$$

The released implementation takes the maximum directly over the target network; despite the original paper’s DDQN terminology, this is a target-network DQN update rather than strict Double DQN, which would select with the online network and evaluate with the target network. Here χ is the feature vector, p the previous position, and θ'_i the target parameters. For the two distributions in (4.1), the code evaluates $\text{KLDiv}(\log(\mu_{\theta_i} + 10^{-8}), \mu_{t,p}^* + 10^{-8})$; its direction is therefore $\text{KL}(\mu_{t,p}^* \parallel \mu_{\theta_i})$, with 10^{-8} used only for numerical stability. There is no learned or tuned softmax temperature. The supervision coefficient starts at 256 and decreases linearly to a floor of 128 over 5×10^6 environment steps; it does not decay to zero. The softmax

distributions exist only inside the loss, while action selection remains greedy in the network values at evaluation time.

Two properties explain what this auxiliary term matches. First, the softmax is unchanged by adding the same constant to every action value: for any $c \in \mathbb{R}$,

$$\text{softmax}(Q(x, \cdot)) = \text{softmax}(Q(x, \cdot) - c) \quad \text{for every } c \in \mathbb{R}. \quad (4.2)$$

Choosing $c = \frac{1}{|A|} \sum_{a \in A} Q(x, a)$ shows that each distribution depends on its action-value vector only through that vector *centred across the actions*—the advantage of each action over the state’s average action value—and not on a common offset. The analytical surrogate below uses an analogous centred residual Re ; it is not the trajectory table Q^* and should not be identified with it. The KL term therefore matches relative action preferences, including their separation, rather than the common value level.

Second, removing the common offset makes the auxiliary target less restrictive than direct L_2 regression onto hindsight returns. It still asks the network to imitate relative preferences derived from a realised future path, so their out-of-sample usefulness is an empirical assumption, not an optimality guarantee. This is why the term is treated as regularisation rather than as supervised recovery of the market’s true value function.

At every visited state this construction supplies an auxiliary gradient over all target positions, including positions not selected in that transition; the ablation in Section 4.5.3 evaluates its empirical effect.

In Algorithm 1, p and a are indices into the position pool, and the immediate term is the simulator’s one-step marked-to-bid reward for moving from $\text{pos}(p)$ to $\text{pos}(a)$. The $-\infty$ notation is the mathematical form of an effectively prohibitive action mask; the no-change action remains feasible, so the maximum and the softmax normaliser always contain a finite entry. The released helper exposes a discount argument, but the training environment passes the reported default $\gamma_l = 1$; the algorithm therefore shows the recursion that was actually evaluated. Its terminal row is zero and the backward loop ends at $t = 0$, avoiding an out-of-range access at $t = N - 1$. The resulting table is specific to the supplied historical chunk and its replay cost model.

Algorithm 1 Construction of the Hindsight Teacher Values

```

1: Input: time series  $\mathcal{D} = \{x_t\}_{t=0}^{N-1}$ , commission rate  $\sigma$ , action space  $A$ , maximum
   holding  $H$ 
2: Output: table  $Q^*$  of teacher action values indexed by time, position index,
   action index
3: Let  $I = \{0, \dots, |A| - 1\}$  and  $\text{pos}(i) = \frac{iH}{|A|-1}$ ; initialise  $Q^* \in (\mathbb{R} \cup \{-\infty\})^{N \times |A| \times |A|}$ 
   to zero.
4: for  $t \leftarrow N - 2$  down to 0 do
5:   for  $p \leftarrow 0$  to  $|A| - 1$  do
6:     for  $a \leftarrow 0$  to  $|A| - 1$  do
7:       if  $\text{pos}(a) - \text{pos}(p)$  cannot be filled from levels 1:4 then
8:          $Q^*[t, p, a] \leftarrow -\infty$   $\triangleright$  masked action
9:       else
10:         $Q^*[t, p, a] \leftarrow \max_{a' \in I} Q^*[t + 1, a, a'] + \text{pos}(a) p_{t+1}^{b_1} - (\text{pos}(p) p_t^{b_1} +$ 
         $E_t(\text{pos}(a) - \text{pos}(p)))$ 
11:      end if
12:    end for
13:  end for
14: end for
15: return  $Q^*$ 

```

Hindsight-greedy demonstrator. In addition to ϵ -greedy rollouts, the implementation collects a rollout that chooses $\arg \max_a Q^*[t, p, a]$ on the realised training chunk. These are sample-path demonstrations rather than actions available from an oracle at deployment. Algorithm 2 summarises the two rollout streams and their common TD-plus-KL update.

The following analysis is an analytical motivation, not a convergence guarantee for the released learner. It considers a tabular historical-replay MDP with state (t, p) rather than the network’s feature state, and replaces the implemented teacher-to-student softmax-KL by an exact centred-value matching surrogate. Shift invariance explains why both operate on relative action values, but it does not make the two objectives identical. Nor is a formal correspondence established between the implementation’s scheduled $\alpha_{\text{KL}} \in [128, 256]$ and the fixed mixing weight λ below. To avoid a visually ambiguous case-only distinction, the implemented trajectory table remains Q^* , whereas the theoretical optimum is denoted q^{opt} throughout this

Algorithm 2 Efficient Reinforcement Learning with a Q-Teacher

```

1: Input: time series  $\mathcal{D}$  of length  $N$ , commission rate  $\sigma$ , action space  $A$ , number
   of epochs  $n_{\text{ep}}$ , minibatch size  $B$ 
2: Output: network parameters  $\theta$ 
3: Initialise replay buffers  $R_h, R_\epsilon$ , network  $Q_\theta$ , and target network  $Q_{\theta'}$ ; build  $Q^*$ 
   via Algorithm 1.
4: for epoch = 1 to  $n_{\text{ep}}$  do
5:   Reset the trading environment.
6:   for  $t = 0$  to  $N - 2$  do ▷ hindsight-greedy rollout
7:     Choose  $a_h = \arg \max_a Q^*[t, p, a]$ ; execute it to obtain  $(r, s')$  and store
      $(s, a_h, r, s', Q^*[t, p, \cdot])$  in  $R_h$ .
8:   end for
9:   Reset the trading environment.
10:  for  $t = 0$  to  $N - 2$  do ▷ exploration rollout
11:    Choose  $a_\epsilon$  by the  $\epsilon$ -greedy policy; execute it to obtain  $(r, s')$  and store
     $(s, a_\epsilon, r, s', Q^*[t, p, \cdot])$  in  $R_\epsilon$ .
12:  end for
13:  for each scheduled gradient step, once the active buffer holds at least  $B$ 
    transitions do
14:    Sample  $B$  transitions from the corresponding buffer and compute  $L$  by
    (4.1), using  $r + \gamma_l(1 - d) \max_a Q_{\theta'}(s', a)$  with  $\gamma_l = 1$ .
15:    Descend on  $\theta$ ; update the target network by  $\theta' \leftarrow \tau_{\text{poly}}\theta + (1 - \tau_{\text{poly}})\theta'$ .
16:  end for
17:  Decay  $\epsilon$  and  $\alpha_{\text{KL}}$ , retaining the floor  $\alpha_{\text{KL}} \geq 128$ .
18: end for
19: return  $Q_\theta$ 

```

surrogate analysis. For the discounted tabular surrogate, the operator is

$$\begin{aligned} \mathcal{T}q(x, a) = & \lambda \sum_{y \in X} P(y \mid x, a) (r(x, a, y) + \gamma \max_b q(y, b)) \\ & + (1 - \lambda) \left(\frac{1}{|A|} \sum_{a' \in A} q(x, a') + Re(x, a) \right). \end{aligned} \quad (4.3)$$

Here X is the finite state space, P the transition kernel, and q the argument of the operator. The supervision residual is $Re(x, a) = q^{\text{opt}}(x, a) - \frac{1}{|A|} \sum_{a' \in A} q^{\text{opt}}(x, a')$, and $\lambda \in (0, 1]$ weights the Bellman backup against the supervision term.

Proposition 4.1 (The surrogate q^{opt} is a fixed point of $\mathcal{T}$). *Let q^{opt} denote the Bellman-optimal action value of the discounted tabular surrogate. Then $\mathcal{T}q^{\text{opt}} = q^{\text{opt}}$.*

Proof. Substituting q^{opt} into (4.3), the Bellman optimality equation identifies the first term with $q^{\text{opt}}(x, a)$, while the definition of Re identifies the centred-value

term:

$$\begin{aligned}\mathcal{T}q^{\text{opt}}(x, a) &= \lambda q^{\text{opt}}(x, a) + (1 - \lambda) \left(\frac{1}{|A|} \sum_{a'} q^{\text{opt}}(x, a') + Re(x, a) \right) \\ &= \lambda q^{\text{opt}}(x, a) + (1 - \lambda) q^{\text{opt}}(x, a) = q^{\text{opt}}(x, a).\end{aligned}\quad \square$$

Proposition 4.2 (Contraction in the sup-norm). *Let $\gamma \in (0, 1)$. For any action values q_1, q_2 , $\|\mathcal{T}q_1 - \mathcal{T}q_2\|_\infty \leq k \|q_1 - q_2\|_\infty$ with $k = \lambda\gamma + (1 - \lambda) = 1 - \lambda(1 - \gamma) < 1$.*

Proof. The supervision residual $Re(x, a)$ depends only on q^{opt} and not on the argument of $\mathcal{T}$, so it cancels in the difference $\mathcal{T}q_1 - \mathcal{T}q_2$. Bounding the max operator by $|\max_b q_1(y, b) - \max_b q_2(y, b)| \leq \max_b |q_1(y, b) - q_2(y, b)|$,

$$\begin{aligned}\|\mathcal{T}q_1 - \mathcal{T}q_2\|_\infty &\leq \lambda\gamma \|q_1 - q_2\|_\infty + (1 - \lambda) \|q_1 - q_2\|_\infty \\ &= (\lambda\gamma + (1 - \lambda)) \|q_1 - q_2\|_\infty = (1 - \lambda(1 - \gamma)) \|q_1 - q_2\|_\infty.\end{aligned}$$

Since $\gamma \in (0, 1)$ and $\lambda \in (0, 1]$, the factor $k = 1 - \lambda(1 - \gamma) < 1$. When $\gamma = 1$, as in the reported experiments, the factor equals 1 and this contraction proof no longer applies. Finite-horizon backward induction still defines the sample-path teacher in Algorithm 1, but it does not establish convergence of the function-approximation learner. $\square$

Theorem 4.1 (Convergence of the supervised update in the tabular idealisation). *Consider the finite historical-replay MDP with state (t, p) , bounded rewards, and discount $\gamma \in (0, 1)$. Suppose the tabular iterates driven by the operator $\mathcal{T}$ of (4.3) use step sizes satisfying the Robbins–Monro conditions ($\sum_k a_k = \infty$, $\sum_k a_k^2 < \infty$), every state–action pair is visited infinitely often, and the update noise has bounded variance. Then the iterates converge almost surely to the surrogate fixed point q^{opt} .²*

Proof. Propositions 4.1 and 4.2 show that $\mathcal{T}$ is a sup-norm contraction with unique fixed point q^{opt} . Under the stated step-size, visitation, and noise conditions, the stochastic-approximation argument of [81] applies, and the iterates converge almost surely to q^{opt} . $\square$

²The released learner uses function approximation, teacher-to-student softmax-KL, and $\gamma_l = 1$. All three place the reported experiments outside this theorem’s assumptions; the result is analytical motivation only.

4.3.2 Stage II: Construction of the Agent Pool

Because the relation between micro-level features and profitable actions changes across the sample, one network need not perform uniformly over every period. EarnHFT therefore constructs preference-conditioned candidate checkpoints and lets the validation procedure retain specialists for different labelled trends.

Generating diverse agents. Rather than relying on different random seeds to create diversity [59], the four training runs use explicit preferences over chunk returns. The training series (about 2.4 million steps) is divided into fixed $L = 14,400$ -second chunks; these are sampling units, not guaranteed single-trend intervals. A preference parameter β assigns each chunk with buy-and-hold return rate r a sampling priority

$$f(r) = \begin{cases} \frac{e^{\beta r}}{\text{pdf}(r)} & \text{if } Q_{\frac{\zeta}{2}}(R) \leq r \leq Q_{1-\frac{\zeta}{2}}(R), \\ e^{\beta r} & \text{otherwise,} \end{cases} \quad (4.4)$$

where $Q_{\frac{\zeta}{2}}(R)$ is the $\frac{\zeta}{2}$ -quantile of return rates, ζ the clipping quantile, and pdf is a kernel density estimate,

$$\text{pdf}(x) = \frac{1}{nh} \sum_{r \in R} K\left(\frac{x - r}{h}\right), \quad (4.5)$$

where $n = |R|$, bandwidth h is searched around Silverman’s rule [82], and K is a standard normal kernel. Dividing by the estimated density counterweights common returns, while $e^{\beta r}$ tilts sampling towards a preferred return regime; clipping limits the influence of extreme chunks. Different β values thereby induce different training distributions.

Market segmentation and labelling. Specialist selection uses an offline label of the validation path. Algorithm 3 applies the released code’s fixed configuration to the best-bid series: a zero-phase Butterworth filter, turning-point segmentation, and normalised fastDTW merging [83, 84]. The same constants are used for all four markets; they are fixed implementation choices rather than parameters inferred from each dataset.

Algorithm 3 Market Segmentation and Labelling

- 1: **Input:** validation best-bid series $\mathcal{D} = \{p_t^{b_1}\}_{t=0}^{N-1}$; $M = 5$ labels; clipping fraction $\zeta = 0.1$
 - 2: **Output:** a trend label for every point of $\mathcal{D}$
 - 3: $\mathcal{D}' \leftarrow \text{filtfilt}(\mathcal{D})$ using a fourth-order Butterworth low-pass filter with normalised cutoff $W_n = 1/30$.
 - 4: Split $\mathcal{D}'$ at sign changes of its first difference; absorb each segment shorter than 60 seconds into its predecessor.
 - 5: **for** $k \leftarrow 1$ **to** 20 **do**
 - 6: Fit a linear slope to each remaining segment and temporarily assign five quantile labels.
 - 7: For each segment shorter than 3,600 seconds, compute its length-and-level-normalised fastDTW distance to each adjacent candidate.
 - 8: Disallow a merge if the two temporary labels differ by more than one; otherwise merge with the closer neighbour when the distance is below 3×10^{-4} .
 - 9: Stop if no segment was merged in this round.
 - 10: **end for**
 - 11: Refit each segment's normalised slope $r_i = 100 \hat{b}_i / \mathcal{D}'_{t_i}$; collect the slopes into R .
 - 12: Set c_{lo} and c_{hi} to the implementation's order-statistic approximations to $Q_{\zeta/2}(R)$ and $Q_{1-\zeta/2}(R)$.
 - 13: Use the four equally spaced cut points from c_{lo} to c_{hi} to assign labels $0, \dots, 4$.
 - 14: Expand each segment label to its timestamps and **return** the pointwise labels.
-

The distance routine is fastDTW, not the “TWAP” written in one version of the published appendix. It rolls the shorter segment over the longer one, averages the fastDTW distances, and divides by the shorter segment's length times its mean price. Likewise, the implementation does not test a separate continuous slope-difference tolerance: the condition is a discrete temporary-label gap of at most one plus the fastDTW threshold. Because the fourth-order filter uses `filtfilt`, it is non-causal; it is used only for offline validation segmentation and is not an online state feature.

Agent selection. Including all trained agents would inflate the router's action space, so we select a small pool. Each point of the validation set is labelled by Algorithm 3; agents are evaluated across trends and initial positions, and the most profitable agent under each (trend, initial-position) pair is retained, forming a two-dimensional pool indexed by (m, n) for m trends and n initial positions. Two caveats apply to this selection. The 200 candidates are checkpoints drawn from only four training runs (four preference values $\times$ 50 epochs), so many of them

are highly correlated; and the same validation window is used both to select the specialists and to select the router, so the selection is exposed to a winner’s-curse effect [8], and validation performance is likely to overstate test performance.

4.3.3 Stage III: Dynamic Routing

The minute-level router is another target-network value learner. The published method calls it DDQN [20], but the released update, like the low-level update, takes the next-action maximum directly over the target network and is therefore DQN-style rather than strict Double DQN. Acting once per minute reduces the number of decisions by $59/60 = 98.33\%$ relative to a second-level controller. At each step the router is restricted to specialists whose initial-position index matches the current position, leaving one candidate per trend label. No hindsight teacher is built at this level: the horizon is already reduced, and each high-level action invokes an entire specialist rather than choosing a target position.

4.4 Experimental Setup

Datasets. We evaluate on four cryptocurrencies covering mainstream and niche assets and bull, bear, and sideways conditions, summarised in Table 4.1. All four are spot trading pairs on Binance, from which the second-level OHLC and LOB snapshots derive. On each, the last nine days are used for testing, the preceding nine for validation, and the remainder for training. Low-level agents are trained on the training set; the validation set is segmented and labelled for model selection; the router is then trained on the training set and selected on the same validation interval before testing. The released preprocessing uses five LOB levels for state-feature construction, resamples the merged stream to one-second frequency, and forward-fills missing rows. It does not state the timestamp timezone, and the repository omits the selected-feature `.npy` artefacts loaded by the training programs, so the exact 54- and 19-feature lists cannot be reconstructed from the release alone.

Metrics. We report the six financial metrics defined in Section 2.1.3: TR (profit); ASR, ACR, ASoR (risk-adjusted profit); and AVOL, MDD (risk). EarnHFT forms

TABLE 4.1: The four cryptocurrency datasets, with market dynamics, number of seconds, and chronological span. Dates are in YY/MM/DD format.

Dataset	Dynamics	Seconds	From	To
BTC/TUSD	Sideways	4,057,140	23/03/30	23/05/15
BTC/USDT	Sideways	3,884,400	22/09/01	22/10/15
ETH/USDT	Bear	3,970,800	22/05/01	22/06/15
GALA/USDT	Bull	3,970,740	22/07/01	22/08/15

the metric-return series at one-second resolution and uses $m = 365 \times 24 \times 60 \times 60$ intervals per year in the annualisation factors.

Baselines. We compare against six methods. The policy-based RL baselines are PPO [22] and DRA [5]; the value-based RL baselines are DQN [19] and CDQNRP [6]; and the rule-based methods are MACD [11] and IV [85]. The study reuses official hyperparameters where implementations are available and otherwise uses its own paper-guided implementations.

Training. The reported hardware is an RTX 4090 GPU. The commission rate is 0 for BTC/TUSD and 0.015% for BTC/USDT, ETH/USDT, and GALA/USDT.³ Four runs use $\beta \in \{-90, -10, 30, 100\}$ and save 50 epoch checkpoints each, giving 200 correlated candidates rather than 200 independent training runs. The released low-level configuration uses Adam, a replay capacity of 10^6 , minibatches of 512, reward scale 30, a learning rate decaying from 10^{-2} to 5×10^{-4} , α_{KL} decaying from 256 to 128, and Polyak coefficient $\tau_{\text{poly}} = 0.005$. Both levels use $\gamma_l = \gamma_h = 1$. Each EarnHFT configuration and each learned baseline is trained once with fixed seed 12345; the 50 checkpoints are not seed replications. The maximum holding is 0.01 BTC on the two Bitcoin pairs, 0.1 ETH, and 4,000 GALA. The published study reports approximately ten hours for the full experiment set.

³The 0.02% figure in the training-setup paragraph of the published EarnHFT paper is a typographical error: its appendix and author response, together with the released experiment scripts, use 0.015% for these three USDT pairs and zero for BTC/TUSD.

4.5 Results and Analysis

4.5.1 Comparison with Baselines

Table 4.2 reports the four held-out paths. EarnHFT has the largest tabled TR on each and the largest ASR, ACR, and ASoR on three, while several baselines have lower volatility or drawdown. In these runs, the value-based baselines are positive only on some comparatively stable paths, and the policy-based baselines record large losses on ETH. DRA and PPO have identical GALA rows. According to the author, under transaction costs both policy-gradient baselines can converge quickly to an almost unchanging position policy; on this path they converged to the same low-switching behaviour, producing coincident summary metrics. The EarnHFT rows favour return over uniformly low risk, which is consistent with the profit-oriented hindsight signal but does not by itself establish a causal mechanism. Every comparison is a point estimate from one fixed training seed and one nine-day evaluation path per market; there are no seed confidence intervals or significance tests. The ranking is therefore descriptive for these four paths. Figure 4.2 illustrates one ETH episode in which the simulated target position opens and closes within roughly thirty seconds.

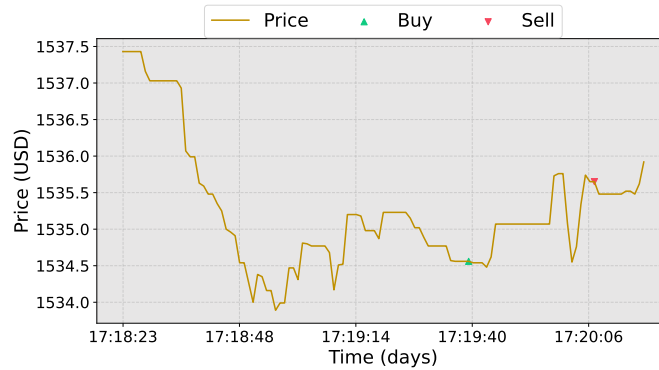


FIGURE 4.2: An illustrative ETH replay. The simulated target position opens and closes within roughly thirty seconds; the corresponding price movement appears only as a pullback at minute scale. The embedded horizontal-axis label “Time (days)” is erroneous: its tick labels are wall-clock times and the episode is shown at second scale.

TABLE 4.2: Performance comparison on four cryptocurrency markets against six baselines (two policy-based, two value-based, and two rule-based methods). Arrows indicate the preferred direction; the **best** entry per market and metric is in bold (colour retained for screen reading). TR, AVOL, and MDD are in percent. Each entry is a single-path point estimate.

Market	Model	TR $\uparrow$	ASR $\uparrow$	ACR $\uparrow$	ASoR $\uparrow$	AVOL $\downarrow$	MDD $\downarrow$
BTCU	DRA	-4.56	-4.28	-19.57	-4.65	42.25	9.24
	PPO	-3.61	-5.25	-22.74	-5.71	27.76	6.41
	CDQNRP	-2.83	-2.91	-14.85	-3.31	37.61	7.38
	DQN	-3.48	-12.37	-35.01	-11.86	11.57	4.09
	MACD	-6.07	-10.11	-25.17	-8.05	24.84	9.98
	IV	-2.99	-3.78	-14.24	-3.27	31.35	8.32
	EarnHFT	0.72	1.22	10.77	0.93	27.08	3.07
BTCT	DRA	-2.65	-4.82	-17.48	-4.77	21.18	5.84
	PPO	-0.60	-14.74	-35.80	-0.14	1.59	0.65
	CDQNRP	-0.60	-19.52	-37.88	-0.74	1.20	0.61
	DQN	0.47	4.21	27.94	1.14	4.38	0.66
	MACD	-4.02	-5.80	-24.21	-4.32	26.87	6.44
	IV	-12.01	-17.83	-38.99	-13.90	27.68	12.66
	EarnHFT	0.99	1.34	7.76	1.01	32.40	5.61
ETH	DRA	-33.37	-9.06	-32.23	-9.20	163.25	45.88
	PPO	-22.61	-10.11	-31.17	-10.39	96.12	31.17
	CDQNRP	-6.82	-24.41	-40.19	-3.11	11.46	6.96
	DQN	-11.02	-9.47	-32.81	-8.43	47.76	13.79
	MACD	-4.29	-1.78	-8.71	-1.19	79.64	16.35
	IV	-27.42	-12.27	-36.01	-9.00	99.67	33.96
	EarnHFT	4.52	2.92	14.30	1.78	67.92	13.89
GALA	DRA	10.56	4.77	41.63	4.44	92.43	10.60
	PPO	10.56	4.77	41.63	4.44	92.43	10.60
	CDQNRP	5.22	4.51	39.42	4.16	47.27	5.41
	DQN	2.94	3.55	32.02	2.66	34.08	3.78
	MACD	2.37	1.79	11.45	1.22	62.89	9.84
	IV	13.95	6.74	55.67	5.41	81.79	9.91
	EarnHFT	19.41	9.77	79.08	7.79	74.94	9.26

4.5.2 Effectiveness of the Hierarchical Framework

Figure 4.3 compares the router with each retained specialist on the reported paths. Checkpoints selected for bull or rally labels tend to hold longer on GALA, whereas pullback and bear-labelled checkpoints close positions more often on ETH. The router records the largest TR in these comparisons. Figure 4.4 shows its selection distribution: ETH selections are spread across several labels, while 67.6% of GALA selections go to the Bull-labelled specialist; BTCT and BTCU concentrate on fewer labels. These associations describe selected checkpoints and should not be read as proof that each network has learned a uniquely identifiable market regime.

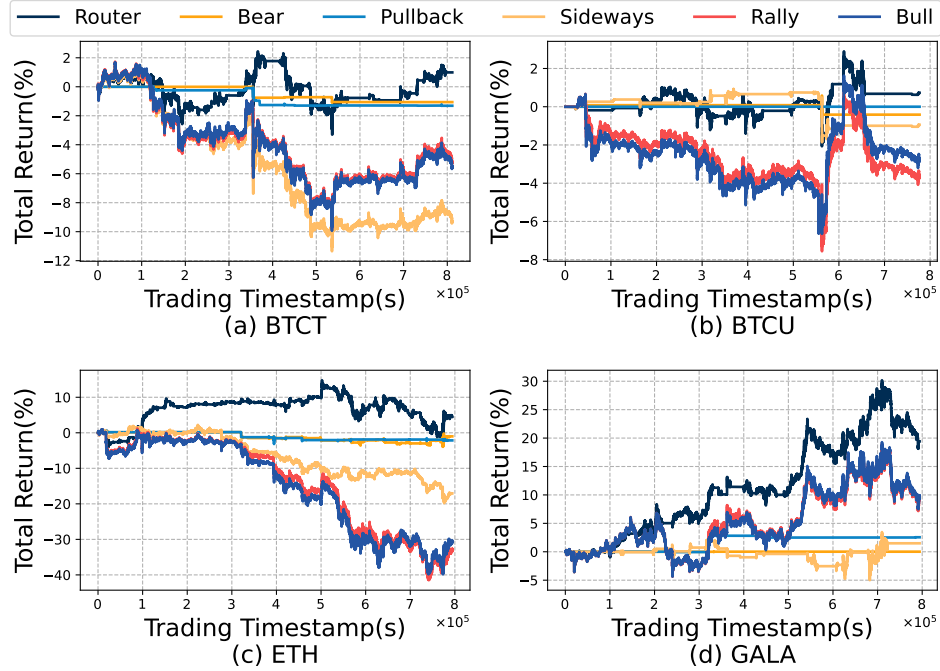


FIGURE 4.3: Reported-path comparison of the router with the retained specialists in its pool. The labels denote the validation regimes for which checkpoints were selected.

4.5.3 Ablation of the Hindsight Teacher

Table 4.3 retains the published labels OS (“optimal supervisor”) and OA (“optimal actor”), but both refer to the sample-path hindsight table defined above: OS is the teacher-to-student KL term and OA the hindsight-greedy demonstration rollout. CS is the study’s reported convergence-step proxy, RS the reward sum at that point, and AHL the average holding length. On GALA, adding OS to the unassisted

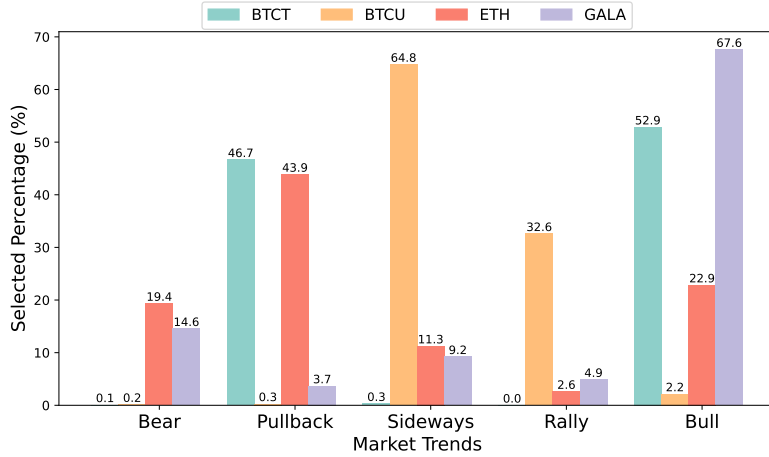


FIGURE 4.4: Distribution of the router’s specialist selections across the four datasets. ETH elicits a comparatively spread-out distribution across trends; GALA concentrates most selections (67.6%) on the Bull specialist; the low-volatility markets concentrate on fewer dynamics.

target-network learner lowers CS from 30,720 to 4,608 and raises RS from -0.01 to 2.89; adding both components gives the largest GALA RS but a larger CS. On ETH, OS leaves CS tied with the unassisted row at 30,720 and changes RS from -29.6 to 4.87, while the two-component row has the largest RS. These two-market ablations support an empirical contribution from the teacher components; they do not establish convergence or stochastic-market optimality.

TABLE 4.3: Ablation of the hindsight-value supervisor (OS in the published table) and hindsight-greedy demonstrator (OA) on ETH and GALA. CS is the reported convergence-step proxy, RS the associated reward sum, and AHL the average holding length. All tied best CS and RS values are in bold (colour retained for screen reading); AHL is descriptive and has no preferred direction.

OA	OS	GALA			ETH		
		CS↓	RS↑	AHL	CS↓	RS↑	AHL
✓	✓	78848	4.43	448	102400	12.32	81.3
✓		102400	0.24	38.7	102400	-1.40	4.15
	✓	4608	2.89	147	30720	4.87	35.8
		30720	-0.01	284	30720	-29.6	39.1

4.5.4 Trading Behaviour and Computational Cost

The reported trajectories show different position patterns across markets. BTCT and BTCU contain relatively few target-position changes; ETH contains more frequent changes; and GALA contains longer holding intervals. BTCU includes the 0.015% commission whereas BTCT is commission-free, so their trading counts are not directly comparable on price dynamics alone. The ETH and GALA patterns are consistent with their labelled bear and bull periods, respectively, but the observational plots do not identify why the networks chose each action or establish that they learned a general regime detector.

Computational cost. The published study reports approximately ten hours on an RTX 4090. This covers constructing the teacher tables, running four preference-conditioned trainings, saving 200 checkpoints, segmenting the validation paths, selecting the pools, and training the routers. At evaluation time the model computation is one specialist-network forward pass per second and one router-network pass per minute. No end-to-end latency benchmark, exchange connectivity, or live order-execution timing is reported, so this computational count is not a live-trading latency claim.

4.6 Chapter Summary

This chapter presented EarnHFT as a three-stage hierarchical RL method for second-level target-position control in historical order-book replay. A sample-path dynamic-programming table supplies teacher-to-student softmax-KL supervision and hindsight-greedy demonstrations; preference-conditioned checkpoints provide a specialist pool; and a minute-level target-network router switches among the retained specialists. The discounted tabular surrogate in Theorem 4.1 motivates centred-value supervision, but the reported function-approximation experiments use a different KL objective and $\gamma_l = \gamma_h = 1$, so the theorem is not a convergence guarantee for them. On the four single-seed, nine-day test paths, EarnHFT has the largest tabled TR on all four and the largest risk-adjusted metrics on three; the ablations on ETH and GALA associate both teacher components with higher reported reward sums. In the language of the thesis, EarnHFT is the first

instantiation of the decompose–specialise–aggregate pattern, with a value-based router selecting one specialist at a time. The next chapter retains the pattern but considers leveraged futures, where aggregation must additionally manage risk and distributional familiarity.

Chapter 5

FineFT: Risk-Aware Ensemble Reinforcement Learning for Futures Trading

5.1 Introduction

Futures are standardised contracts obligating the exchange of an asset at a pre-determined date and price. The instruments traded in this chapter are *perpetual* futures, a variant with no delivery date whose price is kept aligned with the spot price through a periodic funding-fee mechanism. Perpetual futures permit both long and short positions and leveraged exposure, making them a distinct and economically important setting for quantitative methods. They provide the second trading task after the spot position-control study of Chapter 4.

Reinforcement learning has been used across many trading tasks [61, 62], but leveraged perpetual futures add account mechanics and asymmetric loss consequences that a spot-market design does not automatically address. Two motivating issues are illustrated in Figure 5.1.

- **Leverage magnifies reward variation.** Leverage does not change the uncertainty of the underlying market transition; it scales the resulting account

gains and losses. Noisy price changes therefore produce larger-magnitude rewards and more unstable value targets, while the one-dimensional embedding in Figure 5.1(a) offers only an illustrative view of that variation.

- **The policy has no direct signal of distributional familiarity.** A policy fitted on one historical distribution may be unreliable on unfamiliar states [86], and leverage can turn such an error into liquidation. The cluster relocation in Figure 5.1(b) motivates a familiarity signal, but neither that projection nor a VAE score by itself establishes policy competence or a causal macroeconomic explanation.

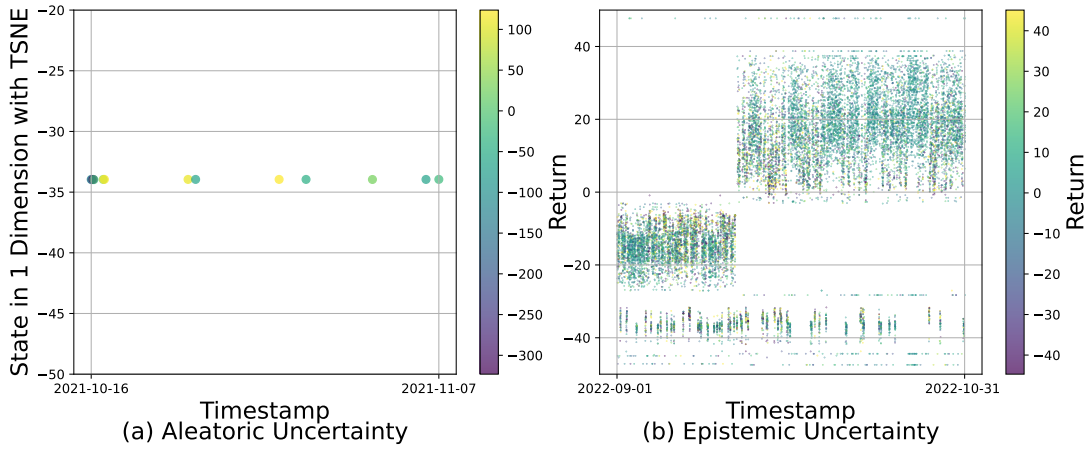


FIGURE 5.1: The two challenges RL faces in futures trading. The horizontal axis is time; the vertical axis is a one-dimensional t-SNE [87] embedding of the market state y_t ; colour encodes the future five-minute return. (a) Aleatoric uncertainty: state values that lie close in the embedding yield widely varying future returns. (b) Distribution shift: a state cluster abruptly relocates during a macro event. The embedding is a one-dimensional t-SNE projection of the roughly 300-dimensional market state: proximity in the projection is illustrative only—it does not establish that the underlying states are nearly identical—and the relocation of a cluster does not by itself establish a causal attribution to the macro event.

This chapter presents FineFT, an efficient and risk-aware ensemble reinforcement-learning method for futures trading, with three stages summarised in Figure 5.2.¹ In the comparative vocabulary of Chapter 1, it constructs differentiated ensemble members and aggregates them with a reconstruction-score rule; the rule is intended as risk control, not calibrated self-knowledge.

¹This chapter has been published in the *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD’26)*.

1. **Stage I (selective-update ensemble).** We initialise N_e deep Q-networks, pre-train them on demonstrations, and then update them *selectively*: for each transition we update only the learner with the smallest temporal-difference error and its neighbours. This mechanism is intended to produce diverse specialists without training each network independently.
2. **Stage II (filter and familiarity model).** We filter the ensemble by profitability under each market dynamic to shrink the pool, and train a variational autoencoder per dynamic as a proxy for distributional familiarity.
3. **Stage III (risk-aware routing).** Guided by VAE losses on recent states, we route to the specialist with the highest familiarity score or invoke a drawdown-conditioned fallback when all scores are low.

On crypto futures with $5\times$ leverage, FineFT is evaluated against twelve state-of-the-art baselines on six metrics. Averaged across the four markets, FineFT records an MDD of 19.46%, compared with 33.82% for RAQR in the complete final baseline set—a reduction of 42.5%—while attaining the highest average total return among the learned methods. This aggregate result is not uniform across individual markets: RAQR has lower MDD on DOT and ETH, while MHFT has higher return on DOT.

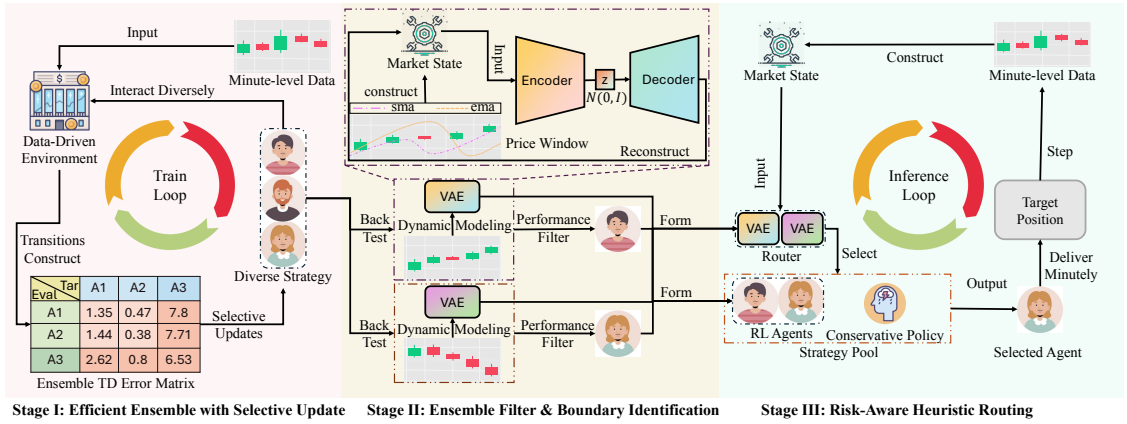


FIGURE 5.2: Overview of the three stages of FineFT. Stage I computes ensemble TD errors for selective updates. Stage II filters the ensemble by performance across dynamics and trains a VAE per dynamic as a distributional familiarity model. Stage III routes among the ensemble and a drawdown-conditioned fallback.

5.2 Problem Formulation

Beyond the concepts of Section 2.1.1, futures trading introduces *leverage* L_t , the ratio of position value to required margin; the *mark price* M_t ; a *market order loss* O_t that captures slippage and commission; a *funding fee* $F_{f,t} = F_{r,t}M_tH_t$ on the signed mark-price notional, where the funding rate $F_{r,t}$ is set by the exchange from the spot–perpetual price gap; and the *margin balance* $V_t = W_t + U_t$, the wallet balance W_t plus unrealised profit and loss U_t . Commission, slippage, and funding payments are realised in the wallet. The released simulator reads the next funding-settlement timestamp from each market-data observation rather than generating settlements from a hard-coded four- or eight-hour interval. The objective is to maximise the margin balance via leveraged market orders on a single asset at a five-minute decision interval.

5.2.1 A Dynamic-Parameter Markov Decision Process

To make the non-stationarity explicit, we model futures trading as a dynamic-parameter MDP (DP-MDP) [77], the tuple $(\mathcal{S}, \mathcal{A}, P, r, \gamma, T, Z, P_z)$ that augments the MDP of Section 2.2.1 with a *dynamic* space Z and its transition P_z . The dynamic z governs the transition $P : \mathcal{S} \times Z \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ but is not revealed to the agent, and it evolves much more slowly than the state. Crucially, this means that *over a short horizon the experienced transitions behave as those of a stationary MDP*—the assumption that legitimises training a specialist per dynamic.

The goal is to learn a set of specialist policies $\Pi = (\pi_{\theta_1}, \dots, \pi_{\theta_{n_z}})$ and router parameters θ_r . Writing π_{Π, θ_r} for their composition, a horizon of T transitions gives

$$(\Pi^*, \theta_r^*) = \arg \max_{\Pi, \theta_r} \mathbb{E}_{\pi_{\Pi, \theta_r}} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right],$$

$$\pi^* = \pi_{\Pi^*, \theta_r^*}.$$

The state s_t consists of around 300 technical indicators y_t and the position H_t ; the reward is the margin-balance change

$$r_t = V_{t+1} - V_t = H_t(M_{t+1} - M_t) - O_t - \mathbb{I}_{t+1}^{\text{fund}} F_{f,t+1}, \quad (5.1)$$

where $\mathbb{I}_{t+1}^{\text{fund}}$ indicates a funding settlement. Thus funding enters both the account state and the learning reward, rather than affecting liquidation alone. The released online environment and the teacher utility use adjacent mark-price indices for the funding notional, an implementation-level indexing discrepancy; (5.1) states the accounting shared by both paths. The action is a target (position, leverage) pair drawn from a finite pool and submitted as a market order. Separately, the online simulator applies the exchange maintenance-margin rule $V_t \leq k|M_t H_t| - j$; this liquidation test is not part of the hindsight teacher described next.

5.3 Method

5.3.1 Stage I: Efficient Ensemble with Selective Update

A mixture of experts performs well on trading tasks but is expensive to construct, because every specialist is trained independently [59, 61]. FineFT instead trains the ensemble jointly: all learners share a pre-training phase that installs common knowledge, after which each transition updates only the best-fitting learner and its neighbours. The aim is an ensemble in which each learner masters one dynamic z of the DP-MDP.

Each of the N_e learners is a multilayer perceptron with identical architecture and a distinct random initialisation drawn after setting one fixed seed for the run; the learners are assigned indices $1, \dots, N_e$.

Pre-training with demonstrations. As new traders are trained together before specialising, we first train all learners on the same demonstration transitions. The released implementation constructs the local transition table of Algorithm 4. Its indices are time t , current action p , and next action a ; for every (t, p) pair it reinitialises a notional account with wallet 10^5 and zero unrealised PnL. It therefore does not propagate a path-dependent margin balance through the recursion. A penalty of 10^{10} is assigned only when a transition reverses the sign of the position in one step or when the requested target position and leverage cannot be executed under the account and order-book constraints. This differs from the published pseudocode, which tests $V_t \leq C$: neither that capital test nor a maintenance-margin

test occurs in the released teacher code. Three auxiliary policies—emptying, maximally long, and maximally short—add diverse transitions. The implementation instantiates the table twice: the value supervisor uses $\gamma = 0.9$, whereas the separate table whose greedy actions form the “perfect” demonstration trajectory uses $\gamma = 1$. The TD target also uses $\gamma = 0.9$. We write the local teacher as $\tilde{Q}$, reserving Q^* for an optimal action-value function. Each learner is updated with a Huber TD error [88], robust to the outliers leverage produces, plus KL supervision:

$$\begin{aligned}\mathcal{L}_{\text{pre}}(\theta_i) &= \mathcal{L}_{\text{TD},i} + \alpha_{\text{KL}} \text{KL}\left(\text{softmax}(\tilde{Q}(s, \cdot)) \parallel \text{softmax}(Q(s, \cdot; \theta_i))\right), \\ \mathcal{L}_{\text{TD},i} &= \mathcal{H}\left(r + \gamma \max_{a'} Q(s', a'; \theta'_i) - Q(s, a; \theta_i)\right),\end{aligned}\tag{5.2}$$

where $\mathcal{H}$ is the Huber loss, quadratic for small errors and linear for large ones, θ'_i denotes the parameters of learner i ’s target network, and α_{KL} is the decaying supervision coefficient of Chapter 4. PyTorch’s `kl_div` call receives the student’s log-probabilities as its input and the teacher distribution as its target, so the implemented direction is $\text{KL}(\text{teacher} \parallel \text{student})$ as written above. Both action-value vectors are converted to distributions by an action-wise softmax with unit temperature; by its shift-invariance, (4.2), the term transfers action rankings while remaining blind to a common shift in value level. The total pre-training loss sums (5.2) over all learners.

Selective update with neighbours. Although dynamics share basic principles, their most profitable strategies conflict; diversity therefore matters. Prior work obtains diversity by training learners independently with preference sampling [61], at high cost. FineFT instead shares transitions selectively. For each transition we identify the learner with the smallest TD error and assign larger update weight to that learner and to its index-neighbours. The smallest TD error is used as a *heuristic compatibility score*: a small TD residual does not in general certify the most accurate value estimate, but it is cheap to compute and indicates that the transition is consistent with the learner’s current estimates. Index adjacency likewise carries no market semantics of its own—the learner ordering is arbitrary—so sharing updates with index-neighbours is a heuristic coupling whose practical effect is to allocate a small group of learners, rather than a single one, to each dynamic. Learners that fit poorly on this transition are left untouched, because their competence is presumed to lie in a different dynamic than the transition

Algorithm 4 Local Feasible-Transition Action-Value Teacher

```

1: Input: LOB series  $B$ , mark-price and funding series of length  $N$ , action space  $A$ 
   inf feasibility penalty  $P_{\text{inf}}$ , discount  $\gamma$ 
2: Output: local teacher table  $\tilde{Q}$ 
3: Initialise  $\tilde{Q}$  with shape  $(N, |A|, |A|)$  and all elements 0; slice  $N - 1$  is terminal.
4: for  $t \leftarrow N - 2$  down to 0 do
5:   for  $p \leftarrow 1$  to  $|A|$  do
6:     for  $a \leftarrow 1$  to  $|A|$  do
7:       Map  $p, a$  to target positions and leverages  $(H(p), L(p))$  and  $(H(a), L(a))$ .
8:       if  $H(p)H(a) < 0$  then
9:          $\tilde{Q}[t, p, a] \leftarrow -P_{\text{inf}}$   $\triangleright$  one-step sign reversal is forbidden
10:      else
11:        Reinitialise  $W \leftarrow 10^5$ ,  $U \leftarrow 0$ , and  $I \leftarrow |M_t H(p)/L(p)|$ .
12:        Simulate  $p \rightarrow a$  against  $B_t$  to obtain the executed target  $\hat{a}$  and local reward  $r_t(p, a)$ .
13:        if  $\hat{a} \neq a$  then
14:           $\tilde{Q}[t, p, a] \leftarrow -P_{\text{inf}}$   $\triangleright$  target is not executable
15:        else
16:           $\tilde{Q}[t, p, a] \leftarrow r_t(p, a) + \gamma \max_{a'} \tilde{Q}[t + 1, a, a']$ 
17:        end if
18:      end if
19:    end for
20:  end for
21: end for
22: return  $\tilde{Q}$ 

```

would only corrupt. This is designed to create a positive feedback loop: as a learner specialises in a dynamic, its TD error there falls, so future transitions from that dynamic are increasingly routed to it, which reinforces the specialisation.

Concretely, for each transition we form an ensemble TD-error matrix $\mathbf{E}$ with entries

$$E_{ij} = \mathcal{H}\left(r + \gamma \max_{a'} Q(s', a'; \theta'_j) - Q(s, a; \theta_i)\right), \quad (5.3)$$

the cross-learner Huber TD error (E_{ii} is learner i 's own error). Algorithm 5 converts $\mathbf{E}$ into a weight matrix $\mathcal{W}$: it locates the best learner $i^* = \arg \min_i E_{ii}$, expands a neighbourhood of radius ν , and assigns diagonal weights that decay with distance from i^* and off-diagonal weights that decay with index difference. For an interior i^* with a full window the diagonal weights decay from one at i^* to one-half at the window edges; when i^* lies within ν of the ensemble boundary, the truncated

window instead drives the farthest retained neighbour’s weight to zero—a boundary asymmetry we keep for simplicity. In the degenerate case $i_{\max} = i_{\min}$ (radius $\nu = 0$ or ensemble size $N_e = 1$) the decay ratio is undefined, and the weight of i^* is set to one directly. The total selective-update loss for a transition is

$$\mathcal{L}_{\text{sel}} = \sum_{i=1}^{N_e} \sum_{j=1}^{N_e} \mathcal{W}_{ij} E_{ij} + \mathbf{d}_{\mathcal{W}} \mathbf{k}_{\text{KL}}^{\top} \quad (5.4)$$

where $\mathbf{d}_{\mathcal{W}} = (\mathcal{W}_{11}, \dots, \mathcal{W}_{N_e, N_e})$ and $\mathbf{k}_{\text{KL}} = (\alpha_{\text{KL}} \text{KL}_1, \dots, \alpha_{\text{KL}} \text{KL}_{N_e})$ are row vectors, and KL_i is learner i ’s teacher-to-student supervision term from (5.2). Thus their matrix product is a scalar and retains the supervision coefficient. The cross-learner terms share experience among learners of similar dynamics while preventing experience poisoning across dissimilar ones.

Heuristic rationale for the selective update. We do not claim a convergence theorem for the selective update; what follows is a mechanism-level rationale, stated together with the assumptions it relies on. In the pre-training phase, every learner is updated with the same local feasible-transition supervision; unlike a full account-state dynamic programme, Algorithm 4 does not represent accumulated wealth or liquidation state. For the selective-update phase, consider transitions from a single dynamic z : by the DP-MDP assumption of Section 5.2.1 they are approximately generated by a stationary MDP M_z , and the update of the best-fitting learner i^* on them is a Huber-TD update under M_z with a non-negative supervision term. For the intended specialisation to follow, several further assumptions are needed: (i) a *fixed assignment* of learners to dynamics, so that i^* does not switch while z is active; (ii) a *tabular* value representation; (iii) a discount $\gamma < 1$; (iv) *non-overlapping TD-error rankings* across dynamics, so that transitions from z are consistently routed to the same learner; and (v) a *monotone decrease* of each specialist’s TD error on its own dynamic, so that the positive feedback loop described above is self-reinforcing. None of these is guaranteed by the algorithm itself: the assignment $i^* = \arg \min_i E_{ii}$ can switch as the networks evolve; a small TD residual need not indicate a small value error; the Huber loss shifts the fixed point away from the optimal action value precisely under the asymmetric, heavy-tailed noise that motivates its use; the cross-learner weights of Algorithm 5 are not normalised—rows

of $\mathcal{W}$ can sum to more than one, so the coupling is not a convex combination—and they pull neighbouring learners towards each other’s bootstrap targets; and the KL supervision draws all learners towards the same local teacher rather than towards per-dynamic optima. The deployed implementation, which uses function approximation, lies outside this tabular idealisation; the result should be read as motivation rather than a guarantee for the deployed algorithm.

Algorithm 5 Construction of the Weight Matrix

```

1: Input: ensemble TD-error matrix  $\mathbf{E} \in \mathbb{R}^{N_e \times N_e}$ , neighbourhood radius  $\nu$ 
2: Output: weight matrix  $\mathcal{W}$ 
3: Initialise  $\mathcal{W}$  with shape  $(N_e, N_e)$  and all elements 0.
4:  $i^* \leftarrow \arg \min_i E_{ii}$ ;  $i_{\min} \leftarrow \max(i^* - \nu, 1)$ ;  $i_{\max} \leftarrow \min(i^* + \nu, N_e)$ 
5: if  $i_{\max} = i_{\min}$  then ▷ degenerate window ( $\nu = 0$  or  $N_e = 1$ )
6:    $\mathcal{W}_{i^*i^*} \leftarrow 1$ ; return  $\mathcal{W}$ 
7: end if
8: for  $i = i_{\min}$  to  $i_{\max}$  do
9:    $\mathcal{W}_{ii} \leftarrow 1 - \frac{|i - i^*|}{i_{\max} - i_{\min}}$  ▷ diagonal decay
10: end for
11: for  $i, j \leftarrow 1$  to  $N_e$  do
12:   if  $i \neq j$  then
13:      $\mathcal{W}_{ij} \leftarrow \min(\mathcal{W}_{ii}, \mathcal{W}_{jj}) \left(1 - \frac{|i - j|}{i_{\max} - i_{\min}}\right)$  ▷ off-diagonal decay
14:   end if
15: end for
16: return  $\mathcal{W}$ 

```

5.3.2 Stage II: Ensemble Filtering and Familiarity Models

After selecting a learner for each inferred dynamic, FineFT requires a signal indicating whether the current state resembles the states associated with that learner.

Filtering by profitability. We divide the market into dynamics by slope, following Chapter 4: the series is low-pass filtered, segmented at local extrema, and adjacent chunks with similar slope are merged until stable, yielding n_z dynamics. We back-test each learner across initial positions and retain, for each dynamic, the learner with the highest average profitability, producing a small, diverse, profitable pool.

Distributional familiarity proxy. The paper uses the term *capability boundary* for the support of a dynamic’s estimated market-state distribution. Operationally, the released pipeline constructs this proxy from labelled validation segments and holds it fixed during test; the stage-III threshold is also tuned on validation. Reconstruction quality is a property of the state distribution rather than the policy: a well-reconstructed state need not be one on which the specialist performs well. Because the evaluation does not measure the association between VAE scores and held-out policy loss, the quantity is treated here as a distributional familiarity proxy rather than a calibrated competence set.

Familiarity scoring with VAEs. We train a variational autoencoder [35] on the market-state representations of each dynamic. Its reconstruction loss is used as a heuristic out-of-distribution score. For n_z dynamics we train n_z VAEs, each on the state set Y^i of its dynamic, minimising

$$\begin{aligned} \mathcal{L}_{\text{VAE}}^i(y) = & \frac{1}{2} \sum_{d=1}^D \left[\frac{(y_d - \mu_d^{\text{dec}})^2}{(\sigma_d^{\text{dec}})^2} + \log((\sigma_d^{\text{dec}})^2) + \log(2\pi) \right] \\ & + \text{KL}(\mathcal{N}(\mu^{\text{enc}}, \text{diag}((\sigma^{\text{enc}})^2)) \parallel \mathcal{N}(0, I)), \quad y \in Y^i. \end{aligned} \quad (5.5)$$

the sum of the D decoder negative log-likelihood terms and the standard-normal latent regulariser. Here μ_d^{dec} and $(\sigma_d^{\text{dec}})^2$ are the decoder’s coordinate-wise output mean and variance, while μ^{enc} and $(\sigma^{\text{enc}})^2$ are the encoder parameters. The released implementation deviates from the Gaussian reparameterisation assumed by this objective: it draws the latent perturbation with `torch.rand_like`, hence from $\text{Uniform}[0, 1]$, rather than with a standard-normal draw, and repeats this stochastic forward pass while scoring deployment states. This is an implementation limitation of the reported VAE scores. After fitting, the pipeline evaluates the negative loss on the same validation-segment file used for VAE training and stores those in-sample values as the reference-score set $\mathcal{R}^i$; it does not use an independent calibration set.

5.3.3 Stage III: Risk-Aware Heuristic Routing

Conservative policy. When the router’s maximum familiarity score is below its threshold, the released policy retains the current action while the peak-to-trough drawdown of the current non-flat holding episode is below 5% and invokes the

rule-based closing action once the drawdown reaches 5%. It does not open a new position in this fallback branch. In particular, an unfamiliar state does not cause an immediate exit: an existing leveraged position can remain open until the holding-episode drawdown threshold is reached.

Routing with VAE losses. Rather than fit another policy, FineFT routes by a fixed rule. Given the u most recent states $Y_T^u = (y_{T-u+1}, \dots, y_T)$, each state’s score under dynamic i is its empirical-CDF position among the reference scores $\mathcal{R}^i$. The implementation smooths these scores with a *normalised, finite-window* exponentially weighted average:

$$\eta_t^i = F_{\mathcal{R}^i}(-\mathcal{L}_{\text{VAE}}^i(y_t)), \quad \bar{\eta}_T^{i,u} = \frac{\sum_{k=0}^{u-1} \rho^k \eta_{T-k}^i}{\sum_{k=0}^{u-1} \rho^k}, \quad (5.6)$$

where $F_{\mathcal{R}^i}$ is the empirical CDF of the reference-score set, $\rho \in (0, 1)$, and the newest observation has weight one. The first u environment steps fill the score deque while the rule-based closing policy acts. Thereafter, if $\max_i \bar{\eta}_T^{i,u} < \tau_{\text{risk}}$, the fallback logic above is used; otherwise the specialist $i^* = \arg \max_i \bar{\eta}_T^{i,u}$ acts. Equality therefore selects a specialist. The three routing parameters $(u, \rho, \tau_{\text{risk}})$ are tuned jointly on the validation return of this complete max-score routing rule, so the maximisation is already represented in the tuning objective. The remaining concern is validation reuse: the same window supports specialist filtering, VAE fitting, in-sample ECDF references, and router tuning, which can overfit model selection to that period. This is the second instantiation of the thesis’s aggregation mechanism—a heuristic driven by generative distribution scores rather than a learned router.

5.4 Experimental Setup

Datasets. We evaluate on four mainstream cryptocurrencies over two years, covering bull, bear, and sideways conditions (Table 5.1). The underlying trades, quotes, derivatives tickers, and order-book snapshots were obtained from Tardis, which offers trial access and commercially available extended historical coverage.²

²<https://tardis.dev/>

Algorithm 6 Risk-Aware Heuristic Routing

```

1: Input: specialist pool  $\Pi$ , VAE models  $\{\mathcal{V}^i\}_{i=1}^{n_z}$ , reference sets  $\{\mathcal{R}^i\}_{i=1}^{n_z}$ 
   state window  $Y_T^u$ , decay  $\rho$ , threshold  $\tau_{\text{risk}}$ 
2: Output: current strategy  $\pi$ 
3: for dynamic  $i = 1$  to  $n_z$  do
4:   Obtain state scores  $\eta_t^i$  from  $\mathcal{V}^i$ ,  $Y_T^u$ , and  $\mathcal{R}^i$ .
5:   Compute the normalised finite-window score  $\bar{\eta}_T^{i,u}$  by (5.6).
6: end for
7:  $i^* \leftarrow \arg \max_i \bar{\eta}_T^{i,u}$ 
8: if  $\bar{\eta}_T^{i^*,u} < \tau_{\text{risk}}$  then
9:   return conservative policy  $\pi^c$  ▷ out-of-distribution
10: else
11:   return specialist  $\pi^{i^*}$ 
12: end if

```

The raw streams are downsampled to a uniform five-minute frequency. The published protocol uses the first year for training, the next six months for validation, and the final six months for testing.

Following the published protocol, market features aligned by timestamp are augmented with rolling indicators. Two filters then reduce the feature set: XGBoost importance removes low-importance variables, and Pearson correlation removes redundant ones. The resulting market state has about 300 dimensions. The validation period is used for ensemble filtering, VAE fitting, score calibration, and routing-hyperparameter selection.

TABLE 5.1: The four cryptocurrency futures datasets, with the reported test-period dynamic, total number of observations before splitting, and chronological span. Dates are in YY/MM/DD format.

Dataset	Test dynamic	Size	From	To
BNB/USDT	Bear	210,241	22/01/01	24/01/01
BTC/USDT	Bull	210,241	22/01/01	24/01/01
DOT/USDT	Sideways	210,241	22/01/01	24/01/01
ETH/USDT	Sideways	210,241	22/01/01	24/01/01

Baselines. We compare against twelve methods: two policy-based (PPO [22], and DRA, an LSTM-based PPO variant [5]); two value-based (DQN [19], and CDQNR [6], the same method that appears under this name in Chapter 4); two rule-based (MACD [11], IV [85]); two ensemble (EDQN, an ensemble DQN in the

style of Averaged-DQN [24], and SUNRISE [25]); one distributional (RAQR, a risk-averse quantile-regression DQN [23]); and three hierarchical or routing-based methods. The last group contains a performance-based trading adaptation named after the Winnow linear-threshold algorithm [60], EHFT (EarnHFT, Chapter 4) [61], and MHFT (MacroHFT) [62]. The same hyperparameters used by FineFT are applied to the baselines where applicable.

Training. Experiments run on four RTX 4090 GPUs, completing in about six hours. The transaction cost is 0.02%, following Binance, with a leverage factor of 5. FineFT’s risk and risk-adjusted metrics are computed from daily returns obtained by grouping the 288 five-minute observations in a day, with annualisation constant $m = 365$. We report the six metrics of Section 2.1.3. Each configuration is represented by one fixed-seed training run, and routing hyperparameters are tuned separately for each market on validation data. The published evaluation additionally applies pairwise Wilcoxon signed-rank tests to daily aggregates across the test periods; those tests assess differences between daily outcome distributions but do not quantify training-seed variance.

5.5 Results and Analysis

5.5.1 Comparison with Baselines

Table 5.2 reports the main comparison. FineFT has the highest average total return and risk-adjusted ratios and the lowest average MDD: 19.46% versus 33.82% for RAQR in the complete baseline set, a relative reduction of 42.5%. FineFT also attains the highest risk-adjusted ratios on three of the four individual markets, but it is not uniformly best: RAQR has lower MDD on DOT and ETH, and MHFT has higher return on DOT.

The remaining comparisons are descriptive rather than causal. CDQNRP exceeds DQN on several cells, but a single backtest does not isolate training stability as the cause. The small DRA–PPO differences do not by themselves identify the effect of the LSTM encoder, and differences between rule-based, ensemble, distributional, and hierarchical methods are confounded by their policies and tuning procedures.

FineFT's combination of high average return and lower average drawdown is consistent with the intended familiarity-gated fallback, but the aggregate table alone cannot attribute the improvement to that component; the component evidence is considered below.

TABLE 5.2: Performance comparison on four crypto futures markets against twelve baselines (four plain, two ensemble, one distributional, three hierarchical or routing-based, and two rule-based). Arrows indicate the preferred direction; the **best** entry per market and metric is in bold (colour retained for screen reading). TR, annual volatility (AVOL), and MDD are in percent. AVOL is corrected relative to the conference version, which reported the unannualised standard deviation of daily returns: every AVOL entry here is the published value multiplied by $\sqrt{365}$; all other entries are unchanged. The Average block is the arithmetic mean of the four per-market values; ratio metrics are averaged directly, so it is a cross-market summary rather than a portfolio-level statistic.

Market	Model	TR $\uparrow$	ASR $\uparrow$	ACR $\uparrow$	ASoR $\uparrow$	AVOL $\downarrow$	MDD $\downarrow$
BNB	DRA	-64.79	-1.51	-2.58	-13.43	124.76	72.88
	PPO	-59.58	-1.64	-2.59	-14.12	106.03	67.37
	CDQNR	-64.79	-1.43	-2.59	-12.98	132.21	72.88
	DQN	-55.85	-1.21	-2.14	-13.72	116.35	66.09
	MACD	-82.85	-3.04	-4.12	-22.67	119.98	88.70
	IV	-71.00	-1.59	-2.73	-14.31	125.90	73.21
	EDQN	-64.79	-1.51	-2.58	-13.43	124.76	72.88
	SUNRISE	-66.75	-1.74	-2.80	-14.45	119.79	74.32
	RAQR	-68.88	-1.84	-3.10	-14.22	123.61	73.27
	Winnow	86.65	2.01	5.99	31.71	104.31	34.93
	EHFT	-69.32	-2.08	-3.26	-17.31	115.59	73.61
	MHFT	-50.42	-0.81	-1.48	-9.10	123.61	67.95
	FineFT	83.70	2.55	11.19	101.17	68.20	15.49
BTC	DRA	21.93	0.94	2.12	4.90	131.82	58.28
	PPO	19.68	0.90	2.01	4.39	130.49	58.30
	CDQNR	19.61	0.85	1.79	4.62	123.04	58.30
	DQN	-82.13	-12.03	-4.94	-51.81	33.82	82.37
	MACD	-26.28	-0.17	-0.31	-9.44	112.91	60.17
	IV	-17.26	0.01	0.02	-4.26	110.81	63.25
	EDQN	25.11	0.98	2.15	5.88	129.91	59.28
	SUNRISE	-7.91	0.37	0.76	-2.06	131.25	63.36
	RAQR	2.84	0.57	1.10	0.71	99.15	51.19
	Winnow	-34.03	-0.23	-0.56	-8.62	135.45	55.30
	EHFT	33.44	1.42	2.73	10.78	81.96	42.61
	MHFT	-25.85	-0.32	-0.57	-7.27	85.21	47.97
	FineFT	76.90	2.13	7.44	39.51	81.96	23.51
DOT	DRA	-65.15	-1.63	-2.63	-15.21	118.45	73.36
	PPO	-65.65	-1.63	-2.64	-15.18	119.79	73.80
	CDQNR	-73.13	-4.12	-4.09	-27.08	75.08	75.58
	DQN	-64.89	-1.35	-2.40	-13.66	130.68	73.42
	MACD	26.07	1.10	2.72	6.24	147.87	60.01
	IV	-66.24	-1.67	-2.70	-15.32	119.60	73.80
	EDQN	-65.65	-1.63	-2.64	-15.18	119.79	73.80
	SUNRISE	-59.18	-1.02	-1.84	-12.51	132.59	73.60
	RAQR	-4.45	-4.47	-1.76	-45.61	1.91	4.72
	Winnow	-81.86	-4.17	-4.63	-26.72	92.66	83.35
	EHFT	89.00	1.95	4.09	19.78	119.98	57.18
	MHFT	220.44	3.48	10.47	65.02	97.44	32.44
	FineFT	18.39	1.08	2.50	14.46	50.44	21.87

TABLE 5.2: FineFT main results (continued): ETH and the four-market arithmetic average.

Market	Model	TR $\uparrow$	ASR $\uparrow$	ACR $\uparrow$	ASoR $\uparrow$	AVOL $\downarrow$	MDD $\downarrow$
ETH	DRA	-46.56	-0.77	-1.44	-10.86	118.07	63.20
	PPO	-46.08	-0.74	-1.39	-10.68	119.02	63.01
	CDQNRP	-46.06	-0.95	-1.69	-11.36	112.34	62.96
	DQN	-73.15	-3.94	-3.88	-27.31	74.32	75.48
	MACD	-73.93	-2.78	-3.49	-21.52	101.26	80.85
	IV	-50.53	-1.04	-1.82	-12.29	112.15	63.87
	EDQN	-46.00	-0.73	-1.39	-10.65	119.02	62.96
	SUNRISE	-81.22	-4.45	-4.57	-23.85	85.59	83.36
	RAQR	-5.94	-4.10	-2.65	-13.75	4.01	6.08
	Winnow	26.36	1.06	3.55	6.98	125.90	37.55
	EHFT	48.72	1.49	3.33	10.66	107.94	48.33
	MHFT	-75.16	-2.21	-3.26	-15.37	120.74	81.79
	FineFT	39.05	1.74	5.78	14.45	56.55	16.96
Average	DRA	-38.64	-0.74	-1.13	-8.65	123.23	66.93
	PPO	-37.91	-0.78	-1.15	-8.90	118.83	65.62
	CDQNRP	-41.09	-1.41	-1.65	-11.70	110.62	67.43
	DQN	-69.00	-4.63	-3.34	-26.62	88.84	74.34
	MACD	-39.25	-1.22	-1.30	-11.85	120.55	72.43
	IV	-51.26	-1.07	-1.81	-11.54	117.11	68.53
	EDQN	-37.83	-0.72	-1.11	-8.35	123.42	67.23
	SUNRISE	-53.77	-1.71	-2.11	-13.22	117.30	73.66
	RAQR	-19.11	-2.46	-1.60	-18.22	57.12	33.82
	Winnow	-0.72	-0.33	1.09	0.84	114.63	52.78
	EHFT	25.46	0.69	1.72	5.98	106.41	55.43
	MHFT	17.25	0.03	1.29	8.32	106.80	57.54
	FineFT	54.51	1.88	6.73	42.40	64.38	19.46

5.5.2 Effectiveness of the Selective-Update Ensemble

Table 5.3 compares FineFT’s pre-trained selective update (FP) against three alternatives on five dynamics: EHFT-style prioritised episode selection (EP), random dataset selection (ER), and FineFT without pre-training (FwoP). Convergence steps (CS) measure efficiency and reward sum (RS) the converged performance. FP and FwoP achieve the strongest converged performance, while EP and FP converge in the fewest steps; overall, FP combines the fewest convergence steps with converged performance on par with the best, which supports sharing transitions selectively—rather than training each learner independently—as improving both efficiency and the performance ceiling.

TABLE 5.3: Convergence and performance of the selective-update ensemble against three training schemes on five dynamics: EHFT prioritised episode selection (EP), random selection (ER), FineFT with pre-training (FP), and without pre-training (FwoP). CS is convergence steps; RS is reward sum. The **best** and second-best entries are marked in bold and underline respectively (colour retained for screen reading).

Method	CS↓	Dyn	RS↑	Method	CS↓	Dyn	RS↑
EP	<u>71712</u>	1	2.86	ER	112320	1	2.86
		2	2.14			2	2.15
		3	0.50			3	0.48
		4	4.98			4	4.98
		5	9.14			5	9.14
FP	66528	1	45.29	FwoP	79488	1	<u>37.34</u>
		2	18.21			2	<u>16.18</u>
		3	<u>4.77</u>			3	6.39
		4	<u>24.48</u>			4	25.53
		5	<u>40.23</u>			5	46.17

5.5.3 Effectiveness of Risk-Aware Routing and Case Studies

The risk-aware router is designed to reduce exposure when recent states receive low familiarity scores. Figure 5.3 shows the routing proportions on BTC over a window described in the published account as spanning the validation–test boundary: the conservative branch dominates and its share rises over the displayed interval. This temporal association is compatible with the intended mechanism, but it does not

identify why performance changes. Two diagnostics provide further context. First, Figure 5.4 relates the updated learner index to market dynamics in a seven-agent run; the visible clustering suggests specialisation, although it is not a controlled test of the selective-update rule. Second, Figure 5.5 shows a BNB exit followed by a price decline near an on-chain event. Because the figure omits the score, threshold, selected policy, and counterfactual action, it cannot establish either that the VAE caused the exit or that the exit caused an avoided loss. The published appendix supplies with- and without-threshold plots for the four markets but no numerical component table. The evidence therefore supports a qualitative mechanism illustration rather than a causal estimate of the router’s contribution to Table 5.2.

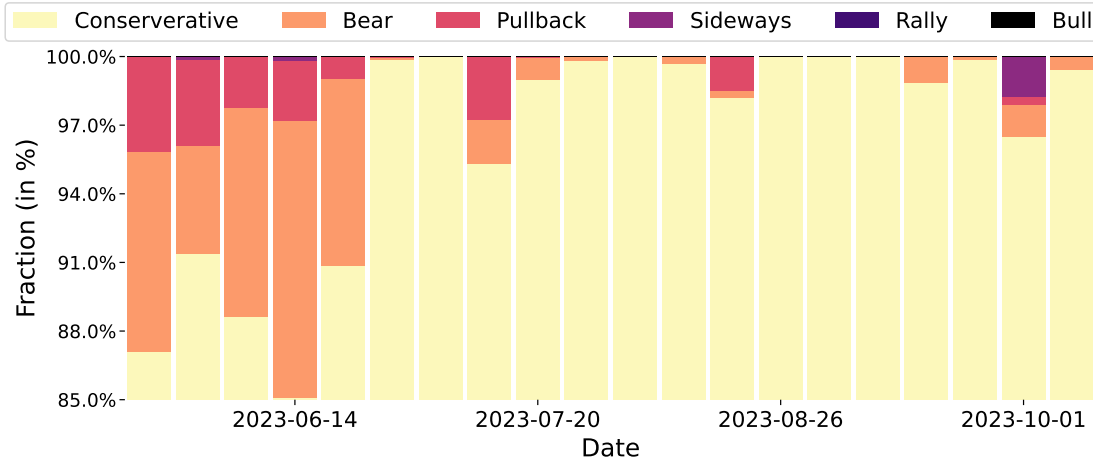


FIGURE 5.3: Routing decisions on BTC futures from early June to early October 2023, spanning the validation–test boundary reported in the paper. Each bar shows the fraction routed to the conservative policy and to each specialist; the vertical axis is truncated at 85% for readability.

5.5.4 Parameter Sensitivity and Computational Cost

The two hyperparameters that most directly shape FineFT’s profit–risk trade-off are the size of the specialist pool and the drawdown threshold of the conservative policy. Table 5.4 varies the pool over three, five, and seven agents while holding the drawdown threshold at its canonical 5%; it then varies that threshold over 1%, 5%, and 10% while holding the pool at five agents. This grid is a *post-hoc* analysis evaluated on the BTC futures test window, after the canonical configuration had been fixed on the validation window; because it touches the test window, it must not be read as evidence of robustness or used to inform design choices. We report

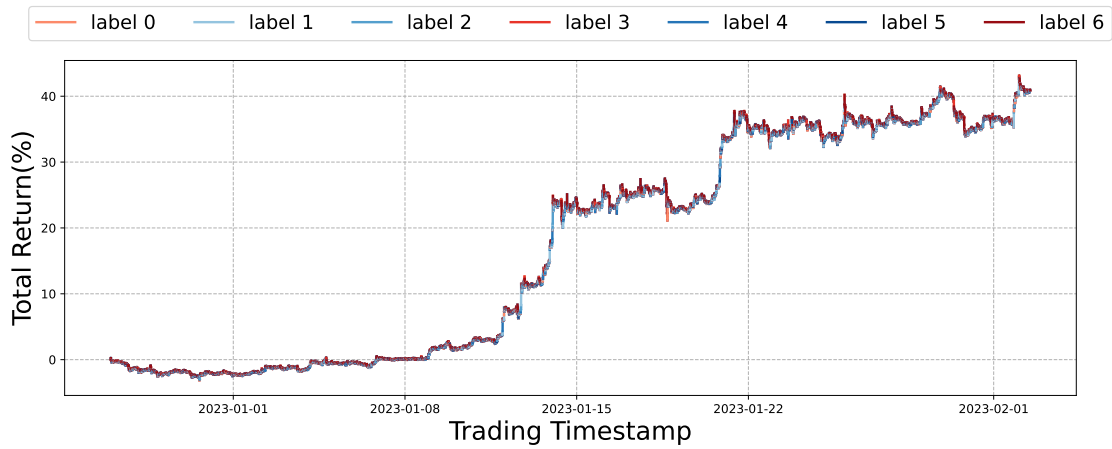


FIGURE 5.4: BTC return curve coloured by the learner index updated in a seven-agent diagnostic run (legend labels 0–6). The horizontal axis is time and the vertical axis cumulative return. The clustering of colours across price dynamics is compatible with specialisation, but this illustrative diagnostic is not a controlled component estimate.

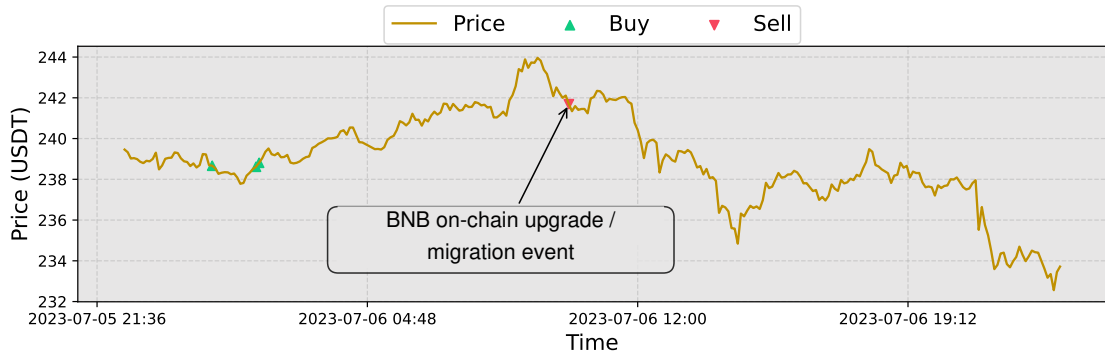


FIGURE 5.5: VAE-driven risk control on BNB. The horizontal axis is time and the vertical axis price; red and green markers are FineFT’s sell and buy signals; the callout marks a BNB on-chain upgrade/migration event. The figure plots only the price series and trading signals: the rolling-window VAE score, the risk threshold, and the identity of the selected policy are not shown, so it illustrates the timing of the exit relative to the event rather than verifying the detection mechanism itself.

it only as a descriptive account of how performance would have varied, and we flag the test-window evaluation itself as a protocol limitation.

In this BTC run, five agents outperform both three and seven; this pattern is compatible with a diversification–specialisation trade-off, but the grid does not isolate data fragmentation as its cause. Tightening the drawdown threshold from 5% to 1% lowers MDD slightly and reduces return, whereas raising it to 10% increases MDD substantially. The response is not monotone in every metric, and

TABLE 5.4: Sensitivity of FineFT to the number of specialists and the draw-down threshold (DDT) of the conservative policy. The canonical configuration (5 agents, 5% DDT) is the one reported in Table 5.2; the other rows vary one axis at a time. TR is total return, SR the annualised Sharpe ratio (the ASR of Section 2.1.3), MDD maximum drawdown. The grid is evaluated post hoc on the BTC test window (see the text). The **best** entry per column is in bold (colour retained for screen reading).

#Agents	DDT	TR(%) $\uparrow$	SR $\uparrow$	MDD(%) $\downarrow$
3	5%	52.74	1.25	39.10
5	5%	76.90	2.13	23.51
7	5%	59.63	1.46	37.50
5	1%	53.82	1.38	22.96
5	5%	76.90	2.13	23.51
5	10%	71.32	1.64	41.85

the alternatives were evaluated post hoc on the test window. We therefore treat the grid as a descriptive sensitivity check rather than evidence that the canonical configuration is generally optimal or robust.

Computational cost. Training the full FineFT pipeline—selective-update ensemble pre-training, per-dynamic VAE fitting, and validation-window hyperparameter tuning—takes approximately six hours on a server with four NVIDIA RTX 4090 GPUs and an AMD Ryzen Threadripper PRO 5995WX CPU. At inference time the cost is dominated by the joint evaluation of one DQN specialist together with the five VAEs that gate it. With PyTorch’s `vmap`, joint inference completes in approximately 0.78ms on a single RTX 4090, only marginally above the 0.28ms of the bare DQN; both are several orders of magnitude below the five-minute decision interval, so the VAE machinery does not constrain the deployment frequency in practice.

5.5.5 Generalisation to Other Asset Classes

The main evaluation is confined to cryptocurrency futures, the setting for which FineFT was designed. The published study additionally reports daily Yahoo Finance experiments on corn and E-mini S&P 500 (ES) futures over 2001–2022, with train/validation/test sizes of 2,994/1,197/1,197 for corn and 2,963/1,185/1,185

for ES. These shorter supplementary experiments compare FineFT with a $5\times$ Buy&Hold reference and a plain DQN; their purpose is to test whether the method’s relative ranking extends beyond cryptocurrency futures.

TABLE 5.5: Published supplementary daily results for a commodity future (corn) and an equity-index future (E-mini S&P 500, ES), against a $5\times$ Buy&Hold reference and a plain DQN. TR is total return, SR the annualised Sharpe ratio, and MDD maximum drawdown; TR and MDD are in percent. The **best** displayed entry per market and metric is in bold.

Market	Model	TR(%) $\uparrow$	SR $\uparrow$	MDD(%) $\downarrow$
Corn	Buy&Hold ($5\times$)	440.71	1.04	91.93
	DQN	503.17	1.27	47.37
	FineFT	730.86	1.74	30.47
ES	Buy&Hold ($5\times$)	288.96	0.88	90.64
	DQN	316.30	1.18	35.31
	FineFT	374.70	1.27	28.74

5.6 Chapter Summary

This chapter presented FineFT, a three-stage risk-aware ensemble RL framework for leveraged futures. A selective-update mechanism trains a pool of specialists, a per-dynamic VAE provides a distributional familiarity proxy, and a heuristic router chooses between a specialist and a drawdown-conditioned fallback. Averaged across the four crypto markets, FineFT records the highest total return and the lowest average MDD, reducing MDD by 42.5% relative to RAQR in the complete baseline set; it is not uniformly dominant on every market. These point estimates are consistent with the intended risk-aware design, but the single-seed evaluation, reuse of validation data across several selection stages, and absence of numerical component outputs limit causal claims. Relative to EarnHFT, FineFT replaces the learned router with a heuristic driven by generative distribution scores and thereby partly decouples aggregation from policy learning. The next chapter moves from trading to signal generation and removes the aggregation mechanism from the learning reward altogether.

Part III

Reinforcement Learning for Signal Generation

Chapter 6

AlphaQD: Quality-Diversity Reinforcement Learning for Formulaic Alpha Mining

6.1 Introduction

The two preceding chapters made trading *decisions*: an agent observed the market and chose how to adjust its position. This chapter changes the task.¹ Rather than acting in the market, the agent must *generate* the predictive signals from which a portfolio is later built. The signals are *formulaic alphas*: short symbolic expressions whose cross-sectional value at each timestamp is treated as a forecast of future returns [17]. Each alpha is a tree of arithmetic, cross-sectional, and time-series operators over a small vocabulary of price and volume features. A portfolio is assembled from a *set* of such alphas.

Formulaic alphas remain the dominant signal-generation paradigm in low-frequency quantitative investment for two reasons that monolithic neural predictors do not satisfy. First, each formula is readable and auditable to risk committees and regulators. Second, a small set of complementary formulas degrades more gracefully than a single black box when the regime shifts. Automating their discovery is, however, a hard exploration problem under three simultaneous pressures: *quality*

¹This chapter is based on the AlphaQD manuscript, which is currently under review.

(the supervisory signal is the information coefficient of a finished formula on noisy data); *diversity* (what is deployed is a set covering different return sources, so a high-quality duplicate is worthless); and *generalisation* (the training window is bounded and the test market is non-stationary).

Diagnosis. The reinforcement-learning line descended from AlphaGen [67] scores each new alpha by its marginal contribution to a running pool. This is structurally a boosting objective, and it has two pathologies, the third gap of Section 3.2. First, the reward changes every time the pool changes, so the policy is trained against a moving target. The policy never sees a stable landscape long enough to commit to coverage, which is especially damaging for exploration. Second, the pool-coupled reward design offers no analogue of the complexity control this chapter seeks: the legal expression space exceeds 10^{20} , so in late rounds the concentration guarantee for the empirical residual maximiser becomes vacuous (Section 6.4). Boosting has a rich generalisation theory of its own—margin bounds among others—but none of it ties the effective hypothesis class of this reward design to the number of retained factors. Figure 6.1 contrasts a pool-dependent AlphaGen-style reward (the correlation-penalised variant of (6.3)) with the static reward AlphaQD uses.

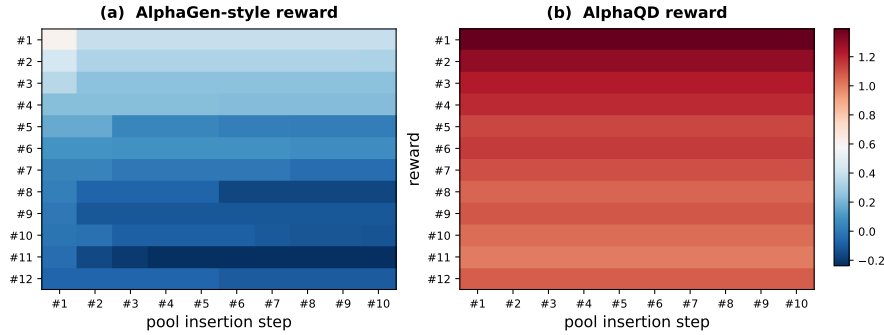


FIGURE 6.1: Per-candidate reward across pool-insertion steps under two reward functions: the pool-dependent correlation-penalised reward of (6.3), an AlphaGen-style objective (left), and the AlphaQD terminal reward (right). Rows are twelve probe alphas; columns are pool-insertion steps; colour encodes the reward (separate scale per panel). The pool-coupled reward shifts as the pool grows; the AlphaQD reward does not.

Remedy. AlphaQD removes this coupling in three stages (Figure 6.2), following the decompose–specialise–aggregate pattern with the aggregation mechanism fully detached from the reward.

1. **Efficient GFlowNet exploration.** A relational-graph GFlowNet samples symbolic expressions and is trained by trajectory balance against a terminal reward that depends *only* on the candidate’s training-window information ratio, so the learning target is invariant to later archive updates.
2. **Return-source MAP-Elites archiving.** Each candidate is back-tested into a daily IC series; its information ratio scores quality, and the normalised IC series is its behaviour descriptor—an empirical proxy for the formula’s return source—placing it in a PCA-discretised MAP-Elites grid [32, 33].
3. **Top- K cell selection.** One high-quality representative per occupied cell is exported, and the top K cells form the deployed pool.

Against eleven baselines on the reported Chinese A-share and U.S. panels, AlphaQD belongs to a competitive group on the signal metrics. On A-shares it attains the highest annualised return and the strongest net Sharpe among the learned formula-mining methods; among the four generative/RL pool-style methods it has the lowest turnover on both panels. The shared protocol supports descriptive within-panel ranking, while the run labels are not controlled random-seed replications (Section 6.5).

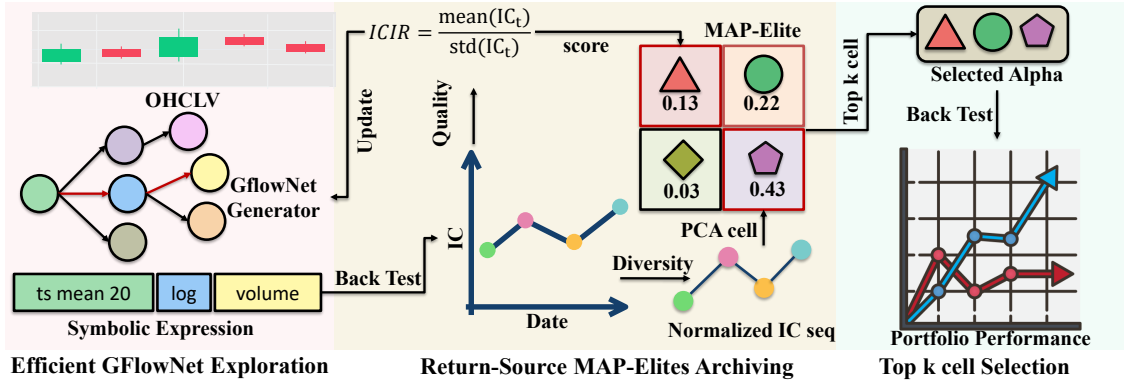


FIGURE 6.2: The three stages of AlphaQD. *Left:* a relational-graph GFlowNet samples a legal symbolic expression from raw inputs and is trained by trajectory balance. *Middle:* the candidate is back-tested into a daily IC series; the information ratio scores quality, the normalised IC series is the return-source descriptor, and the candidate enters a PCA-discretised MAP-Elites grid; the quality score is propagated back to the generator. *Right:* the top K occupied cells export their representatives as the deployed pool, evaluated by a shared ridge back-test.

6.2 Problem Formulation

Let T be the number of trading days in the training window, N the cross-sectional universe size, and $X \in \mathbb{R}^{T \times N \times d}$ the per-asset feature tensor (open, high, low, close, volume, VWAP, and short-window returns). A formulaic alpha is a function $\alpha : \mathbb{R}^{T \times N \times d} \rightarrow \mathbb{R}^{T \times N}$ realised as a reverse-Polish-notation (RPN) token sequence over a typed grammar $\mathcal{G}$, with an action mask rejecting any prefix that cannot complete into a legal formula. Given the h -day forward return $r_{t,i}^{(h)} = \text{close}_{t+h,i} / \text{close}_{t,i} - 1$ (we use $h = 20$), the daily information coefficient and information ratio are, restating the definitions of Section 2.1.3 in this chapter's notation [16],

$$\text{IC}_t(\alpha) = \text{corr}(\alpha_t, r_t^{(h)}), \quad \text{ICIR}(\alpha) = \overline{\text{IC}(\alpha)} / \hat{\sigma}(\text{IC}(\alpha)). \quad (6.1)$$

Mining goal. Let $\mathcal{F}$ be the set of legal formulas. The output is a bounded set $\mathcal{P} \subset \mathcal{F}$ whose members, once combined by a fixed downstream combiner, should predict returns on data not seen during mining. The goal is therefore not the per-formula maximiser $\arg \max_{\alpha \in \mathcal{F}} \text{ICIR}(\alpha)$, which a single high-ICIR formula would solve, but the joint problem

$$\arg \max_{\mathcal{P} \subset \mathcal{F}} \mathbb{E}_{\text{oos}}[\text{IC}(g(\mathcal{P}))] \quad \text{subject to} \quad |\mathcal{P}| \leq K, \quad (6.2)$$

where $g(\cdot)$ is the fixed combiner, K is the pool-size budget, and the expectation is taken over out-of-sample data. The archive-cell map introduced later is denoted separately by $c(\alpha)$. Equation (6.2) is an idealised target rather than an operational objective: no mining procedure can evaluate the out-of-sample expectation, so search must optimise training-window proxies, and the relation between those proxies and (6.2) is precisely the generalisation question that Section 6.4 addresses. The target imposes two structural requirements: each retained alpha must be of high quality, and members must capture different *return sources* so their combination generalises. We use the centred, normalised daily IC series as the empirical proxy for the return source carried by α .

Contrast with AlphaGen. AlphaGen approximates (6.2) by a boosting relaxation: a pool Pool_t is grown one element at a time, and the RL reward is the increment in the combined pool's predictive performance after the combination

model is refitted with the candidate included. Throughout this chapter we work with a correlation-penalised *variant* that captures the pool coupling of this objective in closed form,

$$R_t(\alpha) = \text{ICIR}(\alpha) - \lambda \max_{\alpha_j \in \text{Pool}_t} |\text{corr}(\text{IC}(\alpha), \text{IC}(\alpha_j))|. \quad (6.3)$$

with the correlation penalty defined as zero when Pool_t is empty. Equation (6.3) is a surrogate for—not a restatement of—the original AlphaGen reward, and the analyses and instrumentation that follow (Figure 6.1, Section 6.4) address this variant and the pool-coupled reward class it represents rather than the published system. What matters here is the property the variant shares with the original: every pool update changes $R_t(\cdot)$, so the policy is trained against a moving target.

Static formulation. We retain the per-formula MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma = 1)$, in which s_t is a partial RPN prefix and a_t the next token under the legal-action mask, but we replace (6.3) with a terminal reward that depends only on the candidate and the fixed training window. Diversity is moved out of the reward and into a separate selection rule over a behaviour descriptor of α . The reward function then does not change as the archive grows, restoring stationarity; and the selection rule operates over a bounded number of behaviour cells, yielding—conditional on the realised archive—a generalisation-complexity term that scales with the size of the retained set rather than with $|\mathcal{F}|$ (Section 6.4).

6.3 Method

The three stages exchange candidates and scalar scores but never a pool-dependent reshape of the reward. Algorithm 7 ties them together.

6.3.1 Efficient GFlowNet Exploration

Search surface. A state s_t is the partial RPN prefix and an action a_t the next token; a legal-action mask, derived from stack depth, type consistency, window-parameter constraints, and a terminator rule, confines the search to syntactically

and dimensionally valid expressions. Each prefix is materialised as a typed directed graph—nodes tagged by operator, operand, constant, and window type, edges encoding operator-to-argument relations—and encoded by a two-layer relational graph convolutional network [89] of hidden width 128. The pooled node embeddings feed two heads, $\log P_F(\cdot \mid s_t)$ over legal next tokens and $\log P_B(\cdot \mid s_{t+1})$ over legal predecessors, and a scalar $\log Z$.

Terminal reward. For a completed expression α , AlphaQD evaluates its factor values on the training window and cross-sectionally standardises both the factor and the forward-return target to zero mean and unit population standard deviation on each date. Values that remain non-finite after this operation are replaced by zero. The implementation does not winsorise or rank-normalise the factor before computing Pearson IC; this differs from the winsorisation-and-rank-normalisation description in the original method write-up. It then computes $\text{IC}(\alpha)$ and $\text{ICIR}(\alpha)$ via (6.1), and sets

$$R(\alpha) = \max\{\exp(\beta_q \text{ICIR}(\alpha)), \exp(-10)\}, \quad (6.4)$$

with quality temperature β_q ($\beta_q = 0.5$ in the headline runs; Section 6.6.3) and an exponential floor for an invalid expression, an evaluation failure, or a non-finite or degenerate ICIR. Individual non-finite entries in an otherwise usable daily IC series are zero-filled before the ICIR is formed. The floor also binds for any evaluable candidate with $\text{ICIR}(\alpha) \leq -10/\beta_q$, so below that level the generator does not distinguish syntactically valid but extremely poor formulas from unevaluable ones. Equation (6.4) reads neither the archive nor any pool statistic—not coverage, novelty, cell identity, incumbent quality, or acceptance status—so for a fixed training window and fixed β_q the reward of a candidate is invariant to later archive updates.

Trajectory balance. The generator is trained by trajectory balance [30, 31]: for a terminal trajectory τ of token length L (distinct from the trading-day count T)

with formula $\alpha(\tau)$,

$$\mathcal{L}_{\text{TB}}(\tau) = \left(\log Z + \sum_{t=0}^{L-1} \log P_F(a_t \mid s_t) - \sum_{t=0}^{L-1} \log P_B(s_t \mid s_{t+1}) - \log R(\alpha(\tau)) \right)^2. \quad (6.5)$$

At the optimum, the probability of sampling a terminal expression under P_F is proportional to $R(\alpha)$, so the policy assigns mass to every high-reward mode rather than collapsing onto the single maximiser; under a fixed budget, this mode-covering sampling is the “efficient” part of the stage [30]. Failed evaluations receive the floor reward and still contribute a finite gradient, so the policy learns to avoid them without a separate invalid-expression branch.

6.3.2 Return-Source MAP-Elites Archiving

Quality and behaviour descriptor. The IC series $\text{IC}(\alpha) \in \mathbb{R}^T$ supplies both the quality score $\text{ICIR}(\alpha)$ and a behaviour descriptor that summarises *when* the signal is strong. AlphaQD centres the IC series and normalises it to unit length up to the ε -regularisation in the denominator,

$$\mathbf{r}(\alpha) = \frac{\mathbf{x}(\alpha) - \overline{\mathbf{x}(\alpha)}\mathbf{1}}{\|\mathbf{x}(\alpha) - \overline{\mathbf{x}(\alpha)}\mathbf{1}\|_2 + \varepsilon}, \quad \mathbf{x}(\alpha) = \text{IC}(\alpha), \quad (6.6)$$

removing level and scale (absorbed into ICIR) and keeping only the relative temporal pattern. By construction $\mathbf{r}(\alpha)$ is invariant to any positive affine transform of the IC series—exactly in the $\varepsilon \rightarrow 0$ limit, and up to a negligible $O(\varepsilon)$ term for the machine-precision ε used in practice (Lemma 6.1)—so it isolates which dates the formula explains, not how strong its edge is; we treat it as the empirical proxy for the formula’s return source. The shape is projected by an incremental PCA basis W (canonical dimension $d_b = 2$) to $\mathbf{z}(\alpha) = W^\top \mathbf{r}(\alpha)$, then discretised by per-axis quantile bin edges into a $B \times B$ grid ($B = 10$, so 100 cells), giving the cell index $c(\alpha)$. The initial basis and edges are fitted after the first 2,000 *finite candidates*, not after 2,000 episodes. Those candidates are kept pending and are offered to the newly created cells retrospectively after the fit. Thereafter only IC series of *accepted elites* enter a FIFO buffer of at most 1,024 series. In the

current AIFML experiment grid, every $K_{\text{pca}} = 100$ completed episodes the implementation checks whether at least 32 accepted series are available; if so, it applies `IncrementalPCA.partial_fit` to the buffer, moves each quantile edge towards its new buffer quantile with EMA decay 0.99, and reprojects every stored elite into the updated cells. When reprojection makes a cell exceed capacity, only the κ elites with largest ICIR + PFS are retained. Thus both coordinates and membership can change throughout a 2,000-episode run; the descriptor map is not frozen after warm-up.

Per-cell score and update. For a positive-ICIR candidate, PFS is the mean *ICIR-retention ratio* over the successful perturbations,

$$\text{PFS}(\alpha) = \text{clip}_{[0,2]} \left(\frac{1}{|\mathcal{S}_\alpha|} \sum_{\tilde{\alpha} \in \mathcal{S}_\alpha} \frac{\text{ICIR}(\tilde{\alpha})}{\text{ICIR}(\alpha)} \right).$$

The implementation attempts five perturbations. Each rolling-window node is independently replaced by another legal window. It first searches within the configured $\pm 10\%$ band, but the legal set is $\{10, 20, 30, 40, 50\}$; when that band contains no alternative it samples from the two nearest legal alternatives, so the realised change can substantially exceed 10%. Failed or non-finite perturbations are omitted, and PFS is set to 1 when none succeeds, including formulas with no rolling-window node. Values are clipped to $[0, 2]$. The perturbation routine creates a fresh unseeded NumPy generator on every call, which is an additional source of uncontrolled run-to-run variation.

PFS is computed lazily. If a cell is full, a cheap pre-screen first requires the candidate’s ICIR to be at least the smallest incumbent ICIR up to 10^{-6} . A non-positive-ICIR candidate is rejected from the archive in the current configuration and does not update the post-warm-up PCA buffer. A non-full cell accepts an eligible candidate without a dominance check. In a full cell, a candidate that dominates one or more incumbents evicts the dominated incumbent with smallest ICIR + PFS; a candidate dominated by any incumbent is rejected; otherwise it replaces the incumbent with the smallest sum only when its own sum is strictly larger. This is a bounded score-based cell, not a maintained Pareto front: insertion into a non-full cell can retain dominated members, and one dominance replacement need not

remove every dominated member. The quota still prevents a dense region from occupying more than κ archive slots per cell.

6.3.3 Top- K Cell Selection

For each occupied cell, the representative is the elite with the highest training-window ICIR; the export code has no explicit PFS tie-break, so an exact ICIR tie retains iteration order. Let $K^* = \min(10, 5\kappa)$, i.e. 5κ capped at 10: $K^* = 10$ under the canonical $\kappa = 3$, matching the pool-style baselines, but $K^* = 5$ under $\kappa = 1$, so the rule couples the exported pool size to the per-cell capacity—a coupling that matters when interpreting the κ axis of the sensitivity grid in Section 6.6.3. AlphaQD ranks occupied cells by representative ICIR and exports the top K^* as the pool $\mathcal{P}^*$. The main comparison then uses the shared closed-form ridge combiner [90],

$$\hat{\mathbf{w}} = (X^\top X + I)^{-1} X^\top \mathbf{y},$$

fitted only to tradable training-split asset–date rows with finite targets. There is no intercept or feature standardisation; missing factor values are replaced by zero. The regularisation coefficient is fixed at $\lambda = 1.0$, and the resulting weights are held fixed through validation and test. Positive-ICIR-proportional weights are written with the native AlphaQD artifact, but they are not the weights used by the unified-ridge headline table. Because the per-cell representative inherits the positive-ICIR admission rule and the cells partition formulas by behaviour, the export is designed to encourage positive standalone training-window edge and lower redundancy; whether the exported pool meets the formal premises of Section 6.4 out of sample is not guaranteed (see the discussion after Theorem 6.6).

6.4 Generalisation Properties

The staging is not merely convenient; it is generalisation-favouring. This section gives the formal results, with proofs inline. The deployed descriptor map is data-dependent and changes at the periodic PCA updates described in Section 6.3.2; none of the results below treats that moving partition as fixed unless the premise says so explicitly. One piece of notation is needed by Theorem 6.2: for a cell index

Algorithm 7 AlphaQD: three-stage training and export

- 1: Initialise the RGCN encoder, $\log P_F$, $\log P_B$, $\log Z$, an empty MAP-Elites archive, and an empty PCA buffer.
 - 2: **for** episode = 1, ..., E **do**
 - 3: **Stage 1:** sample a batch of terminal trajectories under P_F and the legal-action mask.
 - 4: **for** each terminal trajectory τ with formula α **do**
 - 5: Evaluate α on the training window; compute $\text{IC}(\alpha)$, $\text{ICIR}(\alpha)$, and $R(\alpha)$ via (6.4).
 - 6: **Stage 2:** update the descriptor and bounded archive as follows.
 - 7: **if** the first 2,000 finite candidates have not yet been collected **then**
 - 8: Append $\mathbf{r}(\alpha)$ to the warm-up set and retain α as pending; at the threshold, fit W and the initial quantile edges and offer all pending positive-ICIR candidates to their cells.
 - 9: **else if** $\text{ICIR}(\alpha) > 0$ and the cell ICIR pre-screen passes **then**
 - 10: Compute PFS from successful legal-window perturbations and apply the bounded cell update rule.
 - 11: **if** α is accepted **then**
 - 12: Append its IC series to the 1,024-element FIFO PCA buffer.
 - 13: **end if**
 - 14: **end if**
 - 15: Accumulate $\mathcal{L}_{\text{TB}}(\tau)$ from (6.5).
 - 16: **end for**
 - 17: Update $\{\log P_F, \log P_B, \log Z\}$ on the accumulated loss.
 - 18: **if** the episode count is a multiple of $K_{\text{pca}} = 100$ and the FIFO buffer has at least 32 series **then**
 - 19: Incrementally update W , move the quantile edges by EMA, reproject all elites, and trim over-capacity cells by ICIR + PFS.
 - 20: **end if**
 - 21: **end for**
 - 22: **Stage 3:** take the highest-ICIR elite per cell as its representative; rank cells by representative ICIR; export the top K^* as $\mathcal{P}^*$ and fit the shared $\lambda = 1$ ridge combiner on training rows only.
-

$c = (b_1, \dots, b_{d_b})$ with $b_j \in \{0, \dots, B - 1\}$, the *anti-cell* is $\text{anti}(c) = (B - 1 - b_1, \dots, B - 1 - b_{d_b})$, the *orbit* of c is the unordered pair $\mathcal{O}(c) = \{c, \text{anti}(c)\}$, and, by a mild abuse of notation, $\mathcal{O}$ also denotes the set of all orbits. A formula and its sign-flip have oppositely signed IC shapes, but empirical quantile edges—especially after EMA updates—need not be symmetric, so antipodal cell membership is an idealised condition rather than a deployed invariant. Throughout, the discussion after each theorem distinguishes such fixed-map statements from the implementation.

6.4.1 Decoupling Quality from Shape

The first result formalises why decoupling quality from the shape descriptor is sound.

Lemma 6.1 (Affine invariance of the shape descriptor). *For a candidate α , write $\mathbf{v} = \mathbf{x}(\alpha) - \overline{\mathbf{x}(\alpha)}\mathbf{1}$ and suppose $\|\mathbf{v}\|_2 > 0$. For any positive affine transform $\mathbf{y} = a\mathbf{x}(\alpha) + b\mathbf{1}$ with $a > 0$, let $\mathbf{r}_{\mathbf{y}}$ denote the shape vector (6.6) built from $\mathbf{y}$. Then $\mathbf{r}_{\mathbf{y}} = \mathbf{r}(\alpha)$ exactly in the limit $\varepsilon \rightarrow 0$, and for $\varepsilon > 0$,*

$$\|\mathbf{r}_{\mathbf{y}} - \mathbf{r}(\alpha)\|_2 = \frac{\varepsilon |a - 1| \|\mathbf{v}\|_2}{(a\|\mathbf{v}\|_2 + \varepsilon)(\|\mathbf{v}\|_2 + \varepsilon)} = O(\varepsilon/\|\mathbf{v}\|_2).$$

Proof. The mean satisfies $\bar{\mathbf{y}} = a\overline{\mathbf{x}(\alpha)} + b$, so $\mathbf{y} - \bar{\mathbf{y}}\mathbf{1} = a\mathbf{v}$ and $\mathbf{r}_{\mathbf{y}} = a\mathbf{v}/(a\|\mathbf{v}\|_2 + \varepsilon)$. Hence

$$\mathbf{r}_{\mathbf{y}} - \mathbf{r}(\alpha) = \left(\frac{a}{a\|\mathbf{v}\|_2 + \varepsilon} - \frac{1}{\|\mathbf{v}\|_2 + \varepsilon} \right) \mathbf{v} = \frac{\varepsilon(a - 1)}{(a\|\mathbf{v}\|_2 + \varepsilon)(\|\mathbf{v}\|_2 + \varepsilon)} \mathbf{v},$$

whose norm is the stated quantity. It vanishes as $\varepsilon \rightarrow 0$ (and for $a = 1$); with $\varepsilon = 0$ the normalisation cancels $a > 0$ exactly. $\square$

Thus, up to an error of order ε —which is fixed at machine-precision scale while $\|\mathbf{v}\|_2$ is bounded away from zero for any non-constant IC series—the descriptor encodes only which dates were strong; the mean and scale are absorbed into the quality score. For the same reason the descriptor is only approximately of unit norm, $\|\mathbf{r}(\alpha)\|_2 = \|\mathbf{v}\|_2/(\|\mathbf{v}\|_2 + \varepsilon) < 1$, and “unit-normalised” is used in this $O(\varepsilon)$ sense throughout. Two alphas with the same temporal pattern but different overall edge occupy the same cell rather than leaking into different ones; a zero-variance IC series ($\|\mathbf{v}\|_2 = 0$) has no meaningful shape and is handled by the reward floor of (6.4) rather than by the descriptor.

6.4.2 Reward Stationarity

The second result is the structural reason the static reward survives even archive-aware exploration extensions.

Theorem 6.2 (Stationary conditional reward under a fixed descriptor map). *Let the per-alpha reward be $R_q(\alpha) = \exp(\beta_q \text{ICIR}(\alpha))$, and assume for this conditional extension that the PCA basis W and the bin edges entering $c(\cdot)$ are fixed. For any fixed target orbit $\mathcal{O}^*$ and any kernel $k : \mathcal{O} \times \mathcal{O} \rightarrow \mathbb{R}_{\geq 0}$ independent of the archive state and training step, the conditional reward $R_{\mathcal{O}^*}(\alpha) = R_q(\alpha) k(\mathcal{O}(c(\alpha)), \mathcal{O}^*)$ is a stationary terminal reward.*

Proof. For each fixed $\mathcal{O}^*$, $R_{\mathcal{O}^*}(\cdot)$ depends on α only through $\text{ICIR}(\alpha)$ on the fixed training window and through the fixed PCA basis and bin edges entering $c(\cdot)$. The kernel k is fixed and $\mathcal{O}^*$ is a parameter, not a function of training-time state. The reward therefore has no time-dependent component. A scheduler that varies $\mathcal{O}^*$ across episodes changes only the per-episode prior over conditions, never the conditional reward itself. $\square$

The fixed-map premise is *not* supplied by the deployed schedule: the current code updates the PCA basis and edges every 100 episodes when its buffer is large enough. The implementation uses the unconditional reward ($k \equiv 1$), whose stationarity is independent of the descriptor map and therefore survives those updates. The conditional result applies only to a hypothetical descriptor-biased extension with a fixed map; it is not a theorem about the deployed rolling-PCA partition.

6.4.3 Search Complexity and a Boosting Counterexample

The next two results contrast AlphaQD’s quality-diversity ensemble with boosting-style residual fitting in the abstract form used by the pool-coupled rewards of Section 6.2; they concern this class of procedures, not the specific published AlphaGen system.

Theorem 6.3 (Residual-edge noise floor). *Let $\mathcal{F} = \{\alpha_1, \dots, \alpha_M\}$ be a finite candidate set. After $t - 1$ boosting rounds with residual r_{t-1} , define the population residual edge and its training-window estimate by*

$$\gamma_t(\alpha) = \mathbb{E}[\langle \alpha, r_{t-1} \rangle], \quad \hat{\gamma}_t(\alpha) = \gamma_t(\alpha) + \xi_{t,\alpha}.$$

Assume that each deviation is sub-Gaussian: for a scale parameter σ and effective sample size n_{eff} ,

$$\mathbb{P}(|\xi_{t,\alpha}| > u) \leq 2 \exp\left(-\frac{n_{\text{eff}}u^2}{2\sigma^2}\right).$$

Let $\alpha_t = \arg \max_{\alpha \in \mathcal{F}} \hat{\gamma}_t(\alpha)$, $\Delta_t = \sup_{\alpha} \gamma_t(\alpha)$, and $\eta_M = \sigma \sqrt{2 \log(2M/\delta)/n_{\text{eff}}}$. Then, with probability at least $1 - \delta$, $\gamma_t(\alpha_t) \geq \Delta_t - 2\eta_M$. When $\Delta_t \leq 2\eta_M$, the bound is non-positive.

Proof. Uniform sub-Gaussian concentration on the finite class $\mathcal{F}$ gives $\sup_{\alpha} |\hat{\gamma}_t(\alpha) - \gamma_t(\alpha)| \leq \eta_M$ with probability at least $1 - \delta$. Writing $\alpha_t^* = \arg \max_{\alpha} \gamma_t(\alpha)$, $\gamma_t(\alpha_t) \geq \hat{\gamma}_t(\alpha_t) - \eta_M \geq \hat{\gamma}_t(\alpha_t^*) - \eta_M \geq \gamma_t(\alpha_t^*) - 2\eta_M = \Delta_t - 2\eta_M$, where the middle step uses the empirical optimality of α_t . $\square$

With $|\mathcal{F}| > 10^{20}$ and n_{eff} bounded by the training-window days, η_M is large relative to plausible late-round residual edges, so the guarantee $\Delta_t - 2\eta_M$ turns non-positive and hence vacuous: the empirical residual winner retains no assurance of positive population edge. This is a statement about the emptiness of the guarantee, not an upper bound on the winner's edge; establishing that late-round winners are in fact noise-dominated would require a matching upper bound or an explicit winner's-curse argument, which we do not attempt.

Theorem 6.4 (Search-complexity contrast). *A K -step boosting procedure over $\mathcal{F}$ with $|\mathcal{F}| = M$ (here K plays the same retained-set-size role as in (6.2)) has an ordered-sequence hypothesis class of size at most M^K , with finite-class complexity $\mathcal{C}_{\text{boost},K} = \sqrt{K \log M/n_{\text{eff}}}$. A quality-diversity rule mapping candidates to at most G occupied cells and exporting one representative per cell has K -subset class of size at most $\binom{G}{K}$, with complexity $\mathcal{C}_{\text{QD},K} = \sqrt{K \log(eG/K)/n_{\text{eff}}}$. For $G \ll M$, $\mathcal{C}_{\text{QD},K} \ll \mathcal{C}_{\text{boost},K}$.*

Proof. The finite-class generalisation bound scales as $\sqrt{\log |\mathcal{H}|/n_{\text{eff}}}$. Substituting $|\mathcal{H}_{\text{boost},K}| \leq M^K$ gives $\log |\mathcal{H}_{\text{boost},K}| \leq K \log M$. Substituting $|\mathcal{H}_{\text{QD},K}| \leq \binom{G}{K}$ and $\log \binom{G}{K} \leq K \log(eG/K)$ gives the QD scaling. $\square$

In the reported grid $G \leq 100$ while $|\mathcal{F}| > 10^{20}$: the class sizes differ by many orders of magnitude, though the complexity terms themselves differ only by a modest factor (about 3.7 under $M = 10^{20}$, $G = 100$, $K = 10$). Two caveats govern

the reading. First, the comparison is conditional on the realised archive: the count $\binom{G}{K}$ treats the per-cell representatives, the PCA basis, and the bin edges as fixed, whereas all three are data-dependent products of a search that itself screened a number of candidates comparable to what boosting sees, and the ridge weights add a continuous layer on top. Theorem 6.4 therefore bounds the complexity of the final selection stage given the archive, not the effective complexity of the full pipeline. Second, the two class-size counts are upper bounds on differently structured procedures, so the contrast is indicative rather than a like-for-like risk comparison.

6.4.4 Per-Cell Selection Bias

When a cell aggregates several candidates and the elite is selected by maximum empirical ICIR, the chosen elite’s empirical ICIR is upward-biased relative to its population value—the formula-mining analogue of a multiple-testing winner’s curse [8]. The bounded per-cell quota controls the final, within-cell component of this bias.

Theorem 6.5 (Per-cell selection bias under a bounded quota). *Fix an occupied cell holding at most κ admitted elites at selection time, each with population quality ϑ_i and training-window estimate $\hat{\vartheta}_i = \vartheta_i + \varepsilon_i$, where each ε_i is sub-Gaussian with $\mathbb{P}(\varepsilon_i > u) \leq \exp(-n_{\text{eff}}u^2/(2\sigma^2))$ for a scale parameter σ and effective sample size n_{eff} . With probability at least $1 - \delta$, the selection bias of the per-cell representative on raw ICIR satisfies $\max_{i \leq \kappa} (\hat{\vartheta}_i - \vartheta_i) \leq \sigma \sqrt{2 \log(\kappa/\delta)/n_{\text{eff}}}$.*

Proof. A union bound on the κ sub-Gaussian tails gives $\mathbb{P}(\max_{i \leq \kappa} \varepsilon_i > u) \leq \kappa \exp(-n_{\text{eff}}u^2/(2\sigma^2))$; setting the right-hand side to δ yields $u = \sigma \sqrt{2 \log(\kappa/\delta)/n_{\text{eff}}}$. $\square$

Under the canonical $\kappa = 3$, $\delta = 0.05$ the bound is $\approx 2.86 \sigma / \sqrt{n_{\text{eff}}}$. Two caveats bound its reach. First, the deployed ranking uses raw ICIR with PFS only as a tie-break (Section 6.3.2); the five PFS perturbations share the same training window and are neither independent nor same-mean estimates of the elite’s quality, so no $\sqrt{1+m}$ -style tightening of the bound is claimed here—any such tightening would be heuristic. Second, and more importantly, the theorem conditions on the κ stored elites as a fixed set, whereas they are themselves the survivors of an empirical-score

tournament over every candidate the cell ever received. The quota therefore bounds the bias of the final within-cell $\arg \max$, but it does not remove the selection bias accumulated during the search—the search-stage multiple-testing winner’s curse [8] lies outside the theorem’s scope.

6.4.5 Ensemble ICIR Lower Bound

The final result is an abstract lemma about equal-weight ensembles; its relation to the deployed pool is discussed below.

Theorem 6.6 (Ensemble ICIR lower bound). *Let $X_1, \dots, X_K$ be daily IC variables satisfying $\mathbb{E}[X_i] \geq \mu_{\min} > 0$, $\text{Var}(X_i) \leq \sigma^2$, and $\text{corr}(X_i, X_j) \leq \rho_{\max}$ for all $i \neq j$, for some $\rho_{\max} \in [0, 1]$. Then the equal-weight ensemble $X_{\text{ens}} = \frac{1}{K} \sum_i X_i$ obeys*

$$\text{ICIR}(X_{\text{ens}}) \geq \frac{\mu_{\min} \sqrt{K}}{\sigma \sqrt{1 + (K-1)\rho_{\max}}}. \quad (6.7)$$

Proof. By linearity, $\mathbb{E}[X_{\text{ens}}] \geq \mu_{\min}$. For the variance,

$$\begin{aligned} \text{Var}(X_{\text{ens}}) &= \frac{1}{K^2} \sum_i \text{Var}(X_i) + \frac{1}{K^2} \sum_{i \neq j} \text{Cov}(X_i, X_j) \\ &\leq \frac{\sigma^2}{K^2} [K + K(K-1)\rho_{\max}] \\ &= \frac{\sigma^2(1 + (K-1)\rho_{\max})}{K}. \end{aligned}$$

so $\text{ICIR}(X_{\text{ens}}) = \mathbb{E}[X_{\text{ens}}] / \sqrt{\text{Var}(X_{\text{ens}})} \geq \mu_{\min} \sqrt{K} / (\sigma \sqrt{1 + (K-1)\rho_{\max}})$. $\square$

The proof also uses $\rho_{\max} \geq 0$: with a negative correlation cap the covariance inequality would reverse. Theorem 6.6 is a statement about any family of random variables satisfying its premises; three gaps separate it from a guarantee for the deployed pool. First, the admission rule enforces a positive *training-window* ICIR, which makes the premise $\mathbb{E}[X_i] \geq \mu_{\min} > 0$ plausible for the exported elites but does not imply it out of sample. Second, the premise requires a uniform cap $\rho_{\max}$ on all pairwise correlations of the exported components. Spreading elites across PCA cells encourages lower redundancy but does not enforce such a cap, and the archive figure reports no pairwise-correlation statistic. Third, the bound concerns the equal-weight average of the components’ daily IC variables, whereas the deployed

signal is a ridge-weighted combination of factor values whose IC is not a linear average of the component ICs. Theorem 6.3 shows that boosting-style residual fitting need not preserve even the first premise, so the analogous bound is not available for it by this route; for AlphaQD too, however, the result is motivation for the export design rather than a finite-sample guarantee for the deployed pool.

6.5 Experimental Setup

Datasets. We evaluate on two markets so the comparison is not anchored to one regime. The implementation is maintained in the AIFML project repository.² The Chinese A-share panel (the canonical alpha-mining setting) is the primary benchmark: the experiment record reports BaoStock daily data for 7,207 instruments over 3,963 trading days from 2010-01-04 to 2026-04-30, with front-adjusted OHLCV and $VWAP = \text{amount}/\text{volume}$. The U.S. panel is an independent check with different microstructure and base rates: the record reports a Tiingo Russell-3000-style panel of 2,571 instruments over 4,109 trading days, using Tiingo’s adjusted price fields. Tradability masks apply at execution: on A-shares an asset must have finite prices, positive volume, normal trading status, non-ST status, and an open-to-previous-close gap within $\pm 9.7\%$ (approximating price-limit constraints); on the U.S. panel, finite prices, positive volume, and a minimum close of one dollar (excluding penny names).

Both datasets are evaluated as fixed retrospective research panels rather than as replications of a historical index membership series. Every method in a panel receives the same instruments, dates, split, labels, execution rule, and costs, so the tables support like-for-like within-panel ranking; the results should not be interpreted as a reconstruction of a point-in-time investable index.

Split and usage. All methods share the nominal split: 2010–2021 for training, 2022 for validation, and 2023-01-01 to 2026-04-30 for testing, with forward-return horizon $h = 20$ trading days. The shared ridge weights are fit only on rows marked as training and are then held fixed. The data loader, however, first loads the entire train-to-test range and gives every split a 20-day future buffer. Consequently

²<https://github.com/qinmoelei/AIFML>

the final 20 training-date labels use prices from the first 20 validation dates, and the final 20 validation-date labels use prices from the first 20 test dates. This is direct cross-split label leakage, not merely a possible overlap. The supplied leakage check compares signal dates after trimming buffers and therefore does not detect target-horizon overlap. At the other end, the final 20 test dates are trimmed from label-based evaluation because no later prices are available.

The test set is also inspected during AlphaQD training. Every 200 episodes, and once again at the end, the logger evaluates a test-set ensemble and writes test IC and RankIC. For a 2,000-episode run this produces ten periodic evaluations plus the final call. Those logged values are not fed into the loss or archive update by the current code, but the protocol is not a one-touch test evaluation and permits monitoring-based selection. The sensitivity grid additionally evaluates all configurations on the same test window and is therefore post-hoc.

Reproducibility of nominal seeds. The result grid can launch runs labelled by seeds $\{0, 1, 2, 12345\}$, but the AlphaQD pipeline stores the argument without calling the repository’s `reseed_everything` helper or otherwise seeding Python, NumPy, or PyTorch before constructing the network and sampler. PFS separately creates an unseeded `numpy.random.default_rng()` for every perturbation call. The four AlphaQD rows underlying a mean and standard deviation are therefore four stochastically different processes carrying seed labels, not deterministic replications controlled by those values; exact reruns are not reproducible from the current AIFML entry point reviewed for this thesis.

Compared methods. We compare against eleven baselines in four families: formulaic (the fixed Alpha101 catalogue [17]); supervised (MLP [91], XGBoost [92], LightGBM [93]); genetic programming (GP single-predictor; GP with mutual-IC filter; GP without filter [63]); and generative/RL (AlphaForge [71], AlphaGenPPO [67], AlphaQCM [68], AlphaSAGE [72]). Pool-style baselines export ten alphas each, matching AlphaQD’s $K^* = 10$.

Shared protocol. Every pool-style method, including AlphaQD, exports a fixed-size pool. That pool is fed to the same closed-form $\lambda = 1$ ridge combiner described in Section 6.3.3, with weights fitted on training-marked rows and held fixed through

validation and test. We report signal-layer IC, ICIR, and RankIC [16] together with portfolio-layer annualised net Sharpe [13], annualised net return (the arithmetic mean daily net return scaled by 252), maximum drawdown [12] (reported as a positive magnitude; smaller is better), and annualised turnover. The portfolio is long-only top-50 rebalanced every twenty trading days, executed at $VWAP_{t+1}$ on A-shares and $Open_{t+1}$ on the U.S. panel, with round-trip costs of 12.5 and 3.5 basis points respectively. Applying this common protocol to every method makes the within-panel ranks the primary comparison.

6.6 Results and Analysis

6.6.1 Main Results

Table 6.1 reports both panels under the unified-ridge protocol, aggregated over four runs labelled $\{0, 1, 2, 12345\}$ where stochastic. For AlphaQD these labels are not applied as random seeds by the current AIFML entry point, as documented in Section 6.5.

A-share findings. On the A-share panel, AlphaQD attains the highest annualised return and the highest RankIC of any method. It is second on IC (behind GP-without-filter) and second on Sharpe (behind only the fixed Alpha101 catalogue). The leading signal-quality cluster comprises GP-without-filter, AlphaForge, AlphaSAGE, and AlphaQD, whose mean ICIRs are separated by less than the run-level standard deviations reported in the table. Within this cluster the portfolio columns differentiate more sharply. GP-without-filter and AlphaForge carry run-level Sharpe standard deviations comparable to their means (0.501 ± 0.573 and 0.534 ± 0.493), so their portfolio outcomes vary widely across runs. AlphaSAGE has the smallest reported run-level deviations in the cluster, but its annualised return is roughly ten points below AlphaQD’s and its turnover 46% higher. Among the four generative/RL pool-style methods, AlphaQD has the lowest turnover, the second-smallest run-level deviations on Sharpe and annualised return (behind AlphaSAGE), and the smallest on maximum drawdown. These are descriptive differences across the available runs; for AlphaQD they are not controlled-seed variance

TABLE 6.1: Main results on the 2023-01-01 to 2026-04-30 test window under the unified-ridge protocol. Entries are mean $\pm$ population standard deviation over four runs labelled $\{0, 1, 2, 12345\}$ where stochastic; deterministic methods carry zero run variance. The current AIFML entry point does not apply those labels as RNG seeds. Portfolio metrics are net of round-trip costs on a long-only top-50 portfolio rebalanced every twenty trading days; maximum drawdown is reported as a positive magnitude (smaller is better). Per column within each panel, the **best**, second-best, and third-best entries are marked in bold, underline, and dotted underline respectively (colour retained for screen reading).

Family	Method	IC $\uparrow$	ICIR $\uparrow$	RankIC $\uparrow$	Sharpe $\uparrow$	AnnRet(%) $\uparrow$	MDD(%) $\downarrow$	TO/yr $\downarrow$
<i>Panel A: Chinese A-share</i>								
Formulaic	Alpha101	0.0487 $\pm$ 0.0000	0.408 $\pm$ 0.000	0.082 $\pm$ 0.000	0.983 $\pm$ 0.000	<u>19.9 $\pm$ 0.0</u>	26.1 $\pm$ 0.0	23.9
	MLP	0.0079 $\pm$ 0.0017	0.111 $\pm$ 0.018	0.017 $\pm$ 0.003	0.319 $\pm$ 0.038	9.1 $\pm$ 1.3	49.7 $\pm$ 1.0	12.9
Supervised	LightGBM	0.0189 $\pm$ 0.0002	0.262 $\pm$ 0.006	0.035 $\pm$ 0.000	0.309 $\pm$ 0.022	10.0 $\pm$ 0.7	52.7 $\pm$ 1.4	<u>12.0</u>
	XGBoost	0.0199 $\pm$ 0.0000	0.266 $\pm$ 0.000	0.036 $\pm$ 0.000	0.236 $\pm$ 0.000	6.9 $\pm$ 0.0	50.6 $\pm$ 0.0	<u>12.4</u>
Genetic Prog.	GP (predictor)	-0.0101 $\pm$ 0.0132	-0.082 $\pm$ 0.118	-0.016 $\pm$ 0.024	0.292 $\pm$ 0.309	6.4 $\pm$ 6.1	40.4 $\pm$ 16.4	3.8
	GP w/ filter	<u>0.0652 $\pm$ 0.0080</u>	0.507 $\pm$ 0.070	<u>0.105 $\pm$ 0.010</u>	0.663 $\pm$ 0.082	<u>18.6 $\pm$ 3.9</u>	39.1 $\pm$ 5.2	15.1
	GP w/o filter	0.0703 $\pm$ 0.0157	0.581 $\pm$ 0.064	<u>0.102 $\pm$ 0.025</u>	0.501 $\pm$ 0.573	10.8 $\pm$ 12.9	35.7 $\pm$ 9.4	20.2
Generative / RL	AlphaForge	0.0636 $\pm$ 0.0099	<u>0.553 $\pm$ 0.030</u>	0.086 $\pm$ 0.016	0.534 $\pm$ 0.493	14.4 $\pm$ 13.4	37.0 $\pm$ 15.7	19.5
	AlphaGen-PPO	0.0435 $\pm$ 0.0215	0.495 $\pm$ 0.219	0.049 $\pm$ 0.043	0.275 $\pm$ 0.551	4.1 $\pm$ 11.3	43.3 $\pm$ 15.4	22.2
	AlphaQCM (FQF)	0.0206 $\pm$ 0.0187	0.255 $\pm$ 0.291	0.020 $\pm$ 0.035	0.053 $\pm$ 0.451	1.7 $\pm$ 12.2	46.8 $\pm$ 12.2	19.3
	AlphaSAGE (GNN)	0.0552 $\pm$ 0.0082	<u>0.555 $\pm$ 0.142</u>	0.095 $\pm$ 0.008	<u>0.730 $\pm$ 0.050</u>	14.9 $\pm$ 1.0	<u>27.4 $\pm$ 3.9</u>	18.3
Ours	AlphaQD	<u>0.0673 $\pm$ 0.0093</u>	0.522 $\pm$ 0.078	0.112 $\pm$ 0.011	<u>0.911 $\pm$ 0.185</u>	24.7 $\pm$ 6.2	<u>31.5 $\pm$ 1.7</u>	12.5
<i>Panel B: U.S. Russell-3000-style universe</i>								
Formulaic	Alpha101	0.0188 $\pm$ 0.0000	0.358 $\pm$ 0.000	<u>-0.002 $\pm$ 0.000</u>	1.511 $\pm$ 0.000	38.2 $\pm$ 0.0	<u>15.9 $\pm$ 0.0</u>	23.0
	MLP	0.0446 $\pm$ 0.0102	0.604 $\pm$ 0.147	<u>-0.002 $\pm$ 0.007</u>	1.796 $\pm$ 0.055	99.8 $\pm$ 5.0	<u>16.3 $\pm$ 0.9</u>	9.1
Supervised	LightGBM	0.0404 $\pm$ 0.0023	0.454 $\pm$ 0.025	-0.009 $\pm$ 0.001	2.090 $\pm$ 0.035	119.1 $\pm$ 2.3	21.6 $\pm$ 1.4	8.7
	XGBoost	0.0335 $\pm$ 0.0000	0.395 $\pm$ 0.000	-0.006 $\pm$ 0.000	2.120 $\pm$ 0.000	129.2 $\pm$ 0.0	23.4 $\pm$ 0.0	9.6
Genetic Prog.	GP (predictor)	-0.0104 $\pm$ 0.0458	-0.073 $\pm$ 0.417	0.004 $\pm$ 0.010	1.202 $\pm$ 0.497	39.4 $\pm$ 45.8	15.2 $\pm$ 5.2	5.7
	GP w/ filter	<u>0.0691 $\pm$ 0.0014</u>	0.512 $\pm$ 0.069	-0.005 $\pm$ 0.003	2.180 $\pm$ 0.043	141.6 $\pm$ 6.4	26.7 $\pm$ 2.4	9.5
	GP w/o filter	0.0717 $\pm$ 0.0031	<u>0.568 $\pm$ 0.013</u>	-0.013 $\pm$ 0.001	<u>2.164 $\pm$ 0.047</u>	<u>141.3 $\pm$ 5.5</u>	24.7 $\pm$ 0.8	10.2
Generative / RL	AlphaForge	0.0481 $\pm$ 0.0227	0.430 $\pm$ 0.181	-0.008 $\pm$ 0.002	1.856 $\pm$ 0.525	107.0 $\pm$ 41.3	23.4 $\pm$ 1.7	11.6
	AlphaGen-PPO	0.0542 $\pm$ 0.0082	0.437 $\pm$ 0.062	-0.010 $\pm$ 0.002	2.056 $\pm$ 0.223	129.8 $\pm$ 15.6	23.3 $\pm$ 1.1	13.8
	AlphaQCM (FQF)	0.0486 $\pm$ 0.0295	0.453 $\pm$ 0.286	-0.002 $\pm$ 0.009	1.942 $\pm$ 0.372	103.8 $\pm$ 46.4	21.1 $\pm$ 3.4	<u>7.9</u>
	AlphaSAGE (GNN)	0.0614 $\pm$ 0.0077	0.539 $\pm$ 0.068	-0.007 $\pm$ 0.003	2.111 $\pm$ 0.094	131.3 $\pm$ 7.4	24.2 $\pm$ 1.5	10.5
Ours	AlphaQD	<u>0.0655 $\pm$ 0.0022</u>	<u>0.551 $\pm$ 0.036</u>	-0.010 $\pm$ 0.001	<u>2.177 $\pm$ 0.020</u>	<u>133.3 $\pm$ 2.0</u>	22.5 $\pm$ 0.2	<u>6.7</u>

estimates. Figure 6.3 plots the cumulative net NAV of the five learned pool-style methods.

U.S. findings. Under the shared U.S. protocol, AlphaQD ranks third on IC, ICIR, and annualised return, and second on net Sharpe and turnover. Among the four generative/RL pool-style methods it also has the smallest reported run-level deviation on every IC-family metric. These relative positions, rather than the absolute level of any one portfolio metric, are the intended cross-method comparison on this panel.

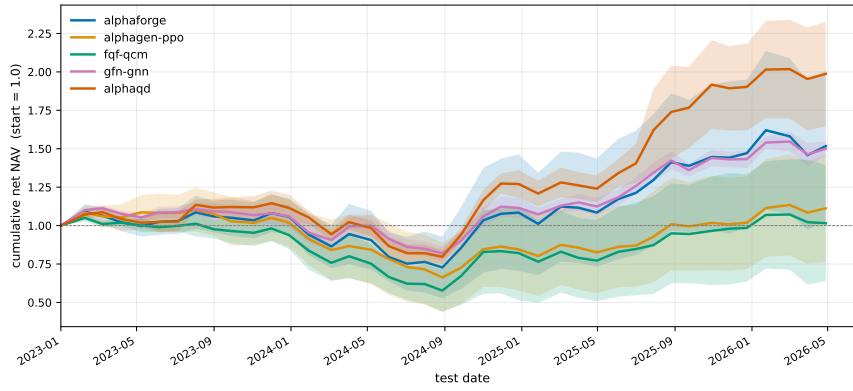


FIGURE 6.3: Cumulative net NAV (start = 1.0) on the A-share test window 2023-01-01 to 2026-04-30 for the five learned pool-style methods. Solid lines are the mean NAV across four runs labelled $\{0, 1, 2, 12345\}$ at each rebalance date; shaded bands span ± 1 population standard deviation; the dashed line marks NAV = 1. The current AIFML entry point does not apply these labels as RNG seeds. The portfolio is the long-only top-50 rebalanced every twenty trading days at VWAP $_{t+1}$ with 12.5 bps round-trip cost.

6.6.2 Archive Diagnostics

The archive snapshot in Figure 6.4 contains 87 occupied cells and 224 stored elites. The largest representative ICIR annotated in the grid is 0.66. These figure-derived values are used consistently as the descriptive archive result; the grid visualises broad coverage of the two-dimensional behaviour space, while it does not by itself establish a uniform correlation bound for the exported pool.

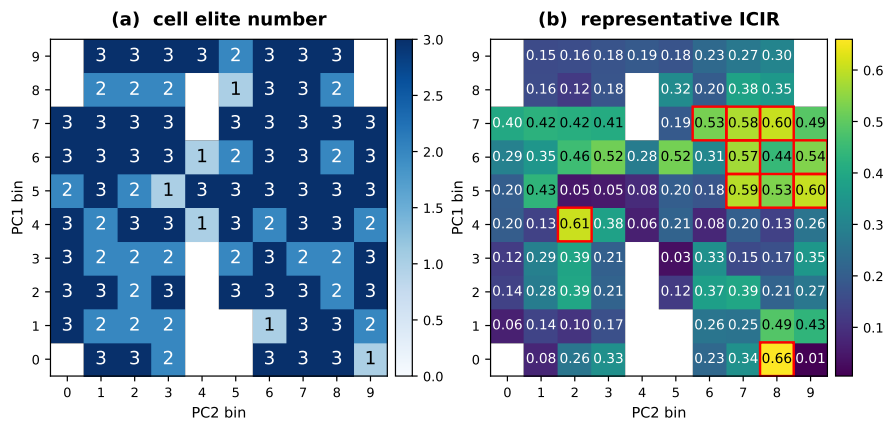


FIGURE 6.4: AlphaQD archive snapshot on the 10×10 behaviour-descriptor grid (vertical axis PC1 bin, horizontal axis PC2 bin). (a) number of stored elites per cell; (b) representative ICIR per cell, with ten cells outlined. The plotted annotations contain 87 occupied cells and 224 elites; the largest annotated representative ICIR is 0.66.

6.6.3 Ablations

A grid over behaviour-descriptor dimension $d_b \in \{1, 2, 3\}$, per-cell capacity $\kappa \in \{1, 3, 5\}$, and reward temperature $\beta_q \in \{0.5, 1, 2, 5\}$ probes the sensitivity of the canonical configuration. Marginal ICIR declines gently from $d_b = 1$ to $d_b = 3$, consistent with a higher-dimensional descriptor spreading candidates over more cells; the canonical $d_b = 2$ sits in the favourable region while preserving an interpretable two-axis archive. Capacity $\kappa \geq 3$ is preferred over $\kappa = 1$. The reward temperature is non-monotone, peaking at $\beta_q = 2$ in this grid; the headline runs retain $\beta_q = 0.5$, which the experiment record associates with lower variability in pilot repetitions. Because the current AIFML entry point does not apply its nominal seed, that pilot variation is run-to-run rather than controlled-seed variation. The full 36-configuration grid is reported in Table 6.2; within-grid variation (ICIR roughly 0.22–0.66 across cells) is larger than the marginal-mean effects, so individual cells do not establish separated hyperparameter effects.

Three qualifications bound what this grid can show. First, it varies only hyperparameters: no core component—the static reward, the MAP-Elites archive, PFS, the behaviour descriptor, or the GFlowNet generator—is removed or replaced, so the grid is a hyperparameter sensitivity study and cannot by itself attribute performance to individual design components. Second, the axes are not pure single-factor manipulations: the cell count is B^{d_b} , so the d_b axis simultaneously changes the archive size, and the export rule $K^* = \min(10, 5\kappa)$ ties the exported pool size to κ , so the $\kappa = 1$ column exports a five-alpha pool while $\kappa \in \{3, 5\}$ export ten—the preference for $\kappa \geq 3$ therefore confounds cell capacity with pool size. Third, all 36 configurations are evaluated on the test window, and the headline AlphaQD runs already log test IC and RankIC every 200 training episodes. The grid is therefore a post-hoc sensitivity analysis on a repeatedly inspected test set, not protocol-clean confirmation of the design choices.

Computational cost. A full training run of AlphaQD on the A-share panel—2,000 GFlowNet trajectory-balance episodes with mini-batch size 64, archive maintenance, and PFS evaluation—completes in approximately five to six hours on a single NVIDIA RTX 4090 GPU. The dominant cost is the per-candidate back-test of the IC series on the full-market training window; archive insertion is $O(1)$ amortised under the per-cell quota and the PCA fit is incremental. Comparable

TABLE 6.2: AlphaQD per-configuration ablation on the A-share panel under the unified-ridge protocol (one run carrying a nominal seed label, test window). The current AIFML entry point does not apply that label to its random-number generators. Columns are descriptor dimension d_b , cell capacity κ , reward temperature β_q , signal IC, ICIR, Spearman rank-IC, and net Sharpe.

d_b	κ	β_q	IC	ICIR	RankIC	Sharpe
1	1	0.5	0.0565	0.455	0.074	0.322
1	1	1.0	0.0584	0.592	0.097	0.951
1	1	2.0	0.0772	0.647	0.127	1.027
1	1	5.0	0.0328	0.246	0.045	0.889
1	3	0.5	0.0740	0.542	0.123	1.186
1	3	1.0	0.0577	0.464	0.094	0.699
1	3	2.0	0.0757	0.630	0.106	0.540
1	3	5.0	0.0760	0.511	0.127	0.835
1	5	0.5	0.0669	0.493	0.096	0.833
1	5	1.0	0.0658	0.560	0.100	0.879
1	5	2.0	0.0683	0.664	0.107	1.010
1	5	5.0	0.0486	0.485	0.077	0.668
2	1	0.5	0.0541	0.461	0.075	0.874
2	1	1.0	0.0258	0.218	0.040	0.909
2	1	2.0	0.0724	0.662	0.106	1.175
2	1	5.0	0.0760	0.646	0.096	0.738
2	3	0.5	0.0736	0.537	0.123	0.955
2	3	1.0	0.0432	0.366	0.047	0.907
2	3	2.0	0.0724	0.541	0.112	0.718
2	3	5.0	0.0712	0.565	0.113	0.869
2	5	0.5	0.0589	0.529	0.087	0.976
2	5	1.0	0.0793	0.613	0.122	0.892
2	5	2.0	0.0375	0.511	0.087	1.005
2	5	5.0	0.0734	0.578	0.123	1.141
3	1	0.5	0.0699	0.505	0.109	0.991
3	1	1.0	0.0318	0.368	0.058	0.669
3	1	2.0	0.0502	0.516	0.080	0.644
3	1	5.0	0.0669	0.534	0.083	1.144
3	3	0.5	0.0756	0.525	0.127	1.040
3	3	1.0	0.0603	0.518	0.078	0.581
3	3	2.0	0.0554	0.616	0.085	0.957
3	3	5.0	0.0355	0.405	0.048	-0.008
3	5	0.5	0.0314	0.352	0.031	0.422
3	5	1.0	0.0741	0.562	0.130	1.025
3	5	2.0	0.0494	0.414	0.087	0.584
3	5	5.0	0.0306	0.342	0.044	0.847

per-baseline budgets (candidate evaluations or GPU-hours) are not reported, so this figure characterises AlphaQD’s absolute cost rather than an efficiency advantage over the baselines. At deployment the cost is negligible: the exported $K^* = 10$ alphas evaluate as fixed symbolic expressions, and the downstream ridge combiner

is a fixed closed-form linear map—its weights, fitted once on the training window (Section 6.5), are held fixed, so producing the signal at each rebalance date is a single matrix–vector product.

6.7 Chapter Summary

This chapter presented AlphaQD, a quality-diversity RL method for formulaic alpha mining. Its central choice is structural: the running pool is removed from the generator’s reward, and diversity, robustness, and selection are handled by a MAP-Elites archive whose behaviour descriptor is the normalised daily IC series—an empirical proxy for a formula’s return source. The training-window reward is thereby invariant to archive updates. Conditional on a realised archive, the final cell-selection class scales with the number of occupied cells rather than the cardinality of the formula space; this is not a complexity bound for the data-dependent search, rolling PCA, and ridge pipeline as a whole. On the reported A-share panel AlphaQD attains the highest annualised return and net Sharpe among learned formula-mining methods; among the four generative/RL pool-style methods it has the lowest turnover on both reported panels. On the U.S. panel it ranks third on IC, ICIR, and annualised return and second on net Sharpe and turnover. Its reported variability is run-to-run rather than controlled-seed variance. In the arc of the thesis, AlphaQD is the endpoint of the aggregation-decoupling trajectory: where EarnHFT learns its router and FineFT derives one heuristically from generative uncertainty, AlphaQD removes the aggregation mechanism from the learning reward entirely. Reward stationarity holds for the deployed unconditional reward despite rolling descriptor updates; the ensemble ICIR lower bound and selection-complexity argument remain idealised results under the explicit premises of Section 6.4.

Part IV

Epilogue

Chapter 7

Conclusion and Future Work

This thesis asked what reinforcement learning can do for quantitative trading, where it fails when moved from one trading task to another, and how those failures can be mitigated. It answered the question through three studies spanning second-level position control, five-minute leveraged trading, and daily signal generation. This final chapter summarises the contributions, revisits the comparative perspective that connects them, states their scope and limitations, and sketches the work they invite.

7.1 Summary of Contributions

EarnHFT (Chapter 4). For second-level high-frequency trading, EarnHFT combines a hindsight, sample-path Q-teacher with trend-conditioned policies and an offline minute-level router. The teacher can be constructed because historical replay makes future prices exogenous to the agent; the corresponding tabular analysis is an idealisation and does not establish optimality for function approximation or an impact-aware live market. In the reported single-seed replays, EarnHFT attains the highest total return on all four datasets, and the component ablation supports the usefulness of value supervision within that experiment.

FineFT (Chapter 5). For leveraged futures, FineFT uses an ensemble TD matrix to assign updates selectively, fits a VAE to each inferred dynamic, and routes

by reconstruction-based familiarity scores with a conservative fallback. Leverage magnifies account-return and reward variation rather than the uncertainty of the underlying price transition. The VAE scores are heuristic indicators of distributional familiarity, not calibrated competence boundaries. Across the four reported crypto markets, FineFT has the highest average total return among learned methods and the lowest reported average maximum drawdown, although the evaluation does not isolate the router as the sole cause and is not uniformly dominant by market.

AlphaQD (Chapter 6). For formulaic alpha mining, AlphaQD separates a candidate-only GFlowNet reward from a MAP-Elites archive keyed on the normalised daily IC-series shape. This removes archive membership from the reward and provides a structured export set. The chapter’s formal results are an abstract equal-weight ICIR lemma and a separate finite-class comparison conditional on a realised archive; neither is a finite-sample guarantee for the fitted ridge portfolio. The method is evaluated on A-share and U.S. panels, with its strongest portfolio result on A-shares.

7.2 The Unifying Perspective Revisited

Across tasks that differ in frequency, instrument, and output, the methods all structure heterogeneity, construct differentiated candidates, and aggregate them. The analogy must be drawn with care. EarnHFT groups historical periods by trend; FineFT assigns transitions to ensemble members from their TD errors; AlphaQD partitions the behaviour space of formulas, not the market. Its cell elites are formulas rather than RL specialists, and a downstream ridge regression combines their outputs. The comparison is therefore an organising vocabulary, not evidence that all three recover locally stationary MDPs.

The narrower connective thread concerns target coupling. EarnHFT trains its router only after freezing the low-level policies; FineFT uses a rule over reconstruction scores; AlphaQD keeps archive membership out of the generator’s reward. This motivates a conditional working hypothesis: when aggregation would change a candidate’s target as other candidates enter or leave the set, separating the aggregation criterion can stabilise that target and clarify the assumptions behind

selection. The studies were neither designed nor executed as a controlled comparison of coupling, so they do not establish a causal performance benefit, and jointly trained mixture-of-experts systems remain valid counterexamples to any universal rule.

There is also concrete methodological reuse. FineFT reuses EarnHFT’s hindsight supervision based on dynamic programming and its slope-based segmentation, while adapting both to leveraged futures and ensemble training. AlphaQD then relocates diversity and set selection outside the generator’s reward. These are related iterations, but conceptual lineage should not be confused with identical implementations.

7.3 Limitations

Each study rests on assumptions that bound its claims.

EarnHFT. The teacher and trend labels use future prices and are available only during historical training. Replay makes prices exactly exogenous to the simulated action, but that is only a small-agent approximation to live trading and cannot support a claim about market impact or optimal order execution. The released learner takes a maximum directly over the target network, so despite the DDQN naming it is not the strict Double-DQN target. The reported runs use one seed (12345); correlated training checkpoints do not provide seed variance, confidence intervals, or significance tests.

FineFT. The reported experiments use one fixed-seed training run per configuration. The validation period supports several stages—specialist filtering, VAE fitting, score calibration, and router selection—so those choices are not independent validation tests. The VAE reconstruction score is a heuristic familiarity measure rather than a calibrated probability of policy failure, and the supplementary Corn/ES results are much lower-frequency than the main cryptocurrency experiments. These limits do not invalidate the reported within-table ranking, but they bound claims about causal mechanisms and transfer.

AlphaQD. In the current AIFML configuration reviewed for this thesis, the descriptor is warmed up on 2,000 finite candidates and the PCA/edge model is scheduled for periodic refitting every 100 episodes; refitting reprojects and re-bins all stored elites. The fixed-map theory is therefore an idealised variant, not a guarantee for the moving archive. The AIFML pipeline accepts a seed argument but does not apply it to the main random generators, while PFS creates fresh entropy-seeded generators, so the labelled seed runs are not independently reproducible as controlled seeds. The training loop evaluates the test panel repeatedly, the 20-day labels cross nominal split boundaries, and the U.S. and Chinese universes are based on contemporary rather than point-in-time membership. These channels prevent a clean interpretation of the reported test results as untouched out-of-sample estimates.

Common scope. All three studies are evaluated offline. EarnHFT and FineFT replay recorded data without modelling latency, queue position, partial fills, or the agent’s market impact; AlphaQD is a daily signal back-test rather than an execution simulation. The resulting tables compare methods under their respective shared experimental protocols, not live trading performance.

Evaluation protocol. The test periods are short relative to the range of possible market conditions, validation data are reused for several design and routing choices, and some sensitivity grids are evaluated on the test window. FineFT’s AVOL values are reported here on an annualised basis, and its MDD comparison uses the complete final baseline set including RAQR. AlphaQD’s archive statistics follow the plotted snapshot consistently. The theoretical results throughout should consequently be read as motivation under explicit premises, not certification of the deployed systems.

7.4 Future Work

Testing reward separation in trading. A direct follow-up is to compare learned, heuristic, and structural aggregation under a controlled position-trading

experiment. Such a study could test whether keeping set membership out of a specialist’s reward improves target stability; the present three studies do not establish that result.

A unified cross-frequency framework. The three studies share a vocabulary but not one mathematical construction. A useful framework would make the distinctions explicit—market regime, transition subset, or formula behaviour—and test candidate construction and aggregation independently at each frequency.

Richer descriptors and generators. AlphaQD’s behaviour descriptor could be augmented with additional economically meaningful axes, such as factor-style exposure, and its GFlowNet generator could be replaced by a language-model symbolic policy while preserving the archive-independent reward, so the archive organises diversity rather than becoming a moving component of the reward.

Risk and uncertainty as first-class objectives. Future work should separate transition noise, model uncertainty, distributional familiarity, and realised account risk rather than treating VAE reconstruction loss as calibrated competence. Point-in-time calibration and proper scoring tests would be required before such signals are used as deployment risk limits.

Toward deployment. Finally, closing the gap to live trading—modelling latency and market impact, and validating the decoupled-aggregation designs under realistic execution—is the necessary step from these methods to systems that operate in real markets.

Reinforcement learning is a useful language for the sequential trading problems studied here, but historical performance is credible only when data alignment, account mechanics, selection, and implementation are made auditable. The three studies motivate a working hypothesis about separating aggregation from a candidate’s reward when set membership would otherwise move that target. It remains a hypothesis to be tested under controlled and reproducible protocols, not a principle proved by this thesis.

List of Author’s Publications

The following publications and manuscripts arose from the research conducted during this doctoral candidature. The candidate is the first author of each item listed under Conference Proceedings and Manuscript Under Review, and a co-author of the items listed under Other Publications.

Conference Proceedings

- **Molei Qin**^{*}, Shuo Sun^{*}, Wentao Zhang, Haochong Xia, Xinrun Wang[†], and Bo An[†], “EarnHFT: Efficient Hierarchical Reinforcement Learning for High Frequency Trading,” in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024. (^{*}*Equal contribution*; [†]*corresponding authors*.) (Chapter 4)
- **Molei Qin**, Xinyu Cai, Yewen Li, Haochong Xia, Chuqiao Zong, Shuo Sun, Xinrun Wang^{*}, and Bo An, “FineFT: Efficient and Risk-Aware Ensemble Reinforcement Learning for Futures Trading,” in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Volume 1 (KDD)*, pp. 1194–1203, 2026, doi:10.1145/3770854.3780187. (^{*}*Corresponding author*.) (Chapter 5)

Manuscript Under Review

- **Molei Qin**, Meiyang Zhu, Chuqiao Zong, and Bo An, “AlphaQD: Quality-Diversity Reinforcement Learning for Formulaic Alpha Mining,” manuscript currently under review for the *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD’27, Cycle 1)*. (Chapter 6)

Other Publications

- Shuo Sun, **Molei Qin**, Wentao Zhang, Haochong Xia, Chuqiao Zong, Jie Ying, Yonggang Xie, Lingxuan Zhao, Xinrun Wang, and Bo An, “Trade-Master: A Holistic Quantitative Trading Platform Empowered by Reinforcement Learning,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- Shuo Sun, **Molei Qin**, Xinrun Wang, and Bo An, “PRUDEX-Compass: Towards Systematic Evaluation of Reinforcement Learning in Financial Markets,” *Transactions on Machine Learning Research (TMLR)*, 2023.

Bibliography

- [1] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018. [3](#), [31](#)
- [2] John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4):875–889, 2001. [4](#), [23](#)
- [3] Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 673–680, 2006. [4](#), [23](#)
- [4] Shuo Sun, Wanqi Xue, Rundong Wang, Xu He, Junlei Zhu, Jian Li, and Bo An. DeepScalper: A risk-aware reinforcement learning framework to capture fleeting intraday trading opportunities. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 1858–1867, 2022. [4](#), [20](#), [27](#)
- [5] Antonio Briola, Jeremy Turiel, Riccardo Marcaccioli, and Tomaso Aste. Deep reinforcement learning for active high frequency trading. *arXiv preprint arXiv:2101.07107*, 2021. [20](#), [27](#), [40](#), [58](#)
- [6] Tian Zhu and Wei Zhu. Quantitative trading through random perturbation Q-network with nonlinear transaction costs. *Stats*, 5(2):546–560, 2022. [4](#), [20](#), [40](#), [58](#)
- [7] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017. [4](#), [20](#)
- [8] Campbell R. Harvey, Yan Liu, and Heqing Zhu. ... and the cross-section of expected returns. *The Review of Financial Studies*, 29(1):5–68, 2016. [4](#), [23](#), [39](#), [84](#), [85](#)
- [9] Chuheng Zhang, Yitong Duan, Xiaoyu Chen, Jianyu Chen, Jian Li, and Li Zhao. Towards generalizable reinforcement learning for trade execution. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, pages 4975–4983, 2023. doi: 10.24963/ijcai.2023/553. [5](#), [21](#), [27](#)

- [10] Ananth Madhavan. Market microstructure: A survey. *Journal of Financial Markets*, 3(3):205–258, 2000. [12](#)
- [11] Gerald Appel. *Technical Analysis: Power Tools for Active Investors*. FT Press, 2005. [13](#), [20](#), [40](#), [58](#)
- [12] Malik Magdon-Ismail and Amir F. Atiya. Maximum drawdown. *Risk Magazine*, 17(10):99–102, 2004. [14](#), [88](#)
- [13] William F. Sharpe. The Sharpe ratio. *The Journal of Portfolio Management*, 21(1):49–58, 1994. [15](#), [88](#)
- [14] Terry W. Young. Calmar ratio: A smoother tool. *Futures*, 20(1):40, 1991. [15](#)
- [15] Frank A. Sortino and Lee N. Price. Performance measurement in a downside risk framework. *The Journal of Investing*, 3(3):59–64, 1994. doi: 10.3905/joi.3.3.59. [15](#)
- [16] Richard C. Grinold and Ronald N. Kahn. *Active Portfolio Management: A Quantitative Approach for Producing Superior Returns and Controlling Risk*. McGraw-Hill, New York, 2 edition, 2000. [15](#), [74](#), [88](#)
- [17] Zura Kakushadze. 101 formulaic alphas. *Wilmott*, 2016(84):72–81, 2016. [15](#), [22](#), [71](#), [87](#)
- [18] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992. [16](#)
- [19] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. [16](#), [40](#), [58](#)
- [20] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016. [16](#), [39](#)
- [21] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Ian Osband, Gabriel Dulac-Arnold, John Agapiou, Joel Z Leibo, and Audrunas Gruslys. Deep Q-learning from demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3223–3230, 2018. [16](#), [23](#), [32](#)
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. [16](#), [40](#), [58](#)
- [23] Will Dabney, Mark Rowland, Marc Bellemare, and Rémi Munos. Distributional reinforcement learning with quantile regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. [17](#), [59](#)

- [24] Oron Anschel, Nir Baram, and Nahum Shimkin. Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning. In *International Conference on Machine Learning*, pages 176–185. PMLR, 2017. [17](#), [23](#), [59](#)
- [25] Kimin Lee, Michael Laskin, Aravind Srinivas, and Pieter Abbeel. SUNRISE: A simple unified framework for ensemble learning in deep reinforcement learning. In *International Conference on Machine Learning*, pages 6131–6141, 2021. [17](#), [23](#), [59](#)
- [26] Richard S Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2):181–211, 1999. [17](#), [31](#)
- [27] Shubham Pateria, Budhitama Subagdja, Ah-hwee Tan, and Chai Quek. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 54(5):1–35, 2021. [17](#), [31](#)
- [28] Rundong Wang, Hongxin Wei, Bo An, Zhouyan Feng, and Jun Yao. Commission fee is not enough: A hierarchical reinforced framework for portfolio management. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 626–633, 2021. [17](#), [21](#)
- [29] Hui Niu, Siyuan Li, and Jian Li. MetaTrader: An reinforcement learning approach integrating diverse policies for portfolio optimization. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 1573–1583, 2022. [17](#), [21](#)
- [30] Emmanuel Bengio, Moksh Jain, Maksym Korablyov, Doina Precup, and Yoshua Bengio. Flow network based generative models for non-iterative diverse candidate generation. In *Advances in Neural Information Processing Systems*, volume 34, pages 27381–27394, 2021. [17](#), [76](#), [77](#)
- [31] Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in GFlowNets. In *Advances in Neural Information Processing Systems*, volume 35, pages 5955–5967, 2022. [17](#), [76](#)
- [32] Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3: 40, 2016. [18](#), [22](#), [73](#)
- [33] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. arXiv preprint arXiv:1504.04909, 2015. [18](#), [22](#), [73](#)
- [34] Tianping Zhang, Yuanqi Li, Yifei Jin, and Jian Li. AutoAlpha: an efficient hierarchical evolutionary algorithm for mining alpha factors in quantitative investment. arXiv preprint arXiv:2002.08245, 2020. [18](#), [22](#)
- [35] Diederik P Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014. [18](#), [23](#), [56](#)

- [36] Lily Zhang, Mark Goldstein, and Rajesh Ranganath. Understanding failures in out-of-distribution detection with deep generative models. In *International Conference on Machine Learning*, pages 12427–12436. PMLR, 2021. [18](#), [23](#)
- [37] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the second ACM international conference on AI in finance*, pages 1–9, 2021. [19](#)
- [38] Shuo Sun, Molei Qin, Wentao Zhang, Haochong Xia, Chuqiao Zong, Jie Ying, Yonggang Xie, Lingxuan Zhao, Xinrun Wang, and Bo An. TradeMaster: A holistic quantitative trading platform empowered by reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. Datasets and Benchmarks Track. [19](#)
- [39] Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. Qlib: An AI-oriented quantitative investment platform. *arXiv preprint arXiv:2009.11189*, 2020. [19](#)
- [40] Shuo Sun, Molei Qin, Xinrun Wang, and Bo An. PRUDEX-Compass: Towards systematic evaluation of reinforcement learning in financial markets. *Transactions on Machine Learning Research*, 2023. [19](#)
- [41] Zihao Zhang, Stefan Zohren, and Stephen Roberts. Deep reinforcement learning for trading. *The Journal of Financial Data Science*, 2(2):25–40, 2020. doi: 10.3905/jfds.2020.1.030. [20](#)
- [42] Thibaut Théate and Damien Ernst. An application of deep reinforcement learning to algorithmic trading. *Expert Systems with Applications*, 173:114632, 2021.
- [43] Yang Li, Wanshan Zheng, and Zibin Zheng. Deep robust reinforcement learning for practical algorithmic trading. *IEEE Access*, 7:108014–108022, 2019. [20](#)
- [44] Yang Liu, Qi Liu, Hongke Zhao, Zhen Pan, and Chuanren Liu. Adaptive quantitative trading: An imitative deep reinforcement learning approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2128–2135, 2020. [20](#)
- [45] Yi Ding, Weiqing Liu, Jiang Bian, Daoqiang Zhang, and Tie-Yan Liu. Investor-Imitator: A framework for trading knowledge extraction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1310–1319, 2018. [20](#)
- [46] Hong-Gi Shin, Ilkyeun Ra, and Yong-Hoon Choi. A deep multimodal reinforcement learning system combined with CNN and LSTM for stock trading. In *2019 International Conference on Information and Communication Technology Convergence (ICTC)*, pages 7–11, 2019. [20](#)

- [47] Lucas de Azevedo Takara, André Alves Portela Santos, Viviana Cocco Mariani, and Leandro dos Santos Coelho. Deep reinforcement learning applied to a sparse-reward trading environment with intraday data. *Expert Systems with Applications*, 238:121897, 2024. doi: 10.1016/j.eswa.2023.121897. [20](#)
- [48] Weiyu Si, Jinke Li, Peng Ding, and Ruonan Rao. A multi-objective deep reinforcement learning approach for stock index future’s intraday trading. In *2017 10th International Symposium on Computational Intelligence and Design (ISCID)*, volume 2, pages 431–436, 2017. [20](#)
- [49] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997. [20](#)
- [50] Peer Nagy, Jan-Peter Calliess, and Stefan Zohren. Asynchronous deep double duelling Q-learning for trading-signal execution in limit order book markets. *arXiv preprint arXiv:2301.08688*, 2023. [20](#)
- [51] Zhengyao Jiang and Jinjun Liang. Cryptocurrency portfolio management with deep reinforcement learning. In *2017 Intelligent Systems Conference (IntelliSys)*, pages 905–913, 2017. [20](#)
- [52] Zhicheng Wang, Biwei Huang, Shikui Tu, Kun Zhang, and Lei Xu. Deep-Trader: A deep reinforcement learning approach for risk-return balanced portfolio management with market conditions embedding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 643–650, 2021. [20](#)
- [53] Hongyang Yang, Xiao-Yang Liu, Shan Zhong, and Anwar Walid. Deep reinforcement learning for automated stock trading: An ensemble strategy. In *Proceedings of the first ACM international conference on AI in finance*, pages 1–8, 2020. [21](#)
- [54] Yuan Gao, Ziming Gao, Yi Hu, Sifan Song, Zhengyong Jiang, and Jionglong Su. A framework of hierarchical deep Q-network for portfolio management. In *ICAART (2)*, pages 132–140, 2021. [21](#)
- [55] Lijun Zha, Le Dai, Tong Xu, and Di Wu. A hierarchical reinforcement learning framework for stock selection and portfolio. In *2022 International Joint Conference on Neural Networks (IJCNN)*, pages 1–7, 2022. [21](#)
- [56] Wentao Zhang, Yilei Zhao, Shuo Sun, Jie Ying, Yonggang Xie, Zitao Song, Xinrun Wang, and Bo An. Reinforcement learning with maskable stock representation for portfolio management in customizable stock pools. In *Proceedings of the ACM Web Conference 2024*, pages 187–198, 2024. [21](#)
- [57] Jingyi Gu, Wenlu Du, AM Muntasir Rahman, and Guiling Wang. Margin Trader: A reinforcement learning framework for portfolio management with margin and constraints. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 610–618, 2023. [21](#)

- [58] Hui Niu, Siyuan Li, Jiahao Zheng, Zhouchi Lin, Bo An, Jian Li, and Jian Guo. IMM: An imitative reinforcement learning approach with predictive representation learning for automatic market making. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 5999–6007, 2024. [21](#)
- [59] Shuo Sun, Xinrun Wang, Wanqi Xue, Xiaoxuan Lou, and Bo An. Mastering stock markets with efficient mixture of diversified trading experts. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2109–2119, 2023. doi: 10.1145/3580305.3599424. [21](#), [37](#), [51](#)
- [60] Nick Littlestone. Learning quickly when irrelevant attributes abound: A new linear-threshold algorithm. *Machine Learning*, 2(4):285–318, 1988. [21](#), [59](#)
- [61] Molei Qin, Shuo Sun, Wentao Zhang, Haochong Xia, Xinrun Wang, and Bo An. EarnHFT: Efficient hierarchical reinforcement learning for high frequency trading. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 14669–14676, 2024. [21](#), [28](#), [47](#), [51](#), [52](#), [59](#)
- [62] Chuqiao Zong, Chaojie Wang, Molei Qin, Lei Feng, Xinrun Wang, and Bo An. MacroHFT: Memory augmented context-aware reinforcement learning on high frequency trading. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4712–4721, 2024. doi: 10.1145/3637528.3672064. [21](#), [47](#), [59](#)
- [63] Trevor Stephens. gplearn: Genetic programming in Python, with a scikit-learn inspired API. <https://github.com/trevorstephens/gplearn>, 2015. Accessed May 2026. [22](#), [87](#)
- [64] Weizhe Ren, Yichen Qin, and Yang Li. Alpha mining and enhancing via warm start genetic programming for quantitative investment. arXiv preprint arXiv:2412.00896, 2024. [22](#)
- [65] Can Cui, Wei Wang, Meihui Zhang, Gang Chen, Zhaojing Luo, and Beng Chin Ooi. AlphaEvolve: A learning framework to discover novel alphas in quantitative investment. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2208–2216, New York, NY, USA, 2021. Association for Computing Machinery. [22](#)
- [66] Brenden K Petersen, Mikel Landajuela, T Nathan Mundhenk, Claudio P Santiago, Soo Kim, and Joanne T Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *International Conference on Learning Representations*, pages 1–24, Online, 2021. OpenReview.net. [22](#)
- [67] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. Generating synergistic formulaic alpha collections via reinforcement learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5476–5486, New York, NY, USA, 2023.

- Association for Computing Machinery. doi: 10.1145/3580305.3599831. [22](#), [72](#), [87](#)
- [68] Zhoufan Zhu and Ke Zhu. AlphaQCM: Alpha discovery in finance with distributional reinforcement learning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 80463–80479, Vancouver, Canada, 2025. PMLR. [22](#), [87](#)
- [69] Tao Ren, Ruihan Zhou, Jinyang Jiang, Jiafeng Liang, Qinghao Wang, and Yijie Peng. RiskMiner: Discovering formulaic alphas via risk seeking Monte Carlo tree search. In *Proceedings of the 5th ACM International Conference on AI in Finance*, pages 752–760, New York, NY, USA, 2024. Association for Computing Machinery. [22](#)
- [70] Feng Xu, Yan Yin, Xinyu Zhang, Tianyuan Liu, Shengyi Jiang, and Zongzhang Zhang. Alpha²: Discovering logical formulaic alphas using deep reinforcement learning. arXiv preprint arXiv:2406.16505, 2024. [22](#)
- [71] Hao Shi, Weili Song, Xinting Zhang, Jiahe Shi, Cuicui Luo, Xiang Ao, Hamid Arian, and Luis Angel Seco. AlphaForge: A framework to mine and dynamically combine formulaic alpha factors. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 12524–12532, Washington, DC, USA, 2025. AAAI Press. [22](#), [87](#)
- [72] Binqi Chen, Hongjun Ding, Ning Shen, Jinsheng Huang, Taian Guo, Luchen Liu, and Ming Zhang. AlphaSAGE: Structure-aware alpha mining via GFlowNets for robust exploration. arXiv preprint arXiv:2509.25055, 2025. [22](#), [87](#)
- [73] Saizhuo Wang, Hang Yuan, Leon Zhou, Lionel Ni, Heung-Yeung Shum, and Jian Guo. Alpha-GPT: Human-AI interactive alpha mining for quantitative investment. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 196–206, Suzhou, China, 2025. Association for Computational Linguistics. [22](#)
- [74] Ziyi Tang, Zechuan Chen, Jiarui Yang, Jiayao Mai, Yongsen Zheng, Keze Wang, Jinrui Chen, and Liang Lin. AlphaAgent: LLM-driven alpha mining with regularized exploration to counteract alpha decay. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2813–2822, New York, NY, USA, 2025. Association for Computing Machinery. doi: 10.1145/3711896.3736838.
- [75] Zhiwei Li, Ran Song, Caihong Sun, Wei Xu, Zhengtao Yu, and Ji-Rong Wen. Can large language models mine interpretable financial factors more effectively? a neural-symbolic factor mining agent model. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3891–3902, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.233. [22](#)

- [76] Julian Bernhard, Stefan Pollok, and Alois Knoll. Addressing inherent uncertainty: Risk-sensitive behavior generation for automated driving using distributional reinforcement learning. In *2019 IEEE Intelligent Vehicles Symposium (IV)*, pages 2148–2155, 2019. [23](#)
- [77] Annie Xie, James Harrison, and Chelsea Finn. Deep reinforcement learning amidst lifelong non-stationarity. *arXiv preprint arXiv:2006.10701*, 2020. [23](#), [50](#)
- [78] Jonathan Brogaard, Terrence Hendershott, and Ryan Riordan. High-Frequency Trading and Price Discovery. *The Review of Financial Studies*, 27(8):2267–2306, 06 2014. ISSN 0893-9454. doi: 10.1093/rfs/hhu032. URL <https://doi.org/10.1093/rfs/hhu032>. [27](#)
- [79] José Almeida and Tiago Cruz Gonçalves. A systematic literature review of investor behavior in the cryptocurrency markets. *Journal of Behavioral and Experimental Finance*, 37:100785, 2023. doi: 10.1016/j.jbef.2022.100785. [27](#)
- [80] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013. [27](#)
- [81] Tommi Jaakkola, Michael Jordan, and Satinder Singh. Convergence of stochastic iterative dynamic programming algorithms. In *Advances in Neural Information Processing Systems*, volume 6, 1993. [36](#)
- [82] Bernard W. Silverman. *Density Estimation for Statistics and Data Analysis*. Chapman & Hall, London, 1986. [37](#)
- [83] Hiroaki Sakoe and Seibi Chiba. Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 26(1):43–49, 1978. [37](#)
- [84] Donald J. Berndt and James Clifford. Using dynamic time warping to find patterns in time series. In *Proceedings of the AAAI-94 Workshop on Knowledge Discovery in Databases*, pages 359–370, 1994. [37](#)
- [85] Tarun Chordia, Richard Roll, and Avanidhar Subrahmanyam. Order imbalance, liquidity, and market returns. *Journal of Financial Economics*, 65(1): 111–130, 2002. [40](#), [58](#)
- [86] Charline Le Lan, Stephen Tu, Adam Oberman, Rishabh Agarwal, and Marc G Bellemare. On the generalization of representations in reinforcement learning. In *International Conference on Artificial Intelligence and Statistics*, pages 4132–4157. PMLR, 2022. [48](#)
- [87] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(11), 2008. [48](#)

-
- [88] Peter J Huber. Robust estimation of a location parameter. In *Breakthroughs in Statistics: Methodology and Distribution*, pages 492–518. Springer, 1992. [52](#)
 - [89] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European Semantic Web Conference*, pages 593–607, Cham, Switzerland, 2018. Springer. [76](#)
 - [90] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. [79](#)
 - [91] Fionn Murtagh. Multilayer perceptrons for classification and regression. *Neurocomputing*, 2(5–6):183–197, 1991. [87](#)
 - [92] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. [87](#)
 - [93] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017. [87](#)